Full Stack Developer Interview Mastery

JavaScript • React • Node.js • System Design • Testing • Behavioral

400+ Curated Questions | From Basics to Internals → Real Interview Flow

"Don't memorize answers — Understand how systems behave."

From writing code → to thinking like an engineer.

- ✓ Covers Real Interview Questions
- ✓ Structured Learning Path
- ✓ Fast Revision Friendly
- ✓ Industry-Level Concepts

Designed for 0 → 3+ Years *Developers* aiming to crack top product companies

By Kaushal Singh | *Full Stack Developer* | *MERN • React Native*

Part 1
JavaScript
140 Questions — Basics to Internals

■ Basics (Q1–Q20)

1 What is JavaScript?

JavaScript is a lightweight, interpreted, high-level programming language primarily used for web development. It is single-threaded, dynamically typed, and supports event-driven, functional, and object-oriented programming paradigms. JS runs in browsers (via engines like V8, SpiderMonkey) and on servers via Node.js.

■ It is the only language natively supported by all browsers.

2 Difference between var, let, const?

var is function-scoped, hoisted (initialised as undefined), and can be redeclared. let is block-scoped, hoisted but NOT initialised (Temporal Dead Zone), cannot be redeclared. const is block-scoped, hoisted but NOT initialised, must be initialised at declaration, and cannot be reassigned (though object properties can change).

Example / Code:

```
var x = 1; let y = 2; const Z = 3; { var x = 10; }console.log(x); // 10 (leaked) {
let y = 20; } console.log(y); // 2 (block scoped)
```

■ Prefer const by default, use let when re-assignment is needed, avoid var.

3 What is hoisting?

Hoisting is JavaScript's behaviour of moving declarations to the top of their scope before execution. var declarations are hoisted and initialised with undefined. Function declarations are fully hoisted (name + body). let/const are hoisted but remain in the Temporal Dead Zone until the declaration line.

Example / Code:

```
console.log(a); // undefined (hoisted) var a = 5; console.log(b); //
ReferenceError (TDZ) let b = 10;
```

4 What is Temporal Dead Zone (TDZ)?

TDZ is the period between entering a block scope and the actual declaration of a let/const variable. Accessing the variable in this zone throws a ReferenceError. It exists to catch usage-before-declaration bugs.

Example / Code:

```
{ console.log(x); // ReferenceError (TDZ) let x = 5; console.log(x); // 5 }
```

5 What are data types in JS?

Primitive: string, number, bigint, boolean, undefined, null, symbol (7 types). Reference/Object: object, array, function, date, map, set, etc. Primitives are immutable and stored by value; objects are stored by reference.

Example / Code:

```
typeof 'hello' // 'string' typeof 42 // 'number' typeof true // 'boolean' typeof
undefined// 'undefined' typeof null // 'object' (famous bug) typeof {} //
'object' typeof [] // 'object'
```

6 Difference between null and undefined?

undefined means a variable has been declared but not assigned a value. null is an intentional absence of value — explicitly set by the programmer. typeof undefined === 'undefined'; typeof null === 'object' (legacy bug).

Example / Code:

```
let a; // undefined let b = null; // null console.log(a == b); // true (loose)
console.log(a === b); // false (strict)
```

7 What is NaN?

NaN (Not a Number) is a special numeric value that represents an invalid or unrepresentable mathematical result. It is of type 'number'. NaN is the only value in JS that is not equal to itself.

Example / Code:

```
console.log(0/0); // NaN console.log('abc' * 2); // NaN console.log(NaN === NaN);
// false console.log(Number.isNaN(NaN)); // true
```

■ Always use Number.isNaN() instead of isNaN() which coerces the argument first.

8 typeof NaN returns what?

typeof NaN returns 'number'. This is because NaN is part of the IEEE 754 floating-point specification and is technically a numeric value that represents an invalid number.

Example / Code:

```
console.log(typeof NaN); // 'number'
```

9 What is type coercion?

Type coercion is JavaScript's automatic conversion of values from one type to another. Implicit coercion happens automatically (e.g., '5' + 3 = '53'); explicit coercion uses Number(), String(), Boolean() etc.

Example / Code:

```
console.log('5' + 3); // '53' (string concat) console.log('5' - 3); // 2 (numeric
sub) console.log('' == false); // true (coercion) console.log(Number('5')); // 5
(explicit)
```

10 == vs ===?

== is loose equality — it performs type coercion before comparing. === is strict equality — it compares both value AND type without coercion. Always prefer === to avoid unexpected bugs.

Example / Code:

```
console.log(0 == '0'); // true (coerced) console.log(0 === '0'); // false
(different types) console.log(null == undefined); // true console.log(null ===
undefined); // false
```

11 What is truthy and falsy?

Falsy values are those that evaluate to false in a boolean context: false, 0, -0, 0n, '', null, undefined, NaN. Everything else is truthy. This matters in if-conditions, logical operators, and ternary expressions.

Example / Code:

```
if ('') console.log('no'); // falsy - skipped if ('hello') console.log('yes'); //
truthy if (0) {} // falsy if ([]) {} // truthy (even empty array!)
```

■ Empty array [] and empty object {} are TRUTHY.

12 What are primitive types?

Primitives are immutable data types stored directly on the stack: string, number, bigint, boolean, undefined, null, symbol. They are copied by value when assigned. Operations on primitives return new values; they never mutate in place.

Example / Code:

```
let a = 'hello'; let b = a; // copy b = 'world'; console.log(a); // 'hello'
(unchanged)
```

13 What are reference types?

Reference types (objects, arrays, functions) are stored on the heap; variables hold a reference (pointer) to the location. Assignment copies the reference, not the value — so two variables can point to the same object.

Example / Code:

```
let obj1 = { x: 1 }; let obj2 = obj1; // same ref obj2.x = 99;
console.log(obj1.x); // 99 (both changed)
```

14 What is a function?

A function is a reusable block of code that performs a specific task. In JS, functions are first-class citizens — they can be assigned to variables, passed as arguments, and returned from other functions.

Example / Code:

```
function greet(name) { return 'Hello, ' + name; } console.log(greet('Alice')); //
Hello, Alice
```

15 Function declaration vs expression?

Declaration: uses the 'function' keyword at the statement level — fully hoisted. Expression: a function assigned to a variable — NOT hoisted (the variable is hoisted but as undefined). Arrow functions are always expressions.

Example / Code:

```
// Declaration (hoisted) console.log(add(2,3)); // 5 function add(a,b){ return
a+b; } // Expression (not hoisted) console.log(mul(2,3)); // TypeError const mul
= function(a,b){ return a*b; };
```

16 What is an arrow function?

Arrow functions are a concise ES6 syntax for functions using =>. They do NOT have their own 'this', 'arguments', 'super', or 'new.target'. They cannot be used as constructors.

Example / Code:

```
const add = (a, b) => a + b; const square = x => x * x; const greet = () =>
'Hello'; console.log(add(2, 3)); // 5
```

17 Difference between arrow and normal function?

1) this: Arrow inherits from enclosing scope; normal has its own this. 2) arguments: Arrow has none; normal has arguments object. 3) Constructor: Arrow cannot be used with new. 4) Hoisting: Normal declarations are hoisted; arrow expressions are not. 5) prototype: Arrow functions have no prototype property.

Example / Code:

```
function Foo() { this.x = 1; } const Bar = () => {}; new Foo(); // OK new Bar();
// TypeError: Bar is not a constructor
```

18 What is IIFE?

Immediately Invoked Function Expression — a function that is defined and called immediately. Used to create a private scope, avoid polluting the global namespace, and encapsulate logic. Common pattern before ES6 modules.

Example / Code:

```
(function() { var secret = 42; console.log(secret); // 42 })();
console.log(typeof secret); // 'undefined'
```

19 What is scope?

Scope determines where variables are accessible. JS has lexical scoping — scope is determined at write-time, not runtime. Variables are looked up by the engine by traversing from the innermost scope outward until found.

Example / Code:

```
let global = 'G'; function outer() { let out = 'O'; function inner() { let inn =
'I'; console.log(global, out, inn); // G O I } inner(); }
```

20 Types of scope?

1) Global scope — accessible everywhere. 2) Function scope — inside a function (var). 3) Block scope — inside {} (let, const). 4) Module scope — variables in ES modules are scoped to the module. 5) Lexical scope — functions remember where they were defined.

Example / Code:

```
var g = 'global'; function fn() { var f = 'function'; if(true) { let b = 'block';
console.log(b); // 'block' } // console.log(b); // ReferenceError }
```

■ Advanced Core (Q21–Q50)

21 What is closure?

A closure is a function that remembers and accesses variables from its outer (enclosing) scope even after that outer function has finished executing. Closures are created every time a function is created.

Example / Code:

```
function counter() { let count = 0; return function() { count++; return count; };
} const inc = counter(); console.log(inc()); // 1 console.log(inc()); // 2
```

■ Closures are the basis for data privacy, module pattern, memoization.

22 Practical use of closures?

1) Data privacy / encapsulation. 2) Function factories. 3) Memoization. 4) Partial application / currying. 5) Event handlers that remember state. 6) Module pattern (before ES6 modules).

Example / Code:

```
// Factory example function multiplier(x) { return (y) => x * y; } const double =
multiplier(2); console.log(double(5)); // 10 console.log(double(9)); // 18
```

23 What is lexical scope?

Lexical scope means a function's scope is determined by where it is defined in the source code, not where it is called. Inner functions have access to variables of outer functions due to the scope chain formed at parse time.

Example / Code:

```
function outer() { let x = 10; function inner() { console.log(x); // 10 - lexical
scope } return inner; } const fn = outer(); fn(); // 10 (even after outer
finished)
```

24 What is execution context?

An Execution Context is the environment in which JS code runs. It contains: Variable Object (variables, functions), Scope Chain, and 'this'. Types: Global EC (created on start), Function EC (per function call), Eval EC (rare).

Example / Code:

```
// Each function call creates a new EC function A() { let a = 1; function B() {
let b = 2; // EC has: b, scope chain -> A's EC -> global EC } B(); }
```

25 What is call stack?

The call stack is a LIFO (Last In First Out) data structure that tracks active execution contexts. When a function is called, its EC is pushed; when it returns, it's popped. Stack overflow happens when recursion is too deep.

Example / Code:

```
function A() { B(); } function B() { C(); } function C() { console.log('C'); }
A(); // Stack: [Global, A, B, C] -> C pops -> B pops -> A pops
```

26 What is event loop?

The event loop continuously monitors the call stack and callback queue. When the call stack is empty, it takes the first callback from the queue and pushes it onto the stack. This allows asynchronous operations (setTimeout, I/O) in a single-threaded environment.

Example / Code:

```
console.log('1'); setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3')); console.log('4'); // Output: 1,
4, 3, 2 // (microtasks before macrotasks)
```

- Microtask queue (Promises) is drained before the next macrotask (setTimeout).

27 Microtask vs macrotask?

Microtasks (Promise callbacks, queueMicrotask, MutationObserver) run after the current task and before any macrotask. Macrotasks (setTimeout, setInterval, setImmediate, I/O) are processed one per event loop tick. All microtasks are flushed before the next macrotask.

Example / Code:

```
setTimeout(() => console.log('macro'), 0); Promise.resolve().then(() =>
console.log('micro')); // Output: micro, macro
```

28 setTimeout vs setImmediate?

setTimeout(fn, 0) schedules after the minimum delay (1ms+), in the timers phase of the event loop. setImmediate (Node.js only) schedules after I/O events, in the check phase. In the top-level script the order is non-deterministic; inside I/O callbacks setImmediate always fires first.

Example / Code:

```
// Inside I/O callback (Node.js): fs.readFile('f', () => { setTimeout(() =>
console.log('timeout'), 0); setImmediate(() => console.log('immediate')); }); //
Always: immediate, timeout
```

29 What is callback?

A callback is a function passed as an argument to another function, to be executed later (asynchronously or after some task). It was the original pattern for async code in JavaScript before Promises and async/await.

Example / Code:

```
function fetchData(url, callback) { // simulate async setTimeout(() =>
callback(null, {data: 'result'}), 1000); } fetchData('/api', (err, data) => {
if(err) return console.error(err); console.log(data); });
```

30 What is callback hell?

Callback hell (pyramid of doom) is deeply nested callbacks that make code hard to read, debug, and maintain. Caused by multiple sequential async operations using callbacks. Solved by Promises, async/await, or named functions.

Example / Code:

```
getUser(id, (user) => { getPosts(user, (posts) => { getComments(posts[0],
(comments) => { // deeply nested - hard to maintain }); }); });
```

- Use async/await to flatten this into readable sequential code.

31 What is a promise?

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: pending, fulfilled, rejected. Created with new Promise((resolve, reject) => {}). Enables chaining with .then()/.catch()/.finally().

Example / Code:

```
const p = new Promise((resolve, reject) => { setTimeout(() => resolve('done'),
1000); }); p.then(val => console.log(val)) // 'done' .catch(err =>
console.error(err));
```

32 Promise states?

1) Pending — initial state, neither fulfilled nor rejected. 2) Fulfilled — operation completed successfully, resolve() was called. 3) Rejected — operation failed, reject() was called. Once settled (fulfilled or rejected) a promise is immutable.

Example / Code:

```
let p = new Promise(res => setTimeout(res, 1000)); console.log(p); // Promise {
<pending> } setTimeout(() => console.log(p), 1100); // Promise { 'undefined' }
(fulfilled)
```

33 async/await?

async/await is syntactic sugar over Promises. An async function always returns a Promise. await pauses execution inside the async function until the Promise resolves, making async code look synchronous. Errors are caught with try/catch.

Example / Code:

```
async function fetchUser(id) { try { const res = await fetch(`/api/users/${id}`);
const user = await res.json(); return user; } catch(err) {
console.error('Error:', err); } }
```

34 Error handling in async/await?

Use try/catch blocks inside async functions to handle rejected Promises. Alternatively, chain .catch() on the async function call. Unhandled Promise rejections should be caught at the top level with process.on('unhandledRejection') in Node.

Example / Code:

```
async function getData() { try { const data = await riskyOperation(); return
data; } catch(error) { console.error('Caught:', error.message); throw error; //
re-throw if needed } finally { console.log('Always runs'); } }
```

35 What is 'this' keyword?

'this' refers to the context in which a function is called. Its value depends on HOW the function is called: 1) Global context: window (non-strict) or undefined (strict). 2) Method call: the object. 3) Constructor: the new object. 4) Arrow: inherits from enclosing scope. 5) call/apply/bind: explicit this.

Example / Code:

```
const obj = { name: 'Alice', greet() { return 'Hi ' + this.name; }, greetArrow: ()
=> 'Hi ' + this.name // this = global! }; console.log(obj.greet()); // Hi Alice
console.log(obj.greetArrow()); // Hi undefined
```

36 this in arrow vs normal function?

Normal functions get 'this' dynamically based on how they are called. Arrow functions do NOT have their own 'this' — they capture 'this' from the surrounding lexical context at definition time. This makes arrow functions ideal for callbacks inside methods.

Example / Code:

```
function Timer() { this.ticks = 0; // Arrow captures Timer's 'this'
setInterval(() => this.ticks++, 1000); } const t = new Timer(); // works
correctly
```

37 bind vs call vs apply?

call(thisArg, arg1, arg2) — invokes the function immediately with given this and individual arguments. apply(thisArg, [args]) — same as call but arguments as array. bind(thisArg, arg1) — returns a NEW function with this permanently bound; doesn't call immediately.

Example / Code:

```
function greet(greeting, punct) { return `${greeting}, ${this.name}${punct}`; }
const obj = { name: 'Bob' }; greet.call(obj, 'Hello', '!'); // Hello, Bob!
greet.apply(obj, ['Hi', '.']); // Hi, Bob. const fn = greet.bind(obj, 'Hey');
fn('?'); // Hey, Bob?
```

38 What is prototype?

Every JavaScript object has an internal [[Prototype]] property pointing to another object (its prototype). Property lookup walks the prototype chain — if a property isn't found on the object, JS looks on its prototype, and so on until null.

Example / Code:

```
function Animal(name) { this.name = name; } Animal.prototype.speak = function() {
return this.name + ' makes a sound'; }; const dog = new Animal('Dog');
console.log(dog.speak()); // Dog makes a sound
console.log(dog.hasOwnProperty('speak')); // false
```

39 Prototype chain?

The prototype chain is the series of linked prototypes from an object up to Object.prototype (whose prototype is null). This is how inheritance works in JS. Object.getPrototypeOf() retrieves the prototype of an object.

Example / Code:

```
const arr = [1, 2, 3]; // arr -> Array.prototype -> Object.prototype -> null
console.log(arr.__proto__ === Array.prototype); // true
console.log(Array.prototype.__proto__ === Object.prototype); // true
```

40 What is inheritance in JS?

JS uses prototypal inheritance — objects inherit directly from other objects via the prototype chain. ES6 classes provide syntactic sugar over this. With classes, 'extends' sets up the prototype chain, and 'super' calls the parent constructor.

Example / Code:

```
class Animal { constructor(name) { this.name = name; } speak() { return
`${this.name} speaks`; } } class Dog extends Animal { speak() { return
super.speak() + ' (woof)'; } } const d = new Dog('Rex'); console.log(d.speak());
// Rex speaks (woof)
```

41 Object.create vs class?

Object.create(proto) creates a new object with the specified prototype — direct prototypal inheritance, no constructor needed. Class syntax is more familiar OOP style, uses constructors, supports extends and super. Both use prototype chains under the hood.

Example / Code:

```
// Object.create const animal = { speak() { return 'generic sound'; } }; const dog
= Object.create(animal); dog.name = 'Rex'; console.log(dog.speak()); // generic
sound
```

42 What is class in JS?

ES6 classes are syntactic sugar over constructor functions and prototype chains. They support constructor, methods, static methods, inheritance (extends), and private fields (#). They are NOT hoisted like function declarations.

Example / Code:

```
class Person { #age; // private field constructor(name, age) { this.name = name;
this.#age = age; } getAge() { return this.#age; } static create(n, a) { return new
Person(n, a); } } const p = Person.create('Alice', 30); console.log(p.getAge());
// 30
```

43 Constructor function?

A constructor function is a regular function called with 'new'. When called with new: 1) A new empty object is created. 2) 'this' is set to that object. 3) The prototype is linked. 4) The object is returned (unless an object is explicitly returned).

Example / Code:

```
function Person(name) { this.name = name; this.greet = function() { return 'Hi, I am ' + this.name; }; } const p = new Person('Alice'); console.log(p.greet()); // Hi, I am Alice
```

44 What is the new keyword?

The new keyword: 1) Creates a blank object. 2) Sets its [[Prototype]] to the constructor's .prototype. 3) Executes the constructor with 'this' = new object. 4) Returns the object (or the explicitly returned object if it's an object).

Example / Code:

```
function Car(model) { this.model = model; } Car.prototype.drive = function() { return 'Driving ' + this.model; }; const c = new Car('Tesla'); console.log(c.drive()); // Driving Tesla console.log(c instanceof Car); // true
```

45 What is shallow copy?

A shallow copy copies the top-level properties. Nested objects/arrays still reference the same memory — modifying them in the copy affects the original. Created via Object.assign(), spread operator {...obj}, or Array.from().

Example / Code:

```
const original = { a: 1, nested: { b: 2 } }; const copy = { ...original }; copy.a = 99; // does NOT affect original copy.nested.b = 99; // DOES affect original! console.log(original.nested.b); // 99
```

46 What is deep copy?

A deep copy creates completely independent copies of all nested values. Methods: 1) JSON.parse(JSON.stringify(obj)) — simple but loses functions, dates, undefined, circular refs. 2) structuredClone() — modern, handles most types. 3) lodash _.cloneDeep() — most complete.

Example / Code:

```
const original = { a: 1, nested: { b: 2 } }; const deep = structuredClone(original); deep.nested.b = 99; console.log(original.nested.b); // 2 (unaffected)
```

47 JSON.parse/stringify limitations?

Limitations: 1) Functions are stripped. 2) undefined values are removed. 3) Date objects become strings. 4) Circular references throw TypeError. 5) Symbols are ignored. 6) BigInt throws. 7) Prototype chain is not preserved.

Example / Code:

```
const obj = { fn: () => 42, date: new Date(), undef: undefined }; const copy = JSON.parse(JSON.stringify(obj)); console.log(copy); // { date: '2024-01-01T...' } fn & undef removed
```

48 structuredClone?

structuredClone() is a modern browser/Node.js API (Node 17+) for deep cloning. It handles: circular references, Date, RegExp, Map, Set, ArrayBuffer, Blob, ImageBitmap. It does NOT handle: functions, DOM nodes, class instances with methods.

Example / Code:

```
const map = new Map([['key', { nested: 1 }]]); const clone = structuredClone(map); clone.get('key').nested = 99; console.log(map.get('key').nested); // 1 (unaffected)
```

49 What is immutability?

Immutability means a value cannot be changed after it is created. Primitives are inherently immutable. Objects can be made immutable with Object.freeze() (shallow) or deep-freeze techniques. Immutability prevents accidental mutations and makes code predictable.

Example / Code:

```
const obj = Object.freeze({ x: 1, y: 2 }); obj.x = 99; // silently fails (strict mode throws) console.log(obj.x); // 1
```

50 Why is immutability important?

1) Predictability — no unexpected mutations. 2) Easier debugging — trace changes clearly. 3) Pure functions — no side effects. 4) React performance — shallow comparison works for change detection. 5) Concurrent programming safety. 6) Time-travel debugging (Redux).

Example / Code:

```
// React: shallow comparison const prev = { count: 1 }; const next = { ...prev, count: 2 }; console.log(prev === next); // false -> React knows to re-render
```

■ ES6+ Features (Q51–Q80)

51 What is destructuring?

Destructuring allows unpacking values from arrays or properties from objects into distinct variables using concise syntax. Supports default values, renaming, and nested destructuring.

Example / Code:

```
// Object destructuring const { name, age = 25 } = { name: 'Alice' }; console.log(name, age); // Alice 25 // Array destructuring const [first, , third] = [1, 2, 3]; console.log(first, third); // 1 3
```

52 Array vs object destructuring?

Array destructuring uses position — order matters, uses []. Object destructuring uses property names — order doesn't matter, uses {}. Both support defaults, rest patterns, and nested destructuring.

Example / Code:

```
// Array - position based const [a, b] = [10, 20]; // Object - name based const { x: myX, y } = { x: 1, y: 2 }; console.log(myX); // 1 (renamed)
```

53 Spread vs rest operator?

Both use ... syntax but in different contexts. Spread expands an iterable into individual elements (in function calls or array/object literals). Rest collects remaining elements into an array/object (in function parameters or destructuring).

Example / Code:

```
// Spread const arr = [1, 2, 3]; console.log(Math.max(...arr)); // 3 const merged = { ...obj1, ...obj2 }; // Rest function sum(first, ...rest) { return first + rest.reduce((a, b) => a + b, 0); } sum(1, 2, 3, 4); // 10
```

54 Template literals?

Template literals use backticks (`) and allow: 1) Multi-line strings. 2) Expression interpolation with ${expr}. 3) Tagged templates (for libraries like styled-components or SQL queries).

Example / Code:

```
const name = 'World'; const msg = `Hello, ${name}!\nToday is ${new Date().toDateString()}`; // Tagged template function tag(strings, ...vals) { return strings.raw[0] + vals[0].toUpperCase(); } console.log(tag`Hello ${name}`); // Hello WORLD
```

55 Default parameters?

Default parameters allow function parameters to have fallback values if no argument (or undefined) is provided. They are evaluated at call time, not at definition time — enabling dynamic defaults.

Example / Code:

```
function greet(name = 'Guest', greeting = 'Hello') { return `${greeting}, ${name}!`; } console.log(greet()); // Hello, Guest! console.log(greet('Alice', 'Hi')); // Hi, Alice!
```

56 Optional chaining (?.) ?

Optional chaining ?. safely accesses nested object properties without throwing if an intermediate value is null or undefined. Returns undefined instead of throwing TypeError. Works with methods ?.() and bracket notation ?.[].

Example / Code:

```
const user = { profile: { name: 'Alice' } }; console.log(user?.profile?.name); //
'Alice' console.log(user?.address?.city); // undefined (no error)
console.log(user?.getName?.()); // undefined (no error)
```

57 Nullish coalescing (??)?

The ?? operator returns the right-hand side only when the left-hand side is null or undefined. Unlike ||, it does NOT trigger on falsy values like 0, '', or false. Use ?? for default values that should allow 0 and empty strings.

Example / Code:

```
const count = 0; console.log(count || 10); // 10 (wrong!) console.log(count ??
10); // 0 (correct!) const name = null; console.log(name ?? 'Guest'); // 'Guest'
```

58 Modules in JS?

ES6 modules (ESM) provide a standard for splitting code into files. Each module has its own scope. Use export to expose values and import to consume them. Modules are statically analyzed (tree-shaking possible) and always in strict mode.

Example / Code:

```
// math.js export const add = (a, b) => a + b; export default function multiply(a,
b) { return a * b; } // app.js import multiply, { add } from './math.js';
console.log(add(2, 3)); // 5 console.log(multiply(2, 3)); // 6
```

59 import vs require?

import is ES6 (ESM) — static, analyzed at parse time, supports tree-shaking, always async in browsers, cannot be inside conditionals. require is CommonJS (Node.js) — dynamic, synchronous, evaluated at runtime, can be inside conditionals.

Example / Code:

```
// CommonJS const fs = require('fs'); if(condition) { const mod =
require('./mod'); } // ESM import fs from 'fs'; // at top level only
import('./mod').then(m => {}); // dynamic import
```

60 Dynamic import?

import() is a function-like syntax that loads a module asynchronously and returns a Promise. Used for code splitting, conditional loading, and lazy loading. Supported in both browser and Node.js environments.

Example / Code:

```
// Lazy load a heavy module async function loadChart() { const { Chart } = await
import('./chart.js'); return new Chart(); } // Conditional if (feature) { const
mod = await import('./feature.js'); }
```

61 Map vs Object?

Map: keys can be ANY type, insertion order is preserved, has size property, better for frequent additions/deletions, iterable directly. Object: keys are strings/symbols only, has prototype properties (can clash), better for JSON, structured data, and static configs.

Example / Code:

```
const map = new Map(); map.set(42, 'number key'); map.set({}, 'object key');
console.log(map.size); // 2 console.log(map.get(42)); // 'number key' for(const
[k, v] of map) console.log(k, v);
```

62 Set vs Array?

Set stores UNIQUE values only — duplicates are automatically removed. Has O(1) lookup with .has(). Array allows duplicates, has O(n) indexOf. Use Set for unique collections and fast membership checks; use Array for ordered lists with duplicates.

Example / Code:

```
const set = new Set([1, 2, 2, 3, 3]); console.log([...set]); // [1, 2, 3]
console.log(set.has(2)); // true // Fast deduplication const unique = [...new
Set(array)];
```

63 WeakMap?

WeakMap is like Map but keys must be objects and are held weakly — if no other reference to the key object exists, it can be garbage collected. Has no size, no iteration. Used for private data, caching, and avoiding memory leaks.

Example / Code:

```
let obj = {}; const cache = new WeakMap(); cache.set(obj, 'cached data');
console.log(cache.get(obj)); // 'cached data' obj = null; // obj can now be GC'd;
cache entry removed
```

64 WeakSet?

WeakSet stores weakly referenced objects only (no primitives). No duplicates, no iteration, no size. Used to track object membership without preventing garbage collection — e.g., tracking 'seen' objects during traversal.

Example / Code:

```
const seen = new WeakSet(); function process(obj) { if(seen.has(obj)) return
'already processed'; seen.add(obj); return 'processed'; }
```

65 Symbol?

Symbol is a primitive type that creates unique, immutable identifiers. Every Symbol() call creates a completely unique value — even with the same description. Used for: unique property keys, well-known symbols (Symbol.iterator), private-like properties.

Example / Code:

```
const id = Symbol('id'); const obj = { [id]: 123, name: 'Alice' };
console.log(obj[id]); // 123 console.log(Object.keys(obj)); // ['name'] (symbols
hidden) console.log(Symbol('x') === Symbol('x')); // false
```

66 Iterators?

An iterator is an object with a next() method that returns { value, done }. done is true when iteration is complete. Any object implementing the iterator protocol can be iterated with for...of, spread, destructuring.

Example / Code:

```
function range(start, end) { return { [Symbol.iterator]() { let cur = start;
return { next() { return cur <= end ? { value: cur++, done: false } : { value:
undefined, done: true }; }}; } }; } console.log([...range(1, 5)]); // [1,2,3,4,5]
```

67 Generators?

Generator functions (function*) return a Generator object. They can pause execution at yield and resume later via next(). Each next() call runs to the next yield and returns { value, done }. Useful for lazy sequences, async control flow, and infinite streams.

Example / Code:

```
function* fibonacci() { let [a, b] = [0, 1]; while(true) { yield a; [a, b] = [b, a
+ b]; } } const fib = fibonacci(); console.log(fib.next().value); // 0
console.log(fib.next().value); // 1 console.log(fib.next().value); // 1
```

68 for...in vs for...of?

for...in iterates over enumerable property KEYS of an object (including prototype chain). for...of iterates over ITERABLE values (arrays, strings, Maps, Sets, generators). Use for...of for arrays/iterables; for...in for object properties.

Example / Code:

```
const arr = [10, 20, 30]; for(const i in arr) console.log(i); // '0' '1' '2' (keys) for(const v of arr) console.log(v); // 10 20 30 (values) const obj = { a: 1, b: 2 }; for(const k in obj) console.log(k); // a, b
```

69 Promise.all vs Promise.allSettled?

Promise.all(promises) — runs in parallel, resolves with all results if ALL succeed, rejects immediately if ANY fails. Promise.allSettled(promises) — runs in parallel, ALWAYS resolves with array of { status, value/reason } for every promise regardless of success/failure.

Example / Code:

```
const p1 = Promise.resolve(1); const p2 = Promise.reject('err'); Promise.all([p1, p2]).catch(e => console.log(e)); // 'err' Promise.allSettled([p1, p2]).then(results => results.forEach(r => console.log(r.status)) ); // fulfilled, rejected
```

70 Promise.race?

Promise.race(promises) resolves or rejects as soon as the FIRST promise settles (either way). Useful for timeouts: race a fetch against a timeout promise. The result/error is that of the fastest promise.

Example / Code:

```
const slow = new Promise(r => setTimeout(() => r('slow'), 2000)); const fast = new Promise(r => setTimeout(() => r('fast'), 500)); Promise.race([slow, fast]).then(console.log); // 'fast'
```

71 Promise.any?

Promise.any(promises) resolves with the value of the FIRST fulfilled promise. Rejects only if ALL promises reject, with an AggregateError. Opposite of Promise.all in behavior — optimistic.

Example / Code:

```
const p1 = Promise.reject('err1'); const p2 = Promise.reject('err2'); const p3 = Promise.resolve('success'); Promise.any([p1, p2, p3]).then(v => console.log(v)); // 'success'
```

72 What is debounce?

Debouncing delays a function's execution until after a specified time has elapsed since the last invocation. If called again before the delay ends, the timer resets. Used for search boxes, resize handlers, window scroll — prevents excessive calls.

Example / Code:

```
function debounce(fn, delay) { let timer; return function(...args) { clearTimeout(timer); timer = setTimeout(() => fn.apply(this, args), delay); }; } const onSearch = debounce((q) => fetchResults(q), 300);
```

73 What is throttle?

Throttling limits a function to execute at most ONCE per specified time window. Unlike debounce, it guarantees the function fires at regular intervals during continuous events. Used for scroll listeners, game loops, real-time updates.

Example / Code:

```
function throttle(fn, limit) { let lastCall = 0; return function(...args) { const now = Date.now(); if(now - lastCall >= limit) { lastCall = now; return fn.apply(this, args); } }; } window.addEventListener('scroll', throttle(updateUI, 100));
```

74 How to implement debounce?

Store a timer reference. On each call, clear the previous timer and set a new one. The actual function only runs if no new call comes in within the delay period. Optionally support 'immediate' mode to call on the leading edge.

Example / Code:

```
function debounce(fn, delay, immediate = false) { let timer; return function(...args) { const callNow = immediate && !timer; clearTimeout(timer); timer = setTimeout(() => { timer = null; if(!immediate) fn.apply(this, args); }, delay); if(callNow) fn.apply(this, args); }; }
```

75 How to implement throttle?

Track the timestamp of the last call. On each invocation, check if enough time has passed. If yes, execute and update the timestamp. If no, skip. Can also use a flag (leading/trailing edge control).

Example / Code:

```
function throttle(fn, limit) { let inThrottle = false; return function(...args) { if(!inThrottle) { fn.apply(this, args); inThrottle = true; setTimeout(() => inThrottle = false, limit); } }; }
```

76 What is currying?

Currying transforms a function with multiple arguments into a sequence of functions, each taking a single argument. f(a, b, c) becomes f(a)(b)(c). Enables partial application and function reuse.

Example / Code:

```
// Manual curry const multiply = a => b => a * b; const double = multiply(2); const triple = multiply(3); console.log(double(5)); // 10 console.log(triple(5)); // 15 // Generic curry const curry = fn => { const arity = fn.length; return function curried(...args) { return args.length >= arity ? fn(...args) : (...more) => curried(...args, ...more); }; };
```

77 What is function composition?

Function composition combines two or more functions so that the output of one becomes the input of the next. compose(f, g)(x) = f(g(x)). pipe() is left-to-right composition. Used in functional programming to build complex logic from simple functions.

Example / Code:

```
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x); const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x); const double = x => x * 2; const addOne = x => x + 1; const square = x => x * x; const transform = pipe(double, addOne, square); console.log(transform(3)); // ((3*2)+1)^2 = 49
```

78 What is memoization?

Memoization caches the results of expensive function calls and returns the cached result when the same inputs occur again. Trades memory for speed. Pure functions are ideal candidates. Used in dynamic programming and expensive computations.

Example / Code:

```
function memoize(fn) { const cache = new Map(); return function(...args) { const key = JSON.stringify(args); if(cache.has(key)) return cache.get(key); const result = fn.apply(this, args); cache.set(key, result); return result; }; } const memoFib = memoize(function fib(n) { return n <= 1 ? n : memoFib(n-1) + memoFib(n-2); });
```

79 What is a pure function?

A pure function: 1) Always returns the same output for the same inputs (deterministic). 2) Has no side effects (doesn't modify external state, no I/O, no mutations). Pure functions are predictable, testable, and composable.

Example / Code:

```
// Pure const add = (a, b) => a + b; // same input -> same output, no side effects // Impure let total = 0; const addToTotal = (n) => { total += n; return total; };
```

```
// modifies external state
```

80 What are side effects?

A side effect is any observable effect outside a function's scope: modifying global state, mutating arguments, making API calls, reading/writing files, DOM manipulation, logging. Side effects are necessary but should be managed carefully and isolated from pure logic.

Example / Code:

```
// Side effects: let count = 0; function increment() { count++; } // modifies
external var function fetchData() { return fetch('/api'); } // network call
function log(msg) { console.log(msg); } // I/O
```

■ Browser & Performance (Q81–Q110)

81 What is DOM?

The Document Object Model (DOM) is a tree-structured representation of an HTML/XML document. The browser creates it from HTML. JavaScript can manipulate the DOM to dynamically change content, structure, and styles.

Example / Code:

```
document.getElementById('btn').addEventListener('click', () => {
document.querySelector('h1').textContent = 'Clicked!'; });
```

82 DOM vs Virtual DOM?

Real DOM: slow to manipulate, re-renders the entire tree on changes, direct browser API. Virtual DOM (React): an in-memory JS object tree. React diffs old vs new virtual DOM (reconciliation) and applies only the minimal real DOM changes (patching). Much faster for frequent updates.

■ Virtual DOM is not inherently faster — it's the batching and minimal updates that make it efficient.

83 Event delegation?

Event delegation attaches a single listener on a parent element to handle events from ALL its children via event bubbling. More performant than attaching listeners to each child, especially for dynamic content.

Example / Code:

```
// Instead of 100 listeners on <li> elements:
document.getElementById('list').addEventListener('click', (e) => {
if(e.target.matches('li')) { console.log('Clicked:', e.target.textContent); } });
```

84 Bubbling vs capturing?

Bubbling (default): event propagates from target element UP to root. Capturing: event propagates DOWN from root to target. Use addEventListener(event, fn, true) for capture phase. stopPropagation() stops both.

Example / Code:

```
// Capture phase (top-down) parent.addEventListener('click', fn, true); //
capture // Bubble phase (bottom-up, default) child.addEventListener('click', fn,
false); // bubble
```

85 stopPropagation vs preventDefault?

stopPropagation() stops the event from propagating further (up for bubbling, down for capture) — other listeners on parent elements won't fire. preventDefault() prevents the browser's default action (e.g., form submission, link navigation) — event still propagates.

Example / Code:

```
link.addEventListener('click', (e) => { e.preventDefault(); // don't navigate
e.stopPropagation(); // don't bubble up handleClick(); });
```

86 What is reflow?

Reflow (layout) happens when the browser recalculates the position and size of all elements. Triggered by: adding/removing elements, changing CSS geometry (width, height, margin), font changes, viewport resize. Expensive — avoid in loops.

Example / Code:

```
// Bad: reads layout, then writes -> causes reflow per loop for(let i = 0; i <
100; i++) { el.style.width = el.offsetWidth + 1 + 'px'; // read+write } // Good:
read all first, then write all const w = el.offsetWidth; for(let i = 0; i < 100;
i++) { el.style.width = (w+i) + 'px'; }
```

87 What is repaint?

Repaint happens when visual styles (color, background, visibility) change without affecting layout. Less expensive than reflow but still costly if frequent. Caused by: color changes, box-shadow, outline, opacity. Reflow always causes repaint.

■ Use opacity and transform for animations — they are GPU-composited and avoid reflow/repaint.

88 What is lazy loading?

Lazy loading defers loading of non-critical resources (images, videos, scripts, route components) until they are needed (near viewport). Reduces initial page load time and bandwidth. Native: loading='lazy' attribute on <img>/<iframe>. In JS: IntersectionObserver.

Example / Code:

```
<img src='photo.jpg' loading='lazy' alt='...'> // Or with IntersectionObserver:
const observer = new IntersectionObserver((entries) => { entries.forEach(e => {
if(e.isIntersecting) { e.target.src = e.target.dataset.src;
observer.unobserve(e.target); } }); });
```

89 What is code splitting?

Code splitting divides your JS bundle into smaller chunks that are loaded on demand. Reduces initial load time. In webpack/Vite: dynamic import(). In React: React.lazy() + Suspense. Route-based splitting is the most common strategy.

Example / Code:

```
// React route-based splitting import { lazy, Suspense } from 'react'; const
Dashboard = lazy(() => import('./Dashboard')); <Suspense fallback={<Spinner />}>
<Dashboard /> </Suspense>
```

90 What is tree shaking?

Tree shaking is the elimination of dead code (exports that are never imported) from the final bundle. Enabled by ES6 static imports (which are statically analyzable). Tools: webpack, Rollup, Vite. CommonJS (require) is not tree-shakeable.

Example / Code:

```
// math.js exports add, subtract, multiply import { add } from './math'; // only
'add' included in bundle // subtract, multiply are tree-shaken away
```

91 What is CORS?

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls which cross-origin requests are allowed. Server must set Access-Control-Allow-Origin headers. Preflight OPTIONS requests are sent for non-simple requests.

Example / Code:

```
// Server (Express) CORS setup: app.use((req, res, next) => {
res.header('Access-Control-Allow-Origin', 'https://myapp.com');
res.header('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE');
res.header('Access-Control-Allow-Headers', 'Content-Type,Authorization'); next();
});
```

92 What is same-origin policy?

The same-origin policy is a browser security restriction that prevents scripts on one origin (protocol + domain + port) from reading responses from a different origin. Protects against CSRF and data theft. CORS is the mechanism to selectively relax this.

■ https://a.com and http://a.com are different origins (different protocol).

93 LocalStorage vs SessionStorage?

Both are Web Storage APIs. localStorage: persists across sessions (until manually cleared), ~5-10MB, shared across all tabs of same origin. sessionStorage: cleared when tab/window is closed, per-tab, ~5MB. Neither sends to server automatically (unlike cookies).

Example / Code:

```
localStorage.setItem('theme', 'dark');
console.log(localStorage.getItem('theme')); // 'dark'
sessionStorage.setItem('step', '2'); // cleared when tab closes
```

94 Cookies vs Storage?

Cookies: sent with every HTTP request (including to server), ~4KB limit, can be HTTP-only/Secure/SameSite, can have expiry, suitable for session IDs. localStorage/sessionStorage: browser-only (not sent to server), larger capacity, simpler API, no expiry for localStorage.

■ Store JWTs in HTTP-only cookies for security — prevents XSS access.

95 What is service worker?

A Service Worker is a script running in the background (separate thread), intercepting network requests, enabling offline caching (via Cache API), push notifications, and background sync. Fundamental to Progressive Web Apps (PWAs).

Example / Code:

```
// sw.js self.addEventListener('fetch', event => { event.respondWith(
caches.match(event.request).then(cached => cached || fetch(event.request) ) );
});
```

96 What is web worker?

Web Workers run JavaScript in a background thread separate from the main thread, enabling CPU-intensive operations without blocking the UI. Workers communicate with the main thread via postMessage(). Cannot access the DOM directly.

Example / Code:

```
// main.js const worker = new Worker('worker.js'); worker.postMessage({ data:
largeArray }); worker.onmessage = (e) => console.log('Result:', e.data); //
worker.js self.onmessage = (e) => { const result = heavyComputation(e.data);
self.postMessage(result); };
```

97 Single-threaded meaning?

JavaScript has a single call stack — it can execute only one piece of code at a time. Long operations block everything else. The browser/Node.js provides async APIs (Web APIs, libuv) that run operations in background threads and return results via callbacks to the event loop.

■ This is why you should never run CPU-heavy tasks synchronously in the main thread.

98 How JS handles concurrency?

JS uses the event loop + async APIs: 1) Synchronous code runs on call stack. 2) Async operations (I/O, timers) are delegated to browser/OS APIs. 3) On completion, callbacks are placed in the queue. 4) Event loop pushes them to call stack when it's empty.

99 Memory leaks?

Memory leaks occur when memory that is no longer needed is not released. Common causes: 1) Unremoved event listeners. 2) Forgotten timers/intervals. 3) Detached DOM nodes still referenced.

4) Closures holding large data. 5) Global variables accumulating.

Example / Code:

```
// Leak: interval keeps reference to element const el =
document.getElementById('btn'); const timer = setInterval(() => el.style.color =
'red', 100); // Fix: clear when done clearInterval(timer);
```

100 How to avoid memory leaks?

1) Remove event listeners with removeEventListener when done. 2) Clear timers with clearInterval/clearTimeout. 3) Nullify references to large objects when no longer needed. 4) Use WeakMap/WeakSet for caches keyed by objects. 5) Avoid global variables. 6) Use Chrome DevTools Memory panel.

101 Garbage collection?

JS uses automatic garbage collection — memory is automatically freed when objects are no longer reachable (no references). The GC periodically scans memory and reclaims unreachable objects. V8 uses generational garbage collection (young/old generation).

102 Mark and sweep?

Mark-and-sweep is the main GC algorithm. Phase 1 (Mark): starting from roots (global, stack), traverse all reachable objects and mark them. Phase 2 (Sweep): scan all memory and free unmarked objects. V8 also uses incremental and concurrent marking to reduce pauses.

103 What is event listener leak?

Attaching event listeners without removing them creates memory leaks because the listener holds a closure reference. Even if the DOM element is removed from the tree, it stays in memory if the listener reference exists.

Example / Code:

```
// Leak const btn = document.getElementById('btn'); btn.addEventListener('click',
heavyHandler); // Later remove the element without removing listener // Fix
btn.removeEventListener('click', heavyHandler); btn.remove();
```

104 Passive event listeners?

Passive listeners (addEventListener('touchstart', fn, { passive: true })) promise the browser that preventDefault() won't be called. The browser can optimize scroll performance by not waiting for the listener to complete before scrolling.

Example / Code:

```
// Passive - browser doesn't wait for JS before scrolling
window.addEventListener('wheel', onScroll, { passive: true });
```

105 requestAnimationFrame?

requestAnimationFrame(callback) schedules a function to run before the next browser repaint (~60fps). Ideal for animations — synchronized with display refresh rate, paused when tab is hidden, more efficient than setTimeout for animations.

Example / Code:

```
function animate() { element.style.left = (parseFloat(element.style.left) + 1) +
'px'; if(parseFloat(element.style.left) < 500) { requestAnimationFrame(animate);
} } requestAnimationFrame(animate);
```

106 requestIdleCallback?

requestIdleCallback(callback) schedules work during browser idle time (when no user interactions or animations). Ideal for non-critical background work: analytics, pre-fetching, cleanup. Receives an IdleDeadline object with timeRemaining().

Example / Code:

```
requestIdleCallback((deadline) => { while(deadline.timeRemaining() > 0 &&
tasksQueue.length) { processTask(tasksQueue.shift()); } }, { timeout: 2000 });
```

107 IntersectionObserver?

IntersectionObserver watches when elements enter/leave the viewport (or a parent container). Replaces scroll event listeners for visibility detection. Used for lazy loading, infinite scroll, analytics, animations on scroll.

Example / Code:

```
const observer = new IntersectionObserver((entries) => { entries.forEach(entry =>
{ if(entry.isIntersecting) { entry.target.classList.add('visible');
observer.unobserve(entry.target); // stop watching } }); }, { threshold: 0.1 });
observer.observe(document.querySelector('.card'));
```

108 MutationObserver?

MutationObserver watches for changes to the DOM (attributes, child nodes, text content). Replaces deprecated Mutation Events. Used for: monitoring dynamic content, implementing undo, detecting ad injection, lazy hydration.

Example / Code:

```
const observer = new MutationObserver((mutations) => { mutations.forEach(m => {
console.log('Changed:', m.type, m.target); }); });
observer.observe(document.body, { childList: true, subtree: true, attributes:
true });
```

109 Performance optimization techniques?

1) Code splitting and lazy loading. 2) Debounce/throttle event handlers. 3) Virtual DOM / minimize reflows. 4) Cache API calls (memoize). 5) Web Workers for heavy computation. 6) Compression (gzip/brotli). 7) CDN. 8) Image optimization (WebP, lazy). 9) Tree shaking. 10) Service Worker caching.

110 Critical rendering path?

The CRP is the sequence of steps to render initial content: 1) Parse HTML -> DOM. 2) Parse CSS -> CSSOM. 3) Combine -> Render Tree. 4) Layout (reflow). 5) Paint. 6) Composite. Optimize: minimize render-blocking resources, inline critical CSS, defer JS.

■ Use <link rel='preload'> for critical assets and <script defer> for JS.

■ Tricky & Interview Questions (Q111–Q140)

111 Closure loop output (var vs let)?

Classic interview trap: using var in a for loop creates one shared variable. All callbacks close over the same variable (its final value). Using let creates a new binding per iteration — each callback captures its own i.

Example / Code:

```
// var - all print 3 for(var i = 0; i < 3; i++) { setTimeout(() => console.log(i),
0); // 3, 3, 3 } // let - prints 0, 1, 2 for(let i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 0); // 0, 1, 2 }
```

112 Output of setTimeout in loop?

When setTimeout(fn, 0) is in a loop, ALL synchronous code finishes first (the loop completes), THEN all setTimeout callbacks run. With var, i has reached its final value by then.

Example / Code:

```
for(var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); } // Synchronous
loop finishes (i=3), then callbacks run // Output: 3, 3, 3
```

■ Fix: use let, or wrap in IIFE: (function(j){ setTimeout(()=>console.log(j),0); })(i);

113 var vs let in loop?

var: function-scoped, one binding for the entire loop. let: block-scoped, new binding per iteration. This is why let is always preferred in loops — each iteration gets its own closed-over variable.

114 Explain event loop with example?

The event loop: 1) Execute all synchronous code (call stack). 2) Process all microtasks (Promises). 3) Take one macrotask (setTimeout) and process it. 4) Repeat. Stack -> Microtask queue -> Macrotask queue.

Example / Code:

```
console.log('sync 1'); setTimeout(() => console.log('macro'), 0);
Promise.resolve().then(() => console.log('micro')); console.log('sync 2'); //
Output: sync 1, sync 2, micro, macro
```

115 Promise chaining output?

Each .then() returns a new Promise. Return values are wrapped in resolved Promises. Throwing or returning rejected Promise propagates to .catch(). Understanding the return value is key.

Example / Code:

```
Promise.resolve(1) .then(v => v + 1) // 2 .then(v => { throw new Error('oops'); })
.catch(e => e.message) // 'oops' .then(v => console.log(v)); // 'oops'
```

116 async/await vs .then()?

Both handle Promises. async/await is synchronous-looking, easier to read/debug, stack traces are cleaner, try/catch for errors. .then() is functional-style, better for parallel chains, no special function needed. They are interchangeable in most cases.

Example / Code:

```
// .then() fetch(url).then(r => r.json()).then(data =>
use(data)).catch(handleError); // async/await async function load() { try { const
r = await fetch(url); const data = await r.json(); use(data); } catch(e) {
handleError(e); } }
```

117 Why can't arrow functions be constructors?

Arrow functions do not have their own 'this' binding or 'prototype' property. Constructors need both: 'this' to attach properties and 'prototype' for the prototype chain. Calling new on an arrow function throws TypeError.

Example / Code:

```
const Foo = () => {}; new Foo(); // TypeError: Foo is not a constructor
console.log(Foo.prototype); // undefined
```

118 What happens when you return an object in a constructor?

If a constructor function explicitly returns an OBJECT, that object is returned instead of 'this'. If it returns a primitive (or nothing), 'this' is returned as normal.

Example / Code:

```
function Person(name) { this.name = name; return { custom: true }; // returns
this object } const p = new Person('Alice'); console.log(p.name); // undefined
console.log(p.custom); // true
```

119 How JS handles memory?

JS uses a heap for dynamic allocations and a stack for execution contexts. The garbage collector automatically reclaims unreachable heap memory (mark-and-sweep). V8 uses generational GC: short-lived objects in 'young generation', long-lived in 'old generation'.

120 How to polyfill bind?

Polyfilling Function.prototype.bind from scratch: create a new function that uses apply to call the original with the given this and merged arguments.

Example / Code:

```
Function.prototype.myBind = function(ctx, ...partial) { const fn = this; return function(...args) { return fn.apply(ctx, [...partial, ...args]); }; }; function greet(g, p) { return `${g}, ${this.name}${p}`; } const hi = greet.myBind({name:'Bob'}, 'Hi'); console.log(hi('!')); // Hi, Bob!
```

121 How to polyfill Array.map?

Implement map by iterating over the array, calling the callback for each element (with value, index, array), and collecting results in a new array.

Example / Code:

```
Array.prototype.myMap = function(callback) { const result = []; for(let i = 0; i < this.length; i++) { if(Object.prototype.hasOwnProperty.call(this, i)) { result.push(callback(this[i], i, this)); } } return result; };
console.log([1,2,3].myMap(x => x*2)); // [2,4,6]
```

122 What is call stack overflow?

A stack overflow happens when the call stack exceeds its maximum size — usually caused by infinite recursion or excessively deep function calls. Throws 'Maximum call stack size exceeded' (RangeError).

Example / Code:

```
function infinite() { return infinite(); } try { infinite(); } catch(e) { console.log(e.message); } // Maximum call stack size exceeded
```

123 What is tail call optimization?

TCO is an optimization where the engine reuses the current stack frame for a tail call (a function call as the last action), preventing stack growth. ES6 specifies TCO but only Safari fully implements it. Enables stack-safe recursion.

Example / Code:

```
// Tail recursive (can be optimized) function factorial(n, acc = 1) { if(n <= 1) return acc; return factorial(n - 1, n * acc); // tail call }
```

124 Why is JS single-threaded?

JS was designed for the browser where DOM manipulation must be safe — two threads modifying the DOM simultaneously would cause race conditions and corruption. Single-threaded keeps it simple and safe. Async is handled via event loop, not multiple threads.

125 Can JS be multi-threaded?

Sort of. Web Workers run JS in background threads but cannot access the DOM. SharedArrayBuffer + Atomics enable shared memory between workers. Node.js worker_threads allow multi-threading. The MAIN thread is always single-threaded for DOM operations.

126 What is Proxy?

Proxy wraps an object and intercepts fundamental operations (get, set, delete, apply, etc.) via 'traps'. Used for validation, logging, reactivity systems (Vue 3), mocking.

Example / Code:

```
const handler = { get(target, prop) { return prop in target ? target[prop] : `Property ${prop} not found`; }, set(target, prop, value) { if(typeof value !== 'number') throw TypeError('Numbers only'); target[prop] = value; return true; } }; const obj = new Proxy({}, handler); obj.x = 42; // OK console.log(obj.x); // 42
console.log(obj.y); // Property y not found
```

127 What is Reflect?

Reflect is a built-in object with static methods mirroring Proxy traps. It provides default behavior for intercepted operations and is the companion to Proxy. Useful in Proxy traps to call default behavior

after custom logic.

Example / Code:

```
const p = new Proxy({}, { set(target, prop, value, receiver) {
console.log(`Setting ${prop} = ${value}`); return Reflect.set(target, prop,
value, receiver); } }); p.x = 10; // logs: Setting x = 10
```

128 What is Intl API?

The Internationalization API (Intl) provides locale-sensitive string comparison, number formatting, date/time formatting, and more. Handles localization without external libraries.

Example / Code:

```
// Number formatting new Intl.NumberFormat('en-IN', { style: 'currency',
currency: 'INR' }).format(1234567); // '■12,34,567.00' // Date new
Intl.DateTimeFormat('hi-IN').format(new Date()); // Hindi date format
```

129 What is BigInt?

BigInt is a primitive type for integers of arbitrary precision, beyond Number.MAX_SAFE_INTEGER (2^53-1). Created by appending n to a literal or using BigInt(). Cannot be mixed with regular numbers in arithmetic.

Example / Code:

```
const big = 9007199254740991n + 1n; console.log(big); // 9007199254740992n
console.log(typeof big); // 'bigint' // Cannot mix: 1n + 1 -> TypeError
```

130 What is Temporal API?

Temporal is a modern TC39 proposal (stage 3) to replace the broken Date API. Provides immutable date/time objects, proper timezone handling, calendar support, and arithmetic. Not yet in all browsers but polyfills exist.

Example / Code:

```
// Future syntax (Temporal API): const now = Temporal.Now.plainDateTimeISO();
const tomorrow = now.add({ days: 1 }); console.log(tomorrow.toString());
```

131 What is shadow DOM?

Shadow DOM is a web standard allowing encapsulated DOM trees attached to elements. Styles and scripts inside shadow DOM don't leak out, and external styles don't leak in. Used by web components for style encapsulation.

Example / Code:

```
const host = document.getElementById('widget'); const shadow =
host.attachShadow({ mode: 'open' }); shadow.innerHTML = ` <style>p { color: red;
}</style> <p>Encapsulated content</p>`;
```

132 What is hydration?

Hydration is the process of attaching JavaScript event listeners to server-rendered HTML, making it interactive. In SSR, the server sends HTML (fast first paint), then the browser downloads JS and 'hydrates' the static HTML into a live React/Vue app.

■ Hydration mismatch errors occur when server HTML differs from what client-side rendering would produce.

133 CSR vs SSR?

CSR (Client-Side Rendering): browser downloads blank HTML + JS, JS renders content. Fast after initial load, poor SEO, slow first paint. SSR (Server-Side Rendering): server renders full HTML, sent to browser. Good SEO, fast first paint, but TTFB may be higher. SSG pre-renders at build time.

134 What is a module bundler?

A module bundler (webpack, Vite, Rollup, Parcel) takes many JS modules and their dependencies and bundles them into one or few files for the browser. Also handles: transpilation, tree-shaking, code splitting, asset optimization.

135 What is V8 engine?

V8 is Google's open-source JS engine (used in Chrome and Node.js). It compiles JS to native machine code using JIT (Just-In-Time) compilation. Features: hidden classes, inline caching, generational GC, Ignition interpreter + TurboFan compiler.

136 How JS code is executed internally?

1) Parsing: source code -> AST (Abstract Syntax Tree). 2) Compilation: AST -> bytecode (Ignition). 3) JIT optimization: hot functions -> optimized machine code (TurboFan). 4) Execution on call stack. 5) Deoptimization if assumptions break.

137 Parsing vs compiling?

Parsing converts source code (text) into an AST — a tree structure representing the code's syntax. Compiling converts the AST to bytecode or machine code. V8 parses eagerly for top-level code but lazily parses function bodies (only compiles when called).

138 JIT compilation?

Just-In-Time compilation compiles code at runtime rather than ahead of time. JS starts interpreted (fast startup), identifies hot code paths, and compiles them to optimized machine code (fast execution). Best of both worlds vs pure interpretation or AOT.

139 Hidden classes?

V8 creates hidden classes (shapes) for objects with the same property structure. Objects sharing a hidden class share optimized property access paths. Adding properties in a different order creates different hidden classes, deoptimizing access. Always initialize properties in the same order.

Example / Code:

```
// Good: same hidden class const a = { x: 1, y: 2 }; const b = { x: 3, y: 4 }; //
Bad: different hidden classes const c = { x: 1 }; c.y = 2; const d = { y: 1 }; d.x
= 2;
```

140 Inline caching?

Inline caching (IC) is a V8 optimization that caches the result of property lookups. After a property access is resolved once, V8 caches the offset. Subsequent accesses of the same property skip the lookup. Monomorphic ICs (one type) are fastest; megamorphic (many types) are slowest.

Part 2
React + Redux

200 Questions — Hooks, Architecture, State Management

■ React Basics (Q1–Q30)

1 What is React?

React is an open-source JavaScript library for building user interfaces, developed by Facebook. It uses a component-based architecture and a virtual DOM for efficient updates. React is declarative — you describe what the UI should look like, and React handles the DOM updates.

2 Why use React?

1) Component reusability. 2) Virtual DOM for performance. 3) Unidirectional data flow (predictable). 4) Rich ecosystem (React Router, Redux, etc.). 5) Large community. 6) Server-side rendering (Next.js). 7) React Native for mobile.

3 What is SPA?

A Single Page Application loads a single HTML page and dynamically updates content via JavaScript without full page reloads. Navigation feels instant, no server round-trips for each page. React is commonly used to build SPAs. Drawback: initial load time and SEO concerns.

4 What is a component?

A component is a reusable, self-contained piece of UI. In React, components are functions (or classes) that accept props and return JSX. They can manage their own state and compose with other components.

Example / Code:

```
const Button = ({ label, onClick }) => ( <button
onClick={onClick}>{label}</button> );
```

5 Functional vs class components?

Functional components are plain JS functions returning JSX. With hooks, they can manage state and lifecycle. Class components extend React.Component, use this.state, and have lifecycle methods. Functional components are preferred today — simpler, no 'this', hooks support, better performance.

Example / Code:

```
// Functional const Hello = ({ name }) => <h1>Hello, {name}</h1>; // Class class
Hello extends React.Component { render() { return <h1>Hello,
{this.props.name}</h1>; } }
```

6 What is JSX?

JSX (JavaScript XML) is a syntax extension allowing HTML-like code in JavaScript. It compiles to React.createElement() calls. Not required but makes code much more readable. Expressions go in {}; attributes use camelCase (className, onClick).

Example / Code:

```
// JSX const elem = <div className='box'>{2 + 2}</div>; // Compiles to: const
elem = React.createElement('div', {className:'box'}, 4);
```

7 How JSX works internally?

Babel transforms JSX into React.createElement(type, props, ...children) calls. With React 17+, the new JSX transform imports jsx() from 'react/jsx-runtime' automatically — no need to import React in

every file.

8 What is Virtual DOM?

The Virtual DOM is an in-memory JavaScript representation of the real DOM tree. React uses it to compute differences (diffing) between renders and then applies only the minimal necessary changes to the real DOM (reconciliation). This batches and optimizes DOM updates.

9 Virtual DOM vs Real DOM?

Real DOM: heavy, slow to update, re-renders the whole tree. Virtual DOM: lightweight JS objects, fast comparison via diffing algorithm, updates only changed parts of real DOM. Virtual DOM acts as a buffer — React computes changes cheaply in JS, then applies them efficiently to the real DOM.

10 What is reconciliation?

Reconciliation is React's algorithm for comparing the current virtual DOM tree with the new one (after state/props change) and computing the minimal set of DOM operations to update the UI. Uses a diffing algorithm with O(n) complexity by making assumptions about component keys and types.

11 What is key in React?

Keys are special attributes that help React identify which items in a list have changed, been added, or removed. Keys must be stable, unique among siblings, and not array indices (unless list is static). They make reconciliation efficient for list updates.

Example / Code:

```
<ul>{items.map(item => <li key={item.id}>{item.name}</li>)}</ul>
```

- Never use array index as key for dynamic lists — causes bugs with reordering.

12 Why are keys important?

Without keys, React re-renders all list items on any change. With stable keys, React knows exactly which item changed and updates only that one. Wrong keys (like indices) cause state mismatch bugs — components don't reset properly when list order changes.

13 What is props?

Props (properties) are read-only inputs passed from parent to child components. They are immutable from the child's perspective — child cannot modify props. They flow down the component tree (unidirectional). Anything can be passed as props: primitives, objects, functions, JSX.

Example / Code:

```
const Greeting = ({ name, age }) => <p>{name} is {age}</p>; <Greeting
name='Alice' age={30} />
```

14 Props vs state?

Props: data passed from parent, read-only in child, controlled externally. State: data managed internally by a component, can change over time (triggers re-render). Props are for configuration; state is for dynamic data that the component owns.

15 What is state?

State is local, mutable data managed by a component. When state changes, React re-renders the component and its children. In functional components, managed with useState hook. State triggers UI updates — it's the source of dynamism in React apps.

Example / Code:

```
const [count, setCount] = useState(0); // count = current value // setCount =
updater function
```

16 What is setState?

setState (in class components) or the setter from useState (functional) schedules a state update and triggers a re-render. State updates may be batched. The setter can take a new value or an updater function receiving the previous state.

Example / Code:

```
// useState setter setCount(count + 1); // direct setCount(prev => prev + 1); //
functional update (safe for batching)
```

17 Is setState synchronous?

No! setState (and useState setters) are asynchronous — React batches multiple state updates and applies them together for performance. You cannot read the updated state immediately after calling setState. Use functional updates or useEffect to react to state changes.

Example / Code:

```
setCount(count + 1); console.log(count); // still old value! Use functional
update. setCount(prev => { console.log(prev); return prev + 1; });
```

18 What is lifting state up?

Lifting state up means moving shared state to the nearest common ancestor component. Child components receive state as props and call callbacks to update it. This is the primary way to share state between sibling components without external state management.

19 What is prop drilling?

Prop drilling is passing props through many layers of components that don't need them, just to reach a deeply nested child. Problems: verbose, hard to refactor, components are tightly coupled. Solutions: Context API, Redux, Zustand, component composition.

20 What are controlled components?

A controlled component has its value/state managed by React state. Input value is bound to state; changes go through onChange handler -> setState -> re-render. The form element's value always reflects React state — single source of truth.

Example / Code:

```
const [value, setValue] = useState(''); <input value={value} onChange={e =>
setValue(e.target.value)} />
```

21 What are uncontrolled components?

Uncontrolled components manage their own state internally (like the DOM does traditionally). Use a ref to read the value when needed. Simpler for basic forms but harder to validate/transform in real time.

Example / Code:

```
const inputRef = useRef(); const handleSubmit = () =>
console.log(inputRef.current.value); <input ref={inputRef} defaultValue='initial'
/>
```

22 What is conditional rendering?

Rendering different UI based on conditions. Methods: 1) if/else. 2) Ternary operator. 3) Logical &&. 4) Switch. 5) Early return.

Example / Code:

```
// Ternary {isLoggedIn ? <Dashboard /> : <Login />} // Logical && {error &&
<ErrorMessage msg={error} />}
```

23 What is list rendering?

Rendering arrays of data as lists of components. Use .map() to transform data arrays into JSX arrays. Each element MUST have a unique key prop for efficient reconciliation.

Example / Code:

```
{users.map(user => ( <UserCard key={user.id} name={user.name} email={user.email}
/> ))}
```

24 What are fragments?

Fragments (<React.Fragment> or <>) let you group elements without adding an extra DOM node. Useful when a component must return multiple elements (a component can only return one root element). The short syntax <> </> doesn't support key attribute.

Example / Code:

```
<> <h1>Title</h1> <p>Paragraph</p> </> // No extra div in DOM
```

25 What is React.StrictMode?

StrictMode is a development tool that highlights potential issues. It: 1) Double-invokes certain functions to detect side effects. 2) Warns about deprecated APIs. 3) Detects unexpected side effects. Has NO effect in production.

Example / Code:

```
<React.StrictMode> <App /> </React.StrictMode>
```

26 What is React root?

React root is the entry point — the top-level container connecting React to the real DOM. Created with createRoot() in React 18 (or ReactDOM.render() in older versions).

Example / Code:

```
import { createRoot } from 'react-dom/client'; const root =
createRoot(document.getElementById('root')); root.render(<App />);
```

27 createRoot vs render?

createRoot() is the React 18 API enabling concurrent features (automatic batching, transitions, etc.). ReactDOM.render() (legacy) uses blocking rendering mode. createRoot also renders into a container element.

28 What is hydration in React?

In SSR, the server sends pre-rendered HTML. The browser displays it immediately (fast first paint). React then 'hydrates' the static HTML — attaches event listeners and makes it interactive. hydrateRoot() is the React 18 API for this.

Example / Code:

```
import { hydrateRoot } from 'react-dom/client';
hydrateRoot(document.getElementById('root'), <App />, initialData);
```

29 What is React lifecycle?

React components go through phases: 1) Mounting — component created and inserted into DOM. 2) Updating — state or props change, component re-renders. 3) Unmounting — component removed from DOM. Hooks map to these phases via useEffect.

30 Mounting vs updating vs unmounting?

Mounting: constructor -> render -> DOM update -> componentDidMount / useEffect([]) runs. Updating: state/props change -> render -> DOM update -> componentDidUpdate / useEffect([dep]) runs. Unmounting: componentWillUnmount / useEffect cleanup runs.

■ Hooks (Q31–Q80)

31 What are hooks?

Hooks are functions (prefixed with 'use') that let functional components use React state, lifecycle, context, and other features. Introduced in React 16.8. Rules: only call at top level (not in loops/conditions), only call from React functions.

32 Why were hooks introduced?

Problems hooks solve: 1) Reusing stateful logic (no HOCs or render props). 2) Complex components with split logic (lifecycle methods mixed concerns). 3) Classes are confusing ('this', binding). Hooks let you extract and share stateful logic as custom hooks.

33 What is useState?

useState(initialValue) returns [state, setter]. The setter triggers a re-render. Initial value can be a primitive, object, or a function (lazy initialization — called once). The setter replaces state (doesn't merge like class setState).

Example / Code:

```
const [user, setUser] = useState(null); const [count, setCount] = useState(() =>
expensiveInit()); // lazy init
```

34 How useState works internally?

React maintains state in a linked list associated with each component instance (fiber). Each useState call appends to this list. The call order must be stable — this is why hooks can't be in conditionals. The setter schedules a re-render of the component.

35 What is useEffect?

useEffect(fn, deps) runs a side effect after render. It replaces componentDidMount, componentDidUpdate, and componentWillUnmount. Returns an optional cleanup function. The dependency array controls when it runs.

36 When does useEffect run?

1) No deps array: after EVERY render. 2) Empty []: after FIRST render only (mount). 3) [dep1, dep2]: after first render AND whenever dep1 or dep2 changes. Cleanup runs before the next effect and on unmount.

Example / Code:

```
useEffect(() => {}, ); // every render useEffect(() => {}, []); // mount only
useEffect(() => {}, [count]); // when count changes
```

37 Dependency array?

The dependency array tells React when to re-run the effect. React compares deps with shallow equality. Include all reactive values used inside the effect (exhaustive-deps ESLint rule). Omitting a dep causes stale closure bugs.

■ Use the eslint-plugin-react-hooks/exhaustive-deps rule to avoid missed dependencies.

38 Cleanup function in useEffect?

Return a function from useEffect to clean up side effects before the next effect runs or on unmount. Used for: unsubscribing, clearing timers, canceling requests, removing event listeners.

Example / Code:

```
useEffect(() => { const sub = dataStream.subscribe(handleData); return () =>
sub.unsubscribe(); // cleanup on unmount/update }, []);
```

39 useEffect vs componentDidMount?

componentDidMount runs once after mount (class). useEffect with [] runs once after mount (functional). Both handle initialization. But useEffect also handles cleanup (return fn) and re-runs on dep changes — it's more general than componentDidMount.

40 useEffect vs useLayoutEffect?

useEffect: runs asynchronously AFTER DOM is painted. No visual flicker but slightly delayed. useLayoutEffect: runs synchronously AFTER DOM update but BEFORE paint. Use for measuring DOM layout, avoiding flicker. Prefers useEffect; use useLayoutEffect only for DOM measurements.

41 What is useRef?

useRef(initialValue) returns a mutable object { current: value }. Persists across renders without causing re-renders. Used for: 1) Accessing DOM elements. 2) Storing mutable values (like previous state, timer IDs). 3) Instance variables (not in state).

Example / Code:

```
const inputRef = useRef(null); <input ref={inputRef} /> const focusInput = () =>
inputRef.current.focus();
```

42 useRef vs state?

useRef: changes don't trigger re-renders, value persists across renders, used for DOM refs and mutable values. State: changes trigger re-renders, used for values that should cause UI updates. Use ref for values that shouldn't cause re-renders.

43 What is useMemo?

useMemo(fn, deps) memoizes the RETURN VALUE of a computation. Recalculates only when deps change. Used for expensive calculations to avoid re-computation on every render.

Example / Code:

```
const sortedList = useMemo( () => [...items].sort((a,b) =>
a.name.localeCompare(b.name)), [items] );
```

44 When to use useMemo?

Use when: 1) Computation is expensive (sorting large arrays, complex math). 2) Result is used as a dependency in other hooks (to maintain referential equality). Do NOT overuse — useMemo itself has overhead and clutters code.

45 What is useCallback?

useCallback(fn, deps) memoizes the FUNCTION REFERENCE. Returns the same function object across renders if deps don't change. Used when passing callbacks to child components that are wrapped in React.memo.

Example / Code:

```
const handleClick = useCallback(() => { onSelect(id); }, [id, onSelect]);
```

46 useCallback vs useMemo?

useCallback(fn, deps) === useMemo(() => fn, deps). useCallback memoizes a function reference. useMemo memoizes a computed value. Use useCallback to prevent child re-renders when passing callbacks; use useMemo to prevent expensive re-computations.

47 What is useContext?

useContext(Context) reads the current value of a Context. Re-renders when context value changes. Avoids prop drilling for globally shared data (theme, auth, locale). Use with createContext() and Provider.

Example / Code:

```
const ThemeContext = createContext('light'); function App() { return
<ThemeContext.Provider value='dark'><Child /></ThemeContext.Provider>; } function
Child() { const theme = useContext(ThemeContext); return <div>{theme}</div>; }
```

48 Context API?

Context provides a way to share values between components without passing props at every level. Created with createContext(), provided via <Context.Provider value={}>, consumed via useContext(). All consumers re-render when context value changes.

49 When to use Context?

Use Context for: theme, locale, authentication, current user, feature flags. Do NOT use for: frequently changing values (causes all consumers to re-render), or replacing Redux for complex state with many actions. Context is for global configuration, not state management.

50 What is a custom hook?

A custom hook is a function starting with 'use' that calls other hooks. Lets you extract and reuse stateful logic across components. It's the primary code reuse mechanism in hooks-based React.

Example / Code:

```
function useWindowSize() { const [size, setSize] = useState({ w:
window.innerWidth, h: window.innerHeight }); useEffect(() => { const update = ()
=> setSize({ w: window.innerWidth, h: window.innerHeight });
window.addEventListener('resize', update); return () =>
window.removeEventListener('resize', update); }, []); return size; }
```

51 Rules of hooks?

1) Only call hooks at the TOP LEVEL (not in conditions, loops, or nested functions). 2) Only call hooks from React function components or custom hooks (not regular JS functions). These rules ensure the hook call order is consistent across renders.

52 Why must hooks be at top level?

React stores hook state in a linked list. The call order determines which state belongs to which hook. If hooks were inside conditions, the order could change between renders, causing React to associate state with the wrong hook — bugs and crashes.

53 What is useReducer?

useReducer(reducer, initialState) is like useState but for complex state logic. Takes a reducer function (state, action) => newState and returns [state, dispatch]. Better than useState when: state has many sub-values, next state depends on previous, complex update logic.

Example / Code:

```
const [state, dispatch] = useReducer(reducer, { count: 0 }); dispatch({ type:
'INCREMENT' }); dispatch({ type: 'SET', payload: 10 });
```

54 useReducer vs useState?

useState: simple scalar or object state. useReducer: complex state with multiple sub-values and many update actions, state transitions based on previous state, co-locating update logic. useReducer makes complex state logic readable and testable.

55 What is dispatch?

dispatch is the function returned by useReducer (and Redux's store.dispatch). Calling dispatch(action) sends an action object to the reducer function, which returns the new state. Actions typically have type and optional payload.

56 What is useImperativeHandle?

useImperativeHandle(ref, createHandle, deps) customizes the instance value exposed when a parent uses ref on a child (with forwardRef). Lets you expose a limited API instead of the full DOM element.

Example / Code:

```
const Input = forwardRef((props, ref) => { const inputRef = useRef();
useImperativeHandle(ref, () => ({ focus: () => inputRef.current.focus(), clear:
() => { inputRef.current.value = ''; } })); return <input ref={inputRef} />; });
```

57 What is forwardRef?

React.forwardRef() lets a component receive a ref from its parent and pass it to a DOM element or child component. Needed because refs don't work like regular props — they're handled specially by

React.

Example / Code:

```
const FancyInput = React.forwardRef((props, ref) => ( <input ref={ref} {...props}
className='fancy' /> )); // Parent: const ref = useRef(); <FancyInput ref={ref}
/>
```

58 What is useId?

useId() generates a unique, stable ID for a component instance. Useful for accessibility attributes (aria-labelledby, htmlFor) where IDs must be unique and consistent across server/client renders.

Example / Code:

```
const id = useId(); return ( <> <label htmlFor={id}>Name:</label> <input id={id}
/> </> );
```

59 What is useTransition?

useTransition() returns [isPending, startTransition]. Wrap non-urgent state updates in startTransition to mark them as lower priority. React can interrupt these to keep the UI responsive (concurrent mode).

Example / Code:

```
const [isPending, startTransition] = useTransition(); const search = (q) => {
startTransition(() => setResults(filter(q))); // non-urgent }; {isPending &&
<Spinner />}
```

60 What is useDeferredValue?

useDeferredValue(value) returns a deferred version of the value. React may use the old value first to keep the UI responsive, then update with the new value when it has time. Similar to useTransition but for values you don't control.

61 What is concurrent rendering?

Concurrent rendering (React 18) lets React prepare multiple versions of the UI simultaneously. React can interrupt, pause, and resume rendering. Enables: useTransition, Suspense, useDeferredValue. Keeps the UI responsive during heavy rendering.

62 What is batching?

Batching groups multiple state updates into a single re-render. React 17: only batched inside event handlers. React 18 (automatic batching): batches ALL state updates everywhere (async, timeouts, promises, native events).

Example / Code:

```
// React 18 - all updates batched in one render setTimeout(() => { setA(1);
setB(2); // only ONE re-render, not two }, 0);
```

63 Automatic batching in React 18?

React 18 automatically batches state updates in ALL contexts: event handlers, timeouts, promises, native event listeners. In React 17, async code would cause separate re-renders. Use ReactDOM.flushSync() to opt out of batching when needed.

64 What is Suspense?

Suspense lets components 'wait' for something (lazy loading, data fetching) and show a fallback while waiting. Used with React.lazy() for code splitting and with data fetching libraries that support Suspense.

Example / Code:

```
<Suspense fallback={<Spinner />}> <LazyComponent /> </Suspense>
```

65 What is lazy loading in React?

React.lazy(() => import('./Component')) dynamically imports a component only when first rendered. Must be wrapped in Suspense. Enables route-based code splitting — reduces initial bundle size.

Example / Code:

```
const Chart = React.lazy(() => import('./Chart')); function App() { return
<Suspense fallback={<Loading />}><Chart /></Suspense>; }
```

66 React.lazy vs dynamic import?

React.lazy() is a React-specific wrapper around dynamic import() that returns a special component. It works with Suspense for loading states. Plain dynamic import() is a general JS feature returning a Promise. React.lazy is built on dynamic import.

67 What is error boundary?

Error boundaries are class components (or libraries like react-error-boundary) that catch JS errors in their child tree, log them, and display a fallback UI instead of crashing the whole app. Hooks cannot create error boundaries — must use class component.

Example / Code:

```
class ErrorBoundary extends React.Component { state = { hasError: false }; static
getDerivedStateFromError() { return { hasError: true }; }
componentDidCatch(error, info) { logError(error, info); } render() { return
this.state.hasError ? <Fallback /> : this.props.children; } }
```

68 Can hooks replace lifecycle methods?

Yes. useEffect([]) = componentDidMount. useEffect([dep]) = componentDidUpdate. useEffect cleanup = componentWillUnmount. shouldComponentUpdate = React.memo + useMemo. Custom hooks can encapsulate complex lifecycle logic more cleanly than class methods.

69 What are stale closures?

A stale closure captures an outdated value of a variable. In useEffect or callbacks, if deps are missing, the function sees the old value of state/props from when it was created, not the current value.

Example / Code:

```
// Bug: stale closure const [count, setCount] = useState(0); useEffect(() => {
const id = setInterval(() => console.log(count), 1000); return () =>
clearInterval(id); }, []); // count is always 0 (stale!) // Fix: add count to
deps, or use ref
```

70 How to avoid infinite re-renders?

1) Don't create new object/array literals or functions in the dependency array without memoization. 2) Use useMemo/useCallback for stable references. 3) Use functional updates to avoid depending on state. 4) Check that effects don't set state unconditionally.

71 What is dependency optimization?

Ensure useEffect/useMemo/useCallback deps are minimal and stable. Primitives are compared by value (safe). Objects/arrays/functions are compared by reference (use useMemo/useCallback to stabilize). Omitting deps causes stale closures; over-including causes too many re-runs.

72 Why do functions re-create on render?

In JS, every function defined inside a component body is a new function object on every render. This means child components or effects depending on that function see a 'changed' value every render. Use useCallback to return the same function reference when deps haven't changed.

73 Memoization in React?

Three tools: 1) React.memo — memoizes component render (shallow props comparison). 2) useMemo — memoizes computed values. 3) useCallback — memoizes function references.

Over-memoizing adds overhead and code complexity — memoize only bottlenecks.

74 When NOT to use useMemo?

1) Simple/fast computations — overhead of useMemo exceeds savings. 2) Values that change almost every render. 3) When code clarity matters more. 4) Premature optimization. Only memoize when profiling shows a real performance problem.

75 When NOT to use useCallback?

1) When the child doesn't use React.memo. 2) When the function changes every render anyway (deps change every time). 3) Simple inline functions. 4) Premature optimization. useCallback only helps when paired with React.memo on the receiving component.

76 What is hook composition?

Hook composition is building complex behavior by combining multiple hooks inside a custom hook. Like function composition — each hook handles one concern, custom hook combines them. Enables clean, reusable, testable logic extraction.

77 What is a side effect in React?

A side effect is anything that affects the outside world from inside a component: data fetching, DOM manipulation, subscriptions, timers, logging. React expects render to be pure — manage side effects in useEffect.

78 Sync vs async effects?

useEffect always runs asynchronously after paint. useLayoutEffect runs synchronously after DOM updates but before paint (use for DOM measurements). For async operations in useEffect, define an async function inside and call it — don't make the effect callback async directly.

Example / Code:

```
useEffect(() => { async function fetchData() { const data = await
api.get('/data'); setData(data); } fetchData(); }, []);
```

79 How React schedules updates?

React 18 uses a priority-based scheduler (Scheduler package). Updates have priorities: SyncLane (user input), InputContinuousLane (scroll/drag), DefaultLane (data fetching), TransitionLane (useTransition), IdleLane. Higher priority updates can interrupt lower priority ones (concurrent mode).

80 What is Fiber architecture?

React Fiber (React 16+) is the internal reconciliation engine. It represents the component tree as a linked list of fiber nodes. Enables: incremental rendering (pause/resume), priority-based scheduling, concurrent mode, error boundaries, Suspense.

■ Advanced React (Q81–Q120)

81 What is React Fiber?

Fiber is the reimplementation of React's reconciliation algorithm (React 16+). It represents each component as a unit of work (fiber node). Enables incremental rendering — React can split work into chunks, pause, prioritize, and abort work. Foundation for concurrent features.

82 How does reconciliation work internally?

React performs a tree traversal comparing new fiber tree with current fiber tree (work-in-progress tree). It creates, updates, or deletes fibers. This happens in two phases: 1) Render phase (interruptible) — compute what changed. 2) Commit phase (synchronous) — apply changes to DOM.

83 What is the diffing algorithm?

React's diffing: 1) Different element types -> destroy old tree, build new tree. 2) Same type elements -> update attributes in place. 3) List items -> use key to match items. This achieves O(n) complexity vs O(n^3) for general tree diffing.

84 Render phase vs commit phase?

Render phase: pure computation, interruptible, can be paused/resumed/abandoned. Determines what changed. Commit phase: applies mutations to the real DOM, runs effects, not interruptible. Happens synchronously.

85 What is concurrent mode?

Concurrent mode lets React prepare multiple UI versions simultaneously and interrupt rendering. Enables: useTransition (non-blocking updates), Suspense for data fetching, automatic batching, useDeferredValue. Keeps UI responsive during heavy computation.

86 What is time slicing?

Time slicing is React's ability to split rendering work into small chunks (slices) across multiple frames. React checks if there's higher-priority work (user input) and yields control to the browser between slices. Prevents long tasks from blocking the UI.

87 What is Suspense for data fetching?

Suspense can pause rendering while data loads. Library must support Suspense (React Query, Relay, etc.) by throwing a Promise. React shows the fallback until the Promise resolves. Simplifies loading states compared to conditional rendering.

88 What are server components?

React Server Components (RSC) run on the server during build/request time. They can directly access databases, file systems, and secrets. They render to a special format (not HTML), and their code is never sent to the client. Zero JavaScript bundle contribution.

89 Client vs server components?

Server components: render on server, no state/effects/browser APIs, can access backend, not in bundle. Client components ('use client'): render on client (and optionally server for hydration), full React capabilities (hooks, events, browser APIs). Default in Next.js 13+ App Router is server.

90 What is streaming SSR?

Streaming SSR sends HTML in chunks (HTTP streaming). The server can send the shell (layout, above-fold content) immediately, while still rendering slower parts. Suspense boundaries define what to stream — fallback shown until each part resolves.

91 What is hydration mismatch?

A hydration mismatch occurs when server-rendered HTML differs from what React expects on the client. Causes: date/time rendering, random values, browser-only APIs, different data between server and client. React logs a warning and re-renders from scratch (slower).

92 What is React.memo?

React.memo(Component) is a HOC that memoizes a functional component. If props are shallowly equal to previous render, React skips re-rendering. Use for expensive components that receive the same props frequently.

Example / Code:

```
const MemoCard = React.memo(({ user }) => ( <div>{user.name}</div> )); // Only
re-renders when user prop reference changes
```

93 React.memo vs useMemo?

React.memo wraps a COMPONENT to prevent re-renders when props are unchanged. useMemo memoizes a COMPUTED VALUE inside a component. Both use memoization but at different levels — component level vs value level.

94 PureComponent?

React.PureComponent is the class component equivalent of React.memo. It implements shouldComponentUpdate with a shallow comparison of props and state. Prevents re-renders when props/state haven't changed shallowly.

95 What is a HOC (Higher-Order Component)?

A HOC is a function that takes a component and returns an enhanced component. Pattern for cross-cutting concerns (logging, auth, data fetching). Convention: name starts with 'with'. Now largely replaced by hooks.

Example / Code:

```
function withAuth(WrappedComponent) { return function AuthComponent(props) {
const { isAuthenticated } = useAuth(); return isAuthenticated ? <WrappedComponent
{...props} /> : <Redirect to='/login' />; }; }
```

96 HOC vs hooks?

HOCs wrap components (nesting, harder to debug, props collision risk). Hooks encapsulate logic inside a component (flat, no nesting, composable). Hooks are preferred for new code. HOCs still useful for third-party library integration and class components.

97 What is render props?

Render props is a pattern where a component accepts a function prop that returns JSX. The component calls the function (sharing internal state as arguments). Enabled code sharing before hooks. Now largely replaced by custom hooks.

Example / Code:

```
function Mouse({ render }) { const [pos, setPos] = useState({ x: 0, y: 0 });
return <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY
})}>{render(pos)}</div>; } <Mouse render={(pos) => <p>Position: {pos.x},
{pos.y}</p>} />
```

98 Render props vs HOC?

Both are code-sharing patterns. Render props are explicit (you see the data passed). HOCs wrap and inject props implicitly (less visible). Both cause 'wrapper hell' with deep nesting. Custom hooks are the preferred modern alternative.

99 What is compound components?

Compound components is a pattern where a parent component implicitly shares state with child components, making them work together as a unit. Uses React.Children or Context internally. Example: <Select>, <Tabs>, <Accordion>.

Example / Code:

```
// <Select value={v} onChange={fn}> // <Select.Option value='a'>A</Select.Option>
// <Select.Option value='b'>B</Select.Option> // </Select>
```

100 What is a portal?

ReactDOM.createPortal(child, container) renders a child component into a different DOM node outside the parent's hierarchy. The component still behaves like a React child (events bubble through React tree). Used for: modals, tooltips, dropdowns.

Example / Code:

```
function Modal({ children }) { return ReactDOM.createPortal( <div
className='modal-overlay'>{children}</div>, document.getElementById('modal-root')
); }
```

101 Use cases of portals?

1) Modals — avoid z-index and overflow issues. 2) Tooltips — position relative to viewport. 3) Dropdown menus — overflow: hidden parent. 4) Toast notifications — always on top. All need to render outside the normal DOM flow.

102 What is ref forwarding?

Ref forwarding allows passing a ref through a component to one of its children (usually a DOM element). Required when you want to access a DOM element inside a library component. Uses React.forwardRef().

103 What are synthetic events?

React wraps native browser events in SyntheticEvent — a cross-browser wrapper. Provides consistent API across browsers. Events are pooled (pre-React 17) or non-pooled (React 17+). Access event properties directly; don't access them in async callbacks without persisting.

104 Event pooling?

Pre-React 17, SyntheticEvent objects were pooled for performance — properties were nullified after the event handler. In React 17+, event pooling was removed. Now event objects are regular JS objects and can be accessed asynchronously.

105 What is batching updates?

React batches multiple state updates together and performs a single re-render. React 18 does this automatically everywhere. React 17 only batched in React event handlers.

106 What is priority scheduling?

React 18's scheduler assigns priorities to updates. User interactions (clicks, keystrokes) get the highest priority. Background updates (data fetching, transitions) get lower priority. High-priority updates can interrupt low-priority ones.

107 What is React Profiler?

React Profiler is a DevTools feature (and <Profiler> component) that measures rendering performance. Shows which components render, how often, and how long. Identifies unnecessary re-renders and performance bottlenecks.

Example / Code:

```
<Profiler id='App' onRender={(id, phase, actualDuration) => { console.log(id,
phase, actualDuration + 'ms'); }}> <App /> </Profiler>
```

108 Performance optimization techniques in React?

1) React.memo + useCallback for unnecessary renders. 2) useMemo for expensive calculations. 3) Virtualize long lists (react-window). 4) Code splitting with React.lazy. 5) Avoid anonymous functions in JSX. 6) Use production build. 7) Debounce/throttle handlers.

109 Code splitting in React?

React.lazy + Suspense for component-level splitting. React Router with lazy() for route-level splitting. Each route loads its JS only when visited — dramatically reduces initial bundle size.

110 Lazy loading images in React?

Options: 1) Native loading='lazy' attribute. 2) IntersectionObserver API. 3) Libraries like react-lazyload or react-intersection-observer. 4) Next.js Image component (automatic optimization).

111 What is tree structure diffing?

React uses heuristics: components of different types produce different trees (replace entirely). Keys help match list items. React optimizes by assuming sibling components don't move (only same-type, same-position updates are reconciled).

112 How React handles large lists?

Long lists cause slow renders. Solutions: 1) Virtualization (react-window, react-virtualized) — only renders visible items. 2) Pagination. 3) Infinite scroll with IntersectionObserver. 4) React.memo on list items. 5) key optimization.

113 Windowing / virtualization?

Virtualization renders only the visible items in a list (the 'window'). As the user scrolls, items are recycled. Reduces DOM nodes from thousands to a dozen. Critical for large datasets (10k+ rows).

114 react-window vs react-virtualized?

react-window: smaller, faster, modern API, recommended for most use cases. react-virtualized: larger, more features (grids, masonry, auto-sizers), but heavier. react-window is a rewrite of react-virtualized by the same author.

115 What is hydration performance?

Slow hydration blocks the main thread. Selective hydration (React 18) hydrates in priority order — Suspense boundaries let React hydrate visible parts first. Progressive hydration defers hydration of off-screen content.

116 What is SSR?

Server-Side Rendering renders React components on the server to HTML string. The browser receives fully-formed HTML (good for SEO, fast first paint). Then React hydrates it. Used with Next.js, Remix.

117 CSR vs SSR vs SSG?

CSR: client renders everything in browser (good DX, poor initial load/SEO). SSR: server renders per request (good SEO, fast FCP, higher server load). SSG: pre-rendered at build time (fastest, best SEO, but content is static until next build).

118 What is Next.js?

Next.js is a React framework by Vercel providing: SSR, SSG, ISR, App Router, API routes, image optimization, font optimization, built-in TypeScript, automatic code splitting, and deployment infrastructure.

119 When to use SSR?

Use SSR when: 1) SEO is critical (content must be crawled). 2) Content changes per request (personalized dashboards). 3) First-paint performance matters. 4) Auth-gated pages. Use SSG for content that doesn't change per user.

120 What are render phases?

React's two-phase commit: Render phase (pure, interruptible) creates a fiber work-in-progress tree describing what needs to change. Commit phase (synchronous, side-effectful) applies DOM changes, runs effects (useLayoutEffect, useEffect) in order.

■ Redux (Q121–Q170)

121 What is Redux?

Redux is a predictable state container for JavaScript apps. State lives in a single store. State is read-only (only actions can change it). Changes are made with pure reducer functions. Works with React via react-redux.

122 Why Redux?

1) Centralized state — single source of truth. 2) Predictable state transitions. 3) Time-travel debugging (DevTools). 4) State is serializable. 5) Easy testing (pure reducers). 6) Useful for large apps with complex state sharing.

123 Core principles of Redux?

1) Single source of truth — one store. 2) State is read-only — only actions change state. 3) Changes via pure functions — reducers are pure functions that return new state from old state + action.

124 What is a Redux store?

The store holds the complete state tree. Created with configureStore() (Redux Toolkit) or createStore(). Provides getState(), dispatch(), and subscribe() methods.

Example / Code:

```
const store = configureStore({ reducer: { counter: counterReducer, users:
usersReducer } }); console.log(store.getState());
```

125 What is a Redux action?

An action is a plain JS object with a 'type' property describing what happened. Optionally has a 'payload'. Dispatched to the store to trigger state changes.

Example / Code:

```
const increment = { type: 'counter/increment' }; const setUser = { type:
'user/set', payload: { id: 1, name: 'Alice' } };
```

126 What is a reducer?

A reducer is a pure function (state, action) => newState. It determines how state changes in response to an action. Must not mutate state directly — return a new state object. Must have no side effects.

Example / Code:

```
function counterReducer(state = { value: 0 }, action) { switch(action.type) {
case 'increment': return { value: state.value + 1 }; case 'decrement': return {
value: state.value - 1 }; default: return state; } }
```

127 What is Redux dispatch?

dispatch(action) is the only way to trigger a state change. It sends an action to the store, which passes it through middleware, then to the reducer. In react-redux: const dispatch = useDispatch().

128 What is Redux middleware?

Middleware is a plugin for the dispatch pipeline. It can inspect actions, modify them, dispatch additional actions, or perform side effects (API calls, logging). Applied with applyMiddleware() or configureStore's middleware option.

129 Redux flow?

User interaction -> dispatch(action) -> middleware (thunk, saga) -> reducer -> new state -> store updates -> connected components re-render.

130 What is immutability in Redux?

Reducers must return new state objects, not mutate the existing state. This enables time-travel debugging, change detection, and predictability. Redux Toolkit uses Immer under the hood, allowing 'mutating' syntax that produces new objects.

Example / Code:

```
// Redux Toolkit (Immer) - looks like mutation but isn't const slice =
createSlice({ reducers: { increment: state => { state.value += 1; } // Immer
handles immutability } });
```

131 Why must reducers be pure?

Pure reducers: 1) Produce predictable, testable behavior. 2) Enable time-travel debugging (replay actions). 3) Allow change detection via shallow equality. 4) Enable multiple instances of a store. 5) Prevent unexpected side effects.

132 What is combineReducers?

combineReducers({ key: reducer }) merges multiple reducers into one root reducer. Each reducer manages its own slice of state. Equivalent to Redux Toolkit's automatically combined slice reducers in configureStore.

133 What is Redux Toolkit (RTK)?

RTK is the official, opinionated toolset for Redux. It includes: configureStore (setup with DevTools), createSlice (actions + reducer), createAsyncThunk (async actions), createEntityAdapter, RTK Query. Reduces boilerplate significantly.

134 Why use Redux Toolkit?

1) Much less boilerplate. 2) Immer for safe mutations. 3) Built-in DevTools. 4) createAsyncThunk for async. 5) RTK Query for data fetching. 6) Official recommendation from Redux team. Before RTK, Redux required a lot of manual setup.

135 What is createSlice?

createSlice auto-generates action creators and action types from reducer functions. Reduces the boilerplate of separate action type constants and action creators.

Example / Code:

```
const counterSlice = createSlice({ name: 'counter', initialState: { value: 0 },
reducers: { increment: state => { state.value += 1; }, setCount: (state, action)
=> { state.value = action.payload; } } }); export const { increment, setCount } =
counterSlice.actions;
```

136 What is createAsyncThunk?

createAsyncThunk generates a thunk that dispatches pending/fulfilled/rejected action types automatically when an async operation runs. Handles async lifecycle in Redux cleanly.

Example / Code:

```
export const fetchUser = createAsyncThunk('users/fetch', async (userId) => {
const res = await axios.get(`/users/${userId}`); return res.data; }); // In slice
extraReducers: builder.addCase(fetchUser.fulfilled, (state, action) => {
state.users.push(action.payload); });
```

137 What is RTK Query?

RTK Query is a data fetching and caching tool built into Redux Toolkit. Automatically generates hooks for fetching, caching, invalidating, and re-fetching data. Eliminates the need for manual loading/error/data state management.

Example / Code:

```
const api = createApi({ reducerPath: 'api', baseQuery: fetchBaseQuery({ baseUrl:
'/api' }), endpoints: (builder) => ({ getUser: builder.query({ query: (id) =>
`users/${id}` }), }) }); const { data, isLoading } = useGetUserQuery(userId);
```

138 Redux vs Context API?

Redux: robust for large apps, DevTools, middleware, time-travel, predictable. Context: built-in, simpler, good for low-frequency updates (theme, auth). Context re-renders all consumers on every change. Redux uses selectors to avoid unnecessary re-renders.

139 When NOT to use Redux?

1) Simple apps with minimal state. 2) State is local to one component. 3) Server state only (use React Query instead). 4) Adds unnecessary complexity for small projects. Don't use Redux just because you're using React.

140 What is a selector?

A selector is a function that extracts and derives data from the Redux state. Keeps components decoupled from state shape. Memoized selectors (reselect) prevent expensive re-computations.

Example / Code:

```
const selectUserById = (state, id) => state.users.entities[id]; const user =
useSelector(state => selectUserById(state, userId));
```

141 What is reselect?

Reselect is a library for creating memoized selectors. createSelector(inputSelectors, resultFn) returns a selector that only recomputes when inputs change. RTK includes createSelector from reselect.

Example / Code:

```
const selectExpensiveData = createSelector( [selectRawData, selectFilter], (raw,
filter) => raw.filter(item => item.type === filter) // memoized );
```

142 What is normalization in Redux?

Normalizing state stores entities by ID in a flat dictionary (like a database table) instead of nested arrays. Prevents duplicate data, simplifies updates, and makes lookups O(1). createEntityAdapter in RTK helps with this.

Example / Code:

```
// Normalized: { ids: [1,2], entities: { 1: {...}, 2: {...} } } // Not normalized:
[{ id: 1, ...}, { id: 2, ...}]
```

143 What is entity adapter?

createEntityAdapter() provides a standardized way to store and manage normalized data in Redux. Auto-generates CRUD operations (addOne, removeOne, updateOne, setAll, etc.) and selectors.

144 What is Redux DevTools?

Redux DevTools is a browser extension providing: action history, state diff viewer, time-travel debugging (replay/rewind), action dispatching, import/export state. Automatically integrated with configureStore.

145 What is a thunk?

A thunk is a function returned by an action creator that receives dispatch (and getState) as arguments. Allows async logic (API calls) inside Redux. redux-thunk middleware intercepts functions dispatched to the store.

Example / Code:

```
export const fetchUsers = () => async (dispatch, getState) => {
dispatch(setLoading(true)); const data = await api.getUsers();
dispatch(setUsers(data)); dispatch(setLoading(false)); };
```

146 Thunk vs Saga?

Thunk: simpler, async/await, less boilerplate, good for most cases. Saga: uses generators, more complex, but better for advanced flows: cancellation, debouncing, race conditions, complex orchestration. Use thunk by default; saga for complex async requirements.

147 What is Redux Saga?

redux-saga is middleware using generator functions to manage side effects. Sagas are long-lived background processes that listen for actions and react accordingly. Supports: takeLatest, takeEvery, put, call, fork, cancel, race.

148 Generator in Saga?

Sagas use function* generators. yield put(action) dispatches an action. yield call(fn, args) calls a function (supports cancellation). yield take(actionType) waits for an action. Generators make async code look synchronous and are easy to test.

149 takeLatest vs takeEvery?

takeLatest: cancels previous running saga when a new action of the same type arrives — good for search (only process the last request). takeEvery: spawns a new saga for every action — good for logging all events.

150 What is Redux Persist?

redux-persist serializes and saves Redux state to storage (localStorage, AsyncStorage) and rehydrates it on app startup. Configure with persistReducer and persistStore.

151 What is store hydration?

Store hydration is the process of initializing Redux state from persisted/SSR data. On first render, preloaded state (from localStorage or server) populates the store. Redux Persist handles this automatically.

152 How to structure a Redux app?

Feature-based (recommended): each feature has its own slice, selectors, and async thunks in one folder. Ducks pattern: actions + reducer in one file per feature. Avoid global state for local UI state.

153 Ducks pattern?

Ducks bundles actions, action creators, and reducer for a feature into a single file ('duck'). Redux Toolkit's createSlice naturally implements the ducks pattern.

154 Feature-based structure?

Feature folders: /features/users/usersSlice.js, /features/users/usersApi.js, /features/users/UserList.jsx. Each feature is self-contained. Scales well for large apps.

155 What is global vs local state?

Global state: shared across many components (user auth, cart, app settings) — use Redux or Context. Local state: relevant to one component only (form input, toggle) — use useState. Don't put everything in Redux.

156 What is derived state?

Derived state is computed from existing state (not stored separately). Use memoized selectors (reselect) to compute derived values. Storing derived state creates sync problems — compute it on demand.

157 What is side effects handling in Redux?

Side effects (API calls, timers, navigation) don't belong in reducers (pure functions). Use: thunks, sagas, Redux observables, or RTK Query. They intercept actions and run async logic before/after

state updates.

158 Async flow in Redux?

1) Dispatch async action creator (thunk). 2) Thunk makes API call. 3) On success, dispatch fulfilled action with data. 4) Reducer updates state. 5) Component re-renders with new data. createAsyncThunk automates this pattern.

159 How to avoid re-renders in Redux?

1) Use specific selectors (not the entire state). 2) Memoize selectors with reselect. 3) Use shallowEqual in useSelector for object returns. 4) Split large reducers. 5) Normalize state to avoid deeply nested subscriptions.

Example / Code:

```
const count = useSelector(state => state.counter.value); // specific const { a, b
} = useSelector(selector, shallowEqual); // object comparison
```

160 Performance issues in Redux?

1) Selectors returning new object references every call. 2) Too many useSelector calls. 3) Large state causing expensive comparisons. 4) Rendering entire component trees on small changes. Fix: memoized selectors, React.memo, splitting components.

161 What is batching in Redux?

Redux Toolkit uses unstable_batchedUpdates from react-dom to batch multiple dispatch calls into a single React re-render. React 18 automatic batching removes the need for this in most cases.

162 What is optimistic update?

An optimistic update immediately updates the UI assuming the API call will succeed, then reverts if it fails. Provides a fast, responsive feel. RTK Query has built-in optimistic update support via onQueryStarted.

Example / Code:

```
// RTK Query optimistic onQueryStarted: async (arg, { dispatch, queryFulfilled })
=> { const patch = dispatch(api.util.updateQueryData('getPosts', undefined, draft
=> { draft.push(newPost); })); try { await queryFulfilled; } catch {
patch.undo(); } // revert on failure }
```

163 What is caching strategy in Redux?

RTK Query caches responses per endpoint + args. Cache entries have a keepUnusedDataFor lifetime. Invalidation via tags (providesTags, invalidatesTags) automatically refetches related queries when mutations occur.

164 What is stale data problem?

After a mutation, cached query data becomes stale. RTK Query solves with tag-based invalidation — mutations declare which tags they invalidate; queries that provide those tags automatically refetch.

165 What is state normalization?

Storing entities in a flat structure by ID prevents nested data, duplication, and complex update logic. Makes state lookups O(1) and updates surgical (change one entity without re-creating arrays).

166 What is selector memoization?

reselect's createSelector caches the last result. If the input selectors return the same values (by reference), the output function is skipped and the cached result is returned. Prevents unnecessary re-renders.

167 What is middleware chaining?

Redux middleware is composed as a chain. Each middleware calls next(action) to pass to the next middleware. The order matters: logger -> thunk -> devtools. configureStore's middleware array controls the order.

168 Logging middleware?

A simple logging middleware logs every action and the resulting state. Useful for debugging.

Example / Code:

```
const loggerMiddleware = store => next => action => { console.log('dispatching:',
action); const result = next(action); console.log('next state:',
store.getState()); return result; };
```

169 Error handling in Redux?

1) In thunks: try/catch and dispatch error action. 2) createAsyncThunk automatically handles rejected state. 3) Global error handler middleware. 4) RTK Query's isError, error from useQuery.

170 Testing Redux?

Test reducers as pure functions (given state + action, assert new state). Test thunks with mock dispatch and getState. Test connected components with a mock store (using redux-mock-store or configureStore with test data).

■ React + Redux Interview Scenarios (Q171–Q200)

171 When to use Redux vs Context?

Context: theme, locale, auth state (low frequency updates). Redux: large apps, complex state logic, frequent updates, need DevTools, sharing state across many distant components. Default to Context/useState, add Redux when you feel the pain.

172 How to optimize a React app?

1) Profile first (DevTools). 2) React.memo for expensive components. 3) useMemo/useCallback for stable references. 4) Code split with lazy(). 5) Virtualize long lists. 6) Debounce heavy handlers. 7) Optimize images. 8) Reduce bundle size.

173 How to prevent unnecessary renders?

1) React.memo on child components. 2) useCallback for function props. 3) useMemo for object/array props. 4) Specific useSelector selectors (not whole state). 5) Split large components. 6) Avoid inline object literals in JSX.

174 How React handles large-scale apps?

1) Feature-based folder structure. 2) Code splitting per route. 3) RTK Query for server state. 4) React.memo and virtualization for performance. 5) TypeScript for type safety. 6) Storybook for component docs. 7) Micro-frontend architecture for very large teams.

175 Folder structure for scalable app?

src/features/[feature]/{component, slice, api, hooks, types}. src/components/ui/ for shared UI. src/app/ for store, router, providers. src/hooks/ for global custom hooks. src/utils/ for helpers.

176 How to design reusable components?

1) Single responsibility. 2) Configurable via props. 3) Compound component pattern for complex widgets. 4) Forward refs when needed. 5) No hard-coded data. 6) Document with Storybook. 7) Prop types / TypeScript interfaces.

177 How to handle API calls in React?

Options: 1) useEffect + useState (basic). 2) Custom useApi hook. 3) React Query / SWR (server state). 4) RTK Query (with Redux). RTK Query or React Query is recommended — handles caching, revalidation, loading/error states automatically.

178 Axios vs fetch?

fetch: built-in, no dependency, returns Response object (need .json()), no request cancellation (use AbortController). Axios: automatic JSON parsing, request/response interceptors, better error handling (rejects on 4xx/5xx), request cancellation, timeout support.

179 How to manage global state?

1) useState/useReducer for local. 2) Context for low-frequency global (theme, auth). 3) Redux/Zustand/Jotai for complex global state. 4) React Query/RTK Query for server state. Don't put server state in Redux.

180 What is the best state management?

No universal best. Guidelines: Local state -> useState. Shared simple -> Context. Complex global -> Redux Toolkit / Zustand. Server data -> React Query / RTK Query. Atomic state -> Jotai / Recoil. Choose based on team, app complexity, and specific needs.

181 How to design a dashboard app?

1) Route-based code splitting. 2) RTK Query for data. 3) Memoized chart components. 4) Virtualized tables. 5) Skeleton loaders. 6) Real-time: WebSocket + Redux. 7) Role-based widget visibility. 8) Error boundaries per widget.

182 How to handle forms at scale?

React Hook Form (minimal re-renders, validation) or Formik. Schema validation with Zod or Yup. For complex multi-step forms: useReducer or form state machine. Uncontrolled inputs with refs for performance in large forms.

183 Controlled vs uncontrolled forms?

Controlled: every keystroke goes through React state (instant validation, formatting, disabled states). Uncontrolled: refs read value on submit (less code, better performance for simple forms). React Hook Form uses uncontrolled inputs by default.

184 Form libraries: Formik vs React Hook Form?

React Hook Form: minimal re-renders (uncontrolled), smaller bundle, better performance, simpler API, growing adoption. Formik: controlled, more established, larger ecosystem. React Hook Form recommended for new projects.

185 How to handle authentication in React?

1) Login -> receive JWT + refresh token. 2) Store access token in memory (or HttpOnly cookie). 3) Auth context/Redux for isAuthenticated, user. 4) PrivateRoute protects routes. 5) Axios interceptor attaches token. 6) Token refresh on 401 response.

186 JWT storage best practices?

Best: HttpOnly cookie (inaccessible to XSS). Acceptable: memory (state variable — lost on refresh). Avoid: localStorage/sessionStorage (vulnerable to XSS). Use SameSite=Strict/Lax + HTTPS for cookies.

187 How to handle role-based UI?

1) Fetch user roles after auth. 2) Store in auth context/Redux. 3) Create a Permission component or usePermission hook. 4) Conditionally render based on role. 5) Server validates permissions for

actual data — client-side is just UX.

188 How to protect routes?

Create a PrivateRoute/RequireAuth component that checks auth state and redirects to login if not authenticated. In React Router v6: wrap routes with element={<RequireAuth><Page /></RequireAuth>}.

Example / Code:

```
function RequireAuth({ children }) { const { user } = useAuth(); const location = useLocation(); if(!user) return <Navigate to='/login' state={{ from: location }} replace />; return children; }
```

189 Lazy loading routes in React?

In React Router, import route components with React.lazy(). Wrap the route outlet with Suspense. Each route chunk loads only when the user navigates to that route.

190 Error handling strategy in React?

1) Error boundaries for unexpected render errors. 2) try/catch in event handlers. 3) useEffect async error handling. 4) RTK Query / React Query built-in error states. 5) Global axios interceptor for API errors. 6) Toast notifications for user-facing errors.

191 Retry failed API calls?

Use axios-retry or implement manually: on 5xx or network error, retry with exponential backoff. RTK Query has retry capability. React Query has retry option. Always limit retry count to avoid infinite loops.

192 Debounce search implementation?

Debounce the search input: wait for user to stop typing (300ms) before making API call. Use custom useDebounce hook or third-party (use-debounce).

Example / Code:

```
function useDebounce(value, delay) { const [debouncedValue, setDebouncedValue] = useState(value); useEffect(() => { const handler = setTimeout(() => setDebouncedValue(value), delay); return () => clearTimeout(handler); }, [value, delay]); return debouncedValue; }
```

193 Infinite scrolling implementation?

Use IntersectionObserver to detect when a sentinel element (last item) enters viewport. On intersection, fetch next page and append. RTK Query handles pagination automatically with fetchNextPage.

194 Pagination vs infinite scroll?

Pagination: user explicitly navigates pages, easy to bookmark, less memory, good for search results. Infinite scroll: continuous feed, no friction, hard to bookmark/return, high memory over time. Use pagination for task-oriented UIs; infinite scroll for content feeds.

195 SEO in React (SPA)?

SPAs have poor SEO because crawlers see empty HTML. Solutions: 1) SSR (Next.js). 2) SSG for static content. 3) Prerendering (react-snap, Prerender.io). 4) Meta tags with react-helmet. 5) Sitemaps. SSR/SSG is the best long-term solution.

196 Performance bottlenecks in React?

1) Unnecessary re-renders. 2) Missing virtualization for long lists. 3) Large bundle (no code splitting). 4) Blocking API calls on mount. 5) Memory leaks (unremoved listeners). 6) Expensive inline calculations without memoization.

197 Bundle size optimization?

1) Code splitting (React.lazy). 2) Tree shaking (ESM imports). 3) Analyze with webpack-bundle-analyzer. 4) Lazy import large libraries. 5) Use lighter alternatives (day.js vs moment.js). 6) Compress assets. 7) CDN for vendor chunks.

198 Monitoring tools for React?

1) Sentry (error tracking). 2) DataDog / New Relic (APM). 3) LogRocket (session replay). 4) React DevTools Profiler (render performance). 5) Lighthouse (Core Web Vitals). 6) Web Vitals library (CLS, LCP, FID).

199 Logging strategy in React?

1) Client errors: Sentry or custom error boundary logging. 2) User actions: analytics events (GA, Mixpanel). 3) Performance: Web Vitals logging. 4) API errors: log in axios interceptor. 5) Different log levels by environment.

200 Real-world architecture of React app?

Layers: UI (components) -> hooks (business logic) -> services (API) -> store (Redux/Query) -> Router. Feature folders per domain. TypeScript throughout. Storybook for components. MSW for API mocking. Cypress for e2e. CI/CD via GitHub Actions.

Part 3. Node.js

130 Questions — Event Loop to Scaling

■ Basics (Q1–Q30)

1 What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model, making it lightweight and efficient. Designed for building scalable network applications (APIs, real-time apps).

Example / Code:

```
// Hello world HTTP server const http = require('http'); http.createServer((req, res) => { res.end('Hello World'); }).listen(3000);
```

2 Why use Node.js?

1) JavaScript everywhere (front+back). 2) Non-blocking I/O — handles thousands of concurrent connections. 3) npm ecosystem (largest package registry). 4) Fast for I/O-heavy workloads. 5) Real-time (WebSockets). 6) Active community.

3 Node.js vs browser JavaScript?

Node.js: no DOM/window/document, file system access (fs), networking, process object, module system (CommonJS + ESM). Browser: DOM, Web APIs, localStorage, fetch (native), restricted file access. Same JS engine (V8) but different environments.

4 What is V8 engine?

V8 is Google's open-source JavaScript engine, written in C++. It compiles JS to machine code via JIT (Ignition interpreter + TurboFan optimizing compiler). Handles memory management, garbage collection. Used in Chrome and Node.js.

5 How Node.js works internally?

Node.js wraps V8 (JS execution) with libuv (async I/O, event loop, thread pool). When async operations (file I/O, network) are triggered, libuv handles them in a thread pool. On completion, callbacks are queued for the event loop to process.

6 Single-threaded meaning in Node?

Node.js runs JavaScript on ONE thread (the main thread). There's no parallel JS execution. However, I/O operations are offloaded to libuv's thread pool (4 threads by default). This allows Node to handle many concurrent operations without blocking.

7 Is Node.js really single-threaded?

The JS execution is single-threaded. But internally, libuv uses a thread pool (default 4 threads) for file system, DNS, compression, and crypto operations. The main event loop runs on one thread; background tasks run on worker threads.

8 What is event-driven architecture?

Event-driven architecture triggers actions based on events. Node.js uses this model: operations register callbacks for events (data, error, end). The event loop drives the execution. No waiting — when an event fires, the callback runs.

9 What is non-blocking I/O?

Non-blocking I/O operations initiate a request and return immediately without waiting for the result. When the I/O completes, a callback is invoked. Contrast with blocking I/O which freezes execution until the operation finishes.

Example / Code:

```
// Non-blocking fs.readFile('file.txt', (err, data) => { console.log(data); });
console.log('This runs immediately, not after readFile'); // Blocking const data
= fs.readFileSync('file.txt'); // freezes here
```

10 Blocking vs non-blocking?

Blocking: execution stops until operation completes (readFileSync). Non-blocking: operation initiated, execution continues, callback fires later (readFile). Node.js's performance advantage comes from avoiding blocking calls.

11 What is REPL?

REPL (Read-Eval-Print Loop) is an interactive Node.js shell. Type expressions, get immediate results. Useful for quick testing, exploring APIs, debugging.

Example / Code:

```
node > 2 + 2 4 > const arr = [1,2,3] > arr.map(x => x*2) [ 2, 4, 6 ]
```

12 What are modules in Node.js?

Modules are reusable blocks of code. Node.js supports CommonJS (require/module.exports) and ES Modules (import/export). Each file is its own module with its own scope.

Example / Code:

```
// CommonJS module.exports = { add: (a,b) => a+b }; const { add } =
require('./math');
```

13 CommonJS vs ES modules?

CommonJS: require(), module.exports, synchronous loading, supported everywhere in Node. ESM: import/export, async loading, static analysis (tree-shaking), use .mjs or 'type: module' in package.json. Node 12+ supports ESM.

14 require vs import?

require: CommonJS, synchronous, runtime evaluated, dynamic (can use in conditionals). import: ESM, static (compile-time), supports tree-shaking, top-level only (use dynamic import() for conditional).

15 What is package.json?

package.json is the manifest file for Node projects. Contains: name, version, scripts (npm run), dependencies (production), devDependencies (development), main entry point, type (commonjs/module), engines.

Example / Code:

```
{ "name": "my-app", "scripts": { "start": "node index.js", "dev": "nodemon
index.js" }, "dependencies": { "express": "^4.18.0" }, "devDependencies": {
"jest": "^29.0.0" } }
```

16 What is npm?

npm (Node Package Manager) is the default package manager for Node.js. It manages installing, publishing, and versioning packages. The npm registry hosts 2M+ packages. Commands: npm install, npm start, npm run, npm publish.

17 npm vs npx?

npm: installs packages (globally or locally). npx: executes npm packages without global install. npx runs the latest version from the registry. Used for: create-react-app, eslint, ts-node, etc.

Example / Code:

```
npx create-react-app myapp # no global install needed npx eslint src/
```

18 What is yarn/pnpm?

Alternatives to npm. Yarn: faster installs (parallel, cache), workspaces, deterministic installs (yarn.lock). pnpm: fastest, saves disk (hard links), strict (no phantom dependencies), ideal for monorepos.

19 What is global vs local package?

Local (-S or default): installed in node_modules/, available only in project. Global (-g): installed in system directory, available as CLI command anywhere. Best practice: install locally for project deps, globally only for CLI tools.

20 What is __dirname?

__dirname is the absolute path of the directory containing the currently executing script file. Available in CommonJS modules. In ESM, use import.meta.url with new URL(', import.meta.url).pathname instead.

Example / Code:

```
console.log(__dirname); // e.g., /home/user/projects/myapp/src
```

21 What is __filename?

__filename is the absolute path of the currently executing script file, including the filename.

Example / Code:

```
console.log(__filename); // e.g., /home/user/projects/myapp/src/index.js
```

22 What is the process object?

process is a global object providing information about, and control over, the current Node.js process: env (environment variables), argv (command line args), exit(), cwd(), pid, platform, version, stdout, stderr, stdin.

Example / Code:

```
console.log(process.env.NODE_ENV); // 'production' console.log(process.argv); //
['node', 'script.js', ...args] process.exit(1); // exit with error code
```

23 process.nextTick?

process.nextTick(callback) queues a callback to run at the END of the current operation, before the event loop continues to the next phase. It has higher priority than Promise microtasks in some contexts and always runs before I/O callbacks.

Example / Code:

```
console.log('1'); process.nextTick(() => console.log('2 - nextTick'));
Promise.resolve().then(() => console.log('3 - promise')); console.log('4'); //
Output: 1, 4, 2, 3
```

24 setImmediate vs setTimeout in Node?

setTimeout(fn, 0): scheduled for the timers phase (1ms+). setImmediate: scheduled for the check phase (after I/O). Inside I/O callbacks: setImmediate always fires before setTimeout(fn, 0). Outside I/O: order is non-deterministic.

25 What is Buffer?

Buffer is a Node.js class for handling binary data (before TypedArray existed). Fixed-size raw memory allocation outside V8 heap. Used for file I/O, TCP streams, crypto.

Example / Code:

```
const buf = Buffer.from('Hello', 'utf8'); console.log(buf); // <Buffer 48 65 6c 6c 6f> console.log(buf.toString()); // 'Hello'
```

26 What are streams?

Streams are abstract interfaces for working with streaming data in Node.js. Instead of loading all data at once, streams process data in chunks. More memory efficient for large files or network data.

Example / Code:

```
// Pipe readable stream to writable
fs.createReadStream('input.txt').pipe(fs.createWriteStream('output.txt'));
```

27 Types of streams?

1) Readable — can read from (fs.createReadStream, http.IncomingMessage). 2) Writable — can write to (fs.createWriteStream, http.ServerResponse). 3) Duplex — both (TCP socket). 4) Transform — duplex that modifies data (zlib.createGzip).

28 What is pipe?

stream.pipe(destination) connects a readable stream to a writable stream, automatically managing backpressure. Replaces manual 'data' event listening and write() calls.

Example / Code:

```
const { pipeline } = require('stream/promises'); await pipeline(
fs.createReadStream('file.txt'), zlib.createGzip(),
fs.createWriteStream('file.txt.gz') );
```

29 What is EventEmitter?

EventEmitter is Node's built-in class for implementing event-driven architecture. Objects emit named events; listeners subscribe. Foundation for streams, http, fs.

Example / Code:

```
const EventEmitter = require('events'); const emitter = new EventEmitter();
emitter.on('data', (payload) => console.log('Received:', payload));
emitter.emit('data', { value: 42 });
```

30 How events work in Node.js?

Events follow the observer pattern. EventEmitter maintains a registry of event names -> array of listeners. emit(event, data) synchronously calls all listeners in order. once() registers a one-time listener. removeListener() unsubscribes.

■ Event Loop & Async (Q31–Q60)

31 What is the Node.js event loop?

The event loop is the mechanism that allows Node.js to perform non-blocking I/O. It continuously checks phases: timers -> pending callbacks -> idle/prepare -> poll -> check -> close callbacks. Runs until no more work remains.

32 Phases of event loop?

1) Timers: setTimeout, setInterval callbacks. 2) Pending callbacks: I/O error callbacks. 3) Idle/Prepare: internal use. 4) Poll: retrieve new I/O events (blocks here if queue empty). 5) Check: setImmediate callbacks. 6) Close: socket.destroy, etc.

33 Microtask queue vs callback queue?

Microtasks (process.nextTick, Promises) run between each event loop phase — they are not part of the loop phases. After each phase (and before the next), ALL microtasks are drained. process.nextTick runs before Promises.

34 process.nextTick vs Promise?

Both are microtasks, but nextTick callbacks run BEFORE Promise callbacks. nextTick has higher priority. Use nextTick for critical before-next-tick operations; Promises for general async.

Example / Code:

```
Promise.resolve().then(() => console.log('Promise')); process.nextTick(() =>
console.log('nextTick')); // Output: nextTick, Promise
```

35 setImmediate vs setTimeout in event loop?

In the main script (outside I/O): order is non-deterministic (depends on OS). Inside I/O callback: setImmediate always comes first (it's in the check phase, right after the poll phase that triggered the I/O callback).

36 What happens in each loop phase?

Timers phase: executes expired timer callbacks. Poll phase: block for new I/O (up to a timeout), process I/O callbacks. If poll queue empty and setImmediate pending -> move to Check. Check phase: run setImmediate callbacks. Close phase: close event callbacks.

37 How Node handles concurrency?

Node's main thread runs JS and the event loop. Async I/O is delegated to libuv's thread pool or OS async APIs. When I/O completes, callbacks are placed in the event loop queue. Main thread processes them sequentially — no locking needed.

38 What is libuv?

libuv is a multi-platform C library providing Node.js with: event loop, async I/O, file system, DNS, thread pool, timers, signal handling. It abstracts OS-level async APIs (epoll on Linux, kqueue on macOS, IOCP on Windows).

39 Thread pool in Node.js?

libuv maintains a thread pool (UV_THREADPOOL_SIZE, default 4) for operations not supported by OS async APIs: file system, DNS lookups, crypto, zlib. Network I/O uses OS-level async (doesn't use thread pool).

40 Default thread pool size?

UV_THREADPOOL_SIZE defaults to 4. Can be increased up to 128: UV_THREADPOOL_SIZE=8 node server.js. Increase when CPU-bound tasks or many concurrent file operations saturate the pool.

41 How are async operations executed?

1) JS calls async function (fs.readFile). 2) libuv dispatches to thread pool or OS async API. 3) Main thread continues. 4) On completion, callback is placed in event loop queue. 5) Event loop picks it up when phase is reached.

42 What is a callback?

A callback is a function passed to another function to be called when an async operation completes. Standard Node.js pattern: (error, result) => {} (error-first callbacks).

43 Callback hell in Node?

Deeply nested callbacks for sequential async operations. Hard to read, debug, and handle errors.

Example / Code:

```
readFile('a', (err, a) => { readFile('b', (err, b) => { writeFile('c', a+b, (err)
=> { // deeply nested }); }); }); // Fix: use async/await
```

44 What are promises in Node?

Promises in Node.js work the same as in browsers. Node.js core APIs have promisified versions in util.promisify() or the 'fs/promises' module. Node 15+ throws on unhandled rejections by default.

45 Promise chaining in Node?

Sequential async operations without callback nesting. Return a promise from .then() to chain the next operation.

Example / Code:

```
fs.promises.readFile('input.txt', 'utf8') .then(data => data.toUpperCase())
.then(upper => fs.promises.writeFile('output.txt', upper)) .catch(console.error);
```

46 async/await in Node?

async/await works in Node.js (supports since Node 7.6). Makes async code readable. Top-level await available in ESM (Node 14.3+).

Example / Code:

```
async function copyFile(src, dest) { const data = await fs.promises.readFile(src,
'utf8'); await fs.promises.writeFile(dest, data); console.log('Done'); }
```

47 Error handling in async code?

Use try/catch with async/await. Handle all rejected Promises — unhandled rejections crash the process in Node 15+. Add process.on('unhandledRejection', handler) as a safety net.

48 try/catch limitations in Node?

try/catch only catches synchronous errors and async errors in async functions (with await). It does NOT catch errors in callbacks or in setTimeout/setImmediate callbacks outside async functions.

Example / Code:

```
// Won't catch: try { setTimeout(() => { throw new Error('async'); }, 0); }
catch(e) { console.log(e); } // missed!
```

49 What is unhandled rejection?

When a Promise rejects and no .catch() or try/catch handles it. In Node 15+, unhandled rejections terminate the process. Listen to process.on('unhandledRejection') as a global handler.

Example / Code:

```
process.on('unhandledRejection', (reason, promise) => { console.error('Unhandled
rejection:', reason); process.exit(1); });
```

50 How to avoid blocking the event loop?

1) Use async fs operations, never Sync versions. 2) Break heavy computation into chunks (setImmediate between chunks). 3) Offload CPU work to Worker Threads. 4) Avoid regex catastrophic backtracking. 5) Don't use JSON.parse/stringify for huge objects synchronously.

51 CPU-intensive task handling in Node?

Options: 1) Worker Threads (Node 12+) — real parallel execution. 2) Child Process (fork) — separate process. 3) Cluster module — multi-process. 4) Native addons (N-API). 5) External service (Python, Go) via HTTP.

Example / Code:

```
const { Worker } = require('worker_threads'); const w = new
Worker('./compute.js', { workerData: { n: 1000000 } }); w.on('message', result =>
console.log(result));
```

52 Worker threads in Node?

worker_threads module (Node 10.5+, stable 12+) provides true multi-threading with shared memory (SharedArrayBuffer). Workers run in separate V8 instances but share memory. Communicate via postMessage or shared buffers.

53 Cluster module?

cluster creates multiple Node.js processes (workers) sharing the same port. Each worker handles requests independently. Master process balances connections. Utilizes multiple CPU cores.

Example / Code:

```
const cluster = require('cluster'); if(cluster.isMaster) { for(let i = 0; i < os.cpus().length; i++) cluster.fork(); } else { app.listen(3000); // each worker listens }
```

54 Child process?

child_process module creates external processes: spawn (streaming I/O), exec (buffered output, shell), execFile (no shell), fork (Node child with IPC). Used for: running shell commands, CPU-heavy tasks in separate process.

55 fork vs spawn vs exec?

spawn: streams stdout/stderr, good for large output, no shell. exec: buffers all output, good for small output, uses shell. fork: special spawn for Node.js scripts with IPC channel built in (child.send(), process.on('message')).

56 IPC in Node?

Inter-Process Communication between parent and forked child. Fork creates an IPC channel. parent.send(msg)/child.process.send(msg). Also: SharedArrayBuffer for threads, unix sockets, Redis pub/sub for cluster communication.

57 What is backpressure?

Backpressure occurs when a writable stream can't consume data as fast as a readable stream produces it. Unmanaged: memory grows, crash. Node.js streams handle backpressure via write() returning false and 'drain' event. pipe() handles this automatically.

58 Stream buffering?

When a writable stream signals it's full (write() returns false), the readable stream should pause and wait for the 'drain' event. Streams have an internal buffer (highWaterMark). Exceeding it causes backpressure.

59 HighWaterMark?

highWaterMark is the soft buffer size limit for streams (default 16KB for byte streams, 16 objects for object mode streams). When internal buffer exceeds highWaterMark, write() returns false (backpressure signal).

60 How Node handles large data?

Streams: process data chunk by chunk without loading entirely into memory. Piping: compose streams efficiently. Generator functions: pull-based iteration. For files: fs.createReadStream instead of fs.readFile.

■ Core Modules (Q61–Q90)

61 What is the fs module?

fs (File System) module provides APIs to interact with the file system: read, write, delete, rename, mkdir files and directories. Offers both synchronous (blocking) and asynchronous (callback/promise) versions.

Example / Code:

```
const fs = require('fs/promises'); const data = await fs.readFile('file.txt', 'utf8'); await fs.writeFile('out.txt', data.toUpperCase());
```

62 fs sync vs async?

Sync (readFileSync): blocks event loop until complete. Simple but unsuitable for servers. Async (readFile, fs.promises): non-blocking, callback or promise-based. Always use async in production servers.

63 readFile vs createReadStream?

readFile: loads entire file into memory — simple but uses more memory for large files. createReadStream: streams data in chunks — memory efficient for large files, supports backpressure.

Example / Code:

```
// Large file processing fs.createReadStream('huge.csv') .pipe(csvParser())
.pipe(processTransform) .pipe(fs.createWriteStream('output.json'));
```

64 What is the path module?

path provides utilities for working with file and directory paths in a cross-platform way. Handles OS differences (/ vs \).

Example / Code:

```
const path = require('path'); console.log(path.join(__dirname, 'public',
'index.html')); console.log(path.extname('file.txt')); // '.txt'
console.log(path.basename('/foo/bar.html')); // 'bar.html'
```

65 path.join vs path.resolve?

path.join(...parts) joins path segments with OS separator — relative path if first part is relative. path.resolve(...parts) builds an absolute path, processing each segment left to right, treating each absolute part as root.

Example / Code:

```
path.join('/foo', 'bar', 'baz') // '/foo/bar/baz' path.resolve('/foo', '/bar') //
'/bar' (resets at absolute)
```

66 What is the http module?

http provides low-level tools to create HTTP servers and make HTTP requests. Foundation for frameworks like Express.

Example / Code:

```
const http = require('http'); http.createServer((req, res) => {
res.writeHead(200, {'Content-Type': 'application/json'});
res.end(JSON.stringify({ message: 'Hello' })); }).listen(3000);
```

67 How to create a server in Node?

Use http.createServer(callback). The callback receives IncomingMessage (req) and ServerResponse (res). Call res.end() or res.write() to send response. Listen on a port.

68 What is the request/response lifecycle in Node?

1) Client sends HTTP request. 2) Node's HTTP server receives it, creates req/res objects. 3) Server callback is invoked. 4) Business logic runs (middleware chain in Express). 5) Response sent via res.end(). 6) Connection kept alive or closed.

69 What is the URL module?

url module provides utilities for URL parsing. new URL(href, base) creates a URL object with searchParams, pathname, hostname, protocol. Replaced the older url.parse() API.

Example / Code:

```
const { URL } = require('url'); const myUrl = new
URL('https://example.com/path?name=Alice&age=30');
console.log(myUrl.searchParams.get('name')); // 'Alice'
```

70 What is querystring?

querystring (deprecated, use URLSearchParams) provides utilities to parse and stringify URL query strings.

Example / Code:

```
const qs = require('querystring'); const parsed = qs.parse('name=Alice&age=30');
// { name: 'Alice', age: '30' } console.log(qs.stringify({ city: 'Mumbai' })); //
'city=Mumbai'
```

71 What is the crypto module?

crypto provides cryptographic functions: hashing (SHA256, MD5), HMAC, encryption (AES), key generation, random bytes, digital signatures.

Example / Code:

```
const { createHash, randomBytes } = require('crypto'); const hash =
createHash('sha256').update('password').digest('hex'); const salt =
randomBytes(16).toString('hex');
```

72 What is hashing?

Hashing is a one-way transformation of data into a fixed-length digest. Same input always produces same output; different input rarely produces same output (collision). Used for: password storage (bcrypt), data integrity (SHA256), cache keys.

73 What is compression in Node?

zlib module provides gzip/deflate/brotli compression and decompression. Express uses compression middleware (gzip) to compress HTTP responses — reduces payload size 70-90% for text/JSON.

74 zlib module?

zlib provides streams for compressing and decompressing data (gzip, deflate, br). Use createGzip() for HTTP response compression, gunzip() for decompression.

Example / Code:

```
const zlib = require('zlib'); fs.createReadStream('file.txt')
.pipe(zlib.createGzip()) .pipe(fs.createWriteStream('file.txt.gz'));
```

75 What is the OS module?

os provides operating system information: platform(), cpus() (CPU count/speed), totalmem(), freemem(), hostname(), tmpdir(), EOL (line ending).

Example / Code:

```
const os = require('os'); console.log(os.cpus().length); // number of CPU cores
console.log(os.freemem() / 1024**3 + ' GB free');
```

76 What is the cluster module?

cluster allows Node.js to create child processes (workers) that share the same port. Enables utilizing all CPU cores. Master process manages workers; each worker runs the server.

77 What is worker_threads?

worker_threads module creates OS threads within the same Node.js process, sharing memory via SharedArrayBuffer/Atomics. True parallel execution for CPU-bound tasks without the overhead of separate processes.

78 Difference: cluster vs worker_threads?

Cluster: multiple processes (isolated memory, more overhead, great for web servers). worker_threads: multiple threads (shared memory, less overhead, great for CPU-bound tasks like image processing). Cluster uses multiple Node instances; workers share one.

79 What is the net module?

net provides an async network API for creating TCP servers and clients. Foundation for the http module. Lower-level than http — work directly with TCP connections and data buffers.

80 What is TLS/HTTPS in Node?

tls module provides TLS/SSL-encrypted stream. https module uses tls to create HTTPS servers. Requires SSL certificate and private key. In production, HTTPS is typically terminated by Nginx/load balancer.

81 What is the DNS module?

dns module provides DNS resolution: dns.lookup (OS resolver), dns.resolve (network DNS). Useful for validating domains, resolving IPs, checking DNS records.

82 What are process signals?

Signals are OS-level notifications sent to processes. Node.js can listen to them via process.on(signal, fn).

Example / Code:

```
process.on('SIGTERM', () => { server.close(() => { console.log('Graceful
shutdown'); process.exit(0); }); });
```

83 SIGINT vs SIGTERM?

SIGINT: interrupt signal (Ctrl+C), typically triggers graceful shutdown. SIGTERM: termination signal from OS/process manager (Docker stop, systemd). Both should be handled for graceful shutdown. SIGKILL cannot be caught.

84 Graceful shutdown?

Graceful shutdown: 1) Stop accepting new connections (server.close()). 2) Wait for in-flight requests to complete. 3) Close DB connections, message queues. 4) Exit process. Prevents data loss and incomplete requests.

85 What is the timers module?

timers provides setTimeout, setInterval, setImmediate, clearTimeout, clearInterval, clearImmediate. Available globally in Node.js. timers/promises provides promise-based versions.

86 setInterval vs setTimeout?

setTimeout(fn, ms): runs once after ms milliseconds. setInterval(fn, ms): runs repeatedly every ms milliseconds. Always clear intervals when done to avoid memory leaks and unexpected behavior.

87 What is the console module?

console provides logging: log, error, warn, info, debug, table, time/timeEnd, group/groupEnd, assert, count. In Node, console.error writes to stderr; console.log to stdout.

88 Debugging in Node?

1) node --inspect or --inspect-brk. 2) Chrome DevTools (chrome://inspect). 3) VS Code debugger. 4) console.log. 5) node:debugger module. 6) Debug package for structured logging.

89 What is the util module?

util provides utility functions: util.promisify (convert callback APIs to Promises), util.inspect (deep object inspection), util.format (printf-style formatting), util.types (type checking).

90 util.promisify?

Converts an error-first callback function into one that returns a Promise.

Example / Code:

```
const { promisify } = require('util'); const readFileAsync =
promisify(fs.readFile); const data = await readFileAsync('file.txt', 'utf8');
```

■ Performance & Scaling (Q91–Q110)

91 How to scale Node.js app?

1) Cluster module (multi-process). 2) Horizontal scaling (multiple machines + load balancer). 3) Worker threads for CPU tasks. 4) Caching (Redis). 5) CDN for static assets. 6) Database connection pooling. 7) PM2 process manager.

92 Horizontal vs vertical scaling?

Vertical: add more resources (RAM, CPU) to one server. Limited by hardware. Horizontal: add more servers. Scales infinitely (with load balancer). Node.js is excellent for horizontal scaling (stateless, fast).

93 What is load balancing?

Load balancing distributes incoming requests across multiple server instances. Algorithms: round-robin, least connections, IP hash. Tools: Nginx, AWS ALB, HAProxy, Node cluster's built-in round-robin.

94 Reverse proxy?

A reverse proxy sits in front of Node.js servers: Nginx, Apache. Benefits: SSL termination, static file serving, load balancing, caching, compression, rate limiting, security headers. Node.js focuses on business logic.

95 Nginx with Node?

Nginx serves as reverse proxy: SSL termination, static assets, gzip compression, rate limiting. Passes dynamic requests to Node.js. Essential in production.

Example / Code:

```
# Nginx config location / { proxy_pass http://localhost:3000; proxy_http_version
1.1; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection
'upgrade'; }
```

96 What is clustering?

Node.js cluster spawns worker processes equal to CPU cores, each running the same server code. OS distributes connections across workers. No shared memory — use Redis for shared state.

97 Sticky sessions?

Sticky sessions (session affinity) route all requests from a user to the same server instance. Required if server stores session state in memory. Configured in load balancer. Better to use Redis for stateless sessions.

98 Stateless vs stateful apps?

Stateless: each request is self-contained (JWT auth). Any server can handle any request. Easy to scale horizontally. Stateful: server stores session data in memory. Requires sticky sessions or shared storage (Redis) for scaling.

99 Caching strategies in Node?

1) In-memory (Map, node-cache) — fast, lost on restart. 2) Redis — distributed, persistent, TTL support. 3) CDN caching for static. 4) HTTP caching headers (Cache-Control, ETag). 5) Database query caching.

100 Redis usage in Node?

Redis: session storage, response caching, rate limiting counters, pub/sub, job queues (BullMQ), leaderboards, distributed locks.

Example / Code:

```
const redis = require('ioredis'); const client = new redis(); await
client.set('key', 'value', 'EX', 3600); // expire in 1hr const val = await
client.get('key');
```

101 What is rate limiting?

Rate limiting restricts the number of requests a client can make in a time window. Prevents abuse, DDoS, and API overuse. Implement with express-rate-limit, or Redis (sliding window counter).

Example / Code:

```
const rateLimit = require('express-rate-limit'); app.use(rateLimit({ windowMs:
15*60*1000, max: 100 })); // 100 req/15min
```

102 What is throttling?

Throttling limits the processing rate (e.g., max 1000 requests/second globally). Different from rate limiting (per-user). Throttling protects the server; rate limiting protects per-user fairness.

103 How to optimize a Node API?

1) Async everything (no sync calls). 2) Connection pooling (DB, Redis). 3) Cache expensive queries. 4) Compress responses (gzip). 5) Pagination (limit large datasets). 6) Indexes on DB queries. 7) Profile with clinic.js or Node's built-in profiler.

104 Database connection pooling?

Pooling maintains a cache of database connections for reuse. Creating connections is expensive (TCP handshake, auth). Pool size should match thread pool/concurrency. Mongoose, pg, mysql2 all support connection pooling.

105 Memory leaks in Node?

Causes: 1) Uncleaned event listeners. 2) Closures holding references. 3) Global caches without TTL. 4) Circular references. 5) External libraries with leaks. Detect: --expose-gc + heapdump, clinic.js, node --inspect + Chrome Memory panel.

106 How to detect memory leaks in Node?

1) Monitor process.memoryUsage() over time. 2) clinic.js (heap profiler). 3) --inspect with Chrome heap snapshot. 4) heapdump + 0x tool. 5) Compare heap snapshots (growing retained heap = leak).

107 Profiling Node app?

1) node --prof (V8 profiler) + node --prof-process. 2) clinic.js (flame, bubbleprof, heapprofiler). 3) 0x (flame graphs). 4) New Relic / DataDog APM. 5) Chrome DevTools via --inspect.

108 Event loop lag?

Event loop lag is the delay between scheduling a callback and its actual execution. High lag = blocked event loop = slow responses. Monitor with clinic.js or custom: setInterval(() => { const lag = Date.now() - expected; }). Target < 10ms.

109 Logging best practices in Node?

1) Structured JSON logs (winston, pino). 2) Log levels (error, warn, info, debug). 3) Include: timestamp, request ID, user ID, error stack. 4) Different transports for different environments. 5) Centralize with ELK or Datadog.

110 Monitoring tools for Node?

1) PM2 (process management + basic monitoring). 2) clinic.js (performance diagnostics). 3) New Relic / DataDog (APM). 4) Prometheus + Grafana (metrics). 5) Sentry (error tracking). 6) CloudWatch (AWS).

■ Advanced & Internals (Q111–Q130)

111 How Node executes JS internally?

Source code -> V8 parsing -> AST -> Ignition bytecode -> (hot code) TurboFan machine code. Async operations dispatched to libuv. Callbacks queued via event loop.

112 What is call stack in Node?

Same as browser — LIFO structure tracking function execution contexts. Synchronous code runs here. Long synchronous operations block the call stack and the event loop.

113 What is heap in Node?

V8 heap is where objects, strings, and closures are allocated. Divided into young generation (new objects, fast GC) and old generation (long-lived objects, slower GC). Monitored via process.memoryUsage().heapUsed.

114 Garbage collection in Node?

V8 uses generational GC: Scavenger (young gen, fast, frequent) and Mark-Compact (old gen, slower, occasional). GC pauses the JS thread momentarily. GC tuned via --max-old-space-size flag.

115 Mark and sweep in Node?

Young gen uses Cheney's copying algorithm (fast). Old gen uses mark-compact: mark all reachable objects, compact remaining (no fragmentation). Concurrent marking (background thread) reduces pause times.

116 What is closure in Node?

Same as browser JavaScript. Functions capture variables from their surrounding scope. In Node, closures on EventEmitter callbacks, streams, and server handlers are common sources of memory leaks if not managed.

117 How modules are cached in Node?

Node caches required modules in require.cache after first load. Subsequent require() calls return cached module — no re-execution. Delete require.cache[key] to force re-evaluation (testing, hot reload).

118 require cache?

require.cache is an object mapping absolute module paths to their module exports. Cache prevents re-execution overhead. Circular dependencies are handled via partial exports (node returns what's exported so far).

119 What is ESM loader in Node?

The ESM loader handles import/export syntax in Node.js (.mjs files or 'type:module' in package.json). Async, supports top-level await, static analysis. Different from CommonJS's synchronous loader.

120 Dynamic imports in Node?

import() function works in Node.js ESM. Also available in CommonJS via async import. Useful for conditionally loading ESM modules from CJS context.

Example / Code:

```
const mod = await import('./module.mjs'); console.log(mod.default);
```

121 What is top-level await?

Top-level await allows using await at the top level of ESM modules (outside async functions). Not available in CommonJS. Useful for async initialization (DB connection, config loading).

Example / Code:

```
// module.mjs const config = await loadConfig(); export const db = await
connectDB(config);
```

122 What is process lifecycle?

1) Start: parse arguments, load modules. 2) Event loop runs: process events. 3) process.exit() or no more work. 4) 'exit' event fires. 5) Process terminates. Hooks: process.on('exit'), process.on('beforeExit').

123 What is the domain module (deprecated)?

domains were an early attempt at catching async errors. Deprecated in Node 4 because of complex semantics. Use try/catch + async/await + proper error propagation instead.

124 What is AsyncLocalStorage?

AsyncLocalStorage (Node 12.17+) provides request-scoped storage without passing context through every function call. Similar to thread-local storage in other languages. Used for: request ID, auth context, tracing spans.

Example / Code:

```
const store = new AsyncLocalStorage(); app.use((req, res, next) => { store.run({
requestId: uuid() }, next); }); // Anywhere in the async call chain: const ctx =
store.getStore(); // { requestId: '...' }
```

125 What is diagnostics_channel?

diagnostics_channel (Node 15+) provides a publish/subscribe channel for diagnostic messages. Libraries can publish events (HTTP requests, DB queries) that monitoring tools subscribe to without tight coupling.

126 What is the inspector protocol?

The inspector protocol (V8 Inspector) enables DevTools debugging and profiling. Activated with --inspect. Supports: breakpoints, heap snapshots, CPU profiling, real-time evaluation.

127 What is a native addon?

Native addons are Node.js modules written in C/C++ compiled to .node files. They can call OS-level APIs, use C libraries, or perform CPU-heavy tasks faster than JS.

Example / Code:

```
const addon = require('./build/Release/addon.node');
```

128 What is N-API?

N-API (now node-api) is a stable ABI-stable C API for building native addons. Unlike older NAN, N-API addons work across Node.js versions without recompilation.

129 What is edge runtime?

Edge runtime is a lightweight JS runtime (based on V8/Web APIs, not full Node.js) for executing code at CDN edge nodes (Vercel Edge, Cloudflare Workers). Faster cold starts, closer to user, but no Node.js APIs (no fs, no Buffer).

130 When NOT to use Node.js?

1) CPU-intensive tasks (image processing, ML, scientific computing) — Python/Go better. 2) Applications requiring heavy parallel computation. 3) When team expertise is in Java/.NET for enterprise systems. 4) Real-time data-heavy apps with complex DB operations where blocking model is simpler.

Part 4
Express.js
80 Questions — Middleware to Production

■ Basics (Q1–Q25)

1 What is Express.js?

Express is a minimal, fast, unopinionated web framework for Node.js. It provides routing, middleware support, HTTP utilities, and template engine integration. Most popular Node.js framework.

Example / Code:

```
const express = require('express'); const app = express(); app.get('/', (req,
res) => res.json({ message: 'Hello World' })); app.listen(3000);
```

2 Why use Express?

1) Minimal boilerplate. 2) Robust routing. 3) Middleware ecosystem. 4) Template engines. 5) Error handling. 6) Huge npm ecosystem. 7) Active community. 8) Foundation for frameworks like NestJS.

3 Express vs Node HTTP?

Node http: low-level, manual routing, raw request parsing. Express: routing via app.get/post/put, automatic request body parsing, middleware chain, helper methods (res.json, res.send), cleaner API.

4 What is middleware?

Middleware functions have access to req, res, and next(). They execute in order and can: modify req/res, end the request, or call next() to pass control to the next middleware. Foundation of Express architecture.

Example / Code:

```
app.use((req, res, next) => { console.log(`${req.method} ${req.url}`); next(); //
pass to next middleware });
```

5 Types of middleware?

1) Application-level (app.use). 2) Router-level (router.use). 3) Error-handling (4 args: err, req, res, next). 4) Built-in (express.json, express.static). 5) Third-party (cors, morgan, helmet).

6 What is next()?

next() is a function passed to each middleware. Calling next() passes control to the next middleware in the stack. next(err) passes an error to the error-handling middleware (skips all regular middleware).

7 Request vs response object?

req: contains request data — headers, body, params, query, cookies, user. res: response builder — json(), send(), status(), redirect(), sendFile(), header(), cookie().

8 What is routing in Express?

Routing determines how the app responds to client requests at specific paths and HTTP methods. Express provides app.get(), post(), put(), delete(), patch(), and all().

9 Route params vs query params?

Route params (:id) are part of the URL path. Query params (?key=val) are after ?. Route params: req.params.id. Query params: req.query.key.

Example / Code:

```
// Route params app.get('/users/:id', (req, res) => res.json(req.params)); // { id: '42' } // Query params // GET /search?q=node&limit=10 app.get('/search', (req, res) => res.json(req.query)); // { q: 'node', limit: '10' }
```

10 What is app.use?

app.use(path?, middleware) mounts middleware at the specified path (or all paths if omitted). It matches any HTTP method and any sub-path of the specified path.

Example / Code:

```
app.use(express.json()); // all routes app.use('/api', apiRouter); // only /api/* routes
```

11 What is app.get/post/etc?

HTTP method-specific route handlers. Only match the specific method.

Example / Code:

```
app.get('/users', getAllUsers); app.post('/users', createUser); app.put('/users/:id', updateUser); app.delete('/users/:id', deleteUser);
```

12 What is Express Router?

express.Router() creates a mini Express app — a modular collection of routes and middleware. Enables separation of concerns by feature/resource.

Example / Code:

```
const router = express.Router(); router.get('/', getUsers); router.post('/', createUser); module.exports = router; // In app.js: app.use('/api/users', userRouter);
```

13 What is express.json()?

Built-in middleware that parses incoming requests with JSON payload (Content-Type: application/json) and populates req.body.

Example / Code:

```
app.use(express.json()); // parses JSON body app.post('/data', (req, res) => { console.log(req.body); // { name: 'Alice' } });
```

14 What is body-parser?

body-parser (now built into Express 4.16+) parses incoming request bodies: JSON, URL-encoded, multipart. express.json() = body-parser.json().

15 What is express.static?

express.static(root) serves static files (HTML, CSS, JS, images) from a directory. Must be mounted before dynamic routes.

Example / Code:

```
app.use('/public', express.static(path.join(__dirname, 'public'))); // GET /public/logo.png -> serves /public/logo.png
```

16 What is CORS in Express?

CORS headers allow cross-origin requests. Use the cors npm package to configure allowed origins, methods, headers.

Example / Code:

```
const cors = require('cors'); app.use(cors({ origin: 'https://myapp.com', methods: ['GET','POST'], credentials: true }));
```

17 How to enable CORS?

Install cors package. app.use(cors()) for all origins (dev). Configure origin, methods, credentials for production.

Example / Code:

```
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true, methods:
'GET,POST,PUT,DELETE' }));
```

18 What is error middleware?

Error-handling middleware has 4 parameters: (err, req, res, next). Called when next(err) is called or an error is thrown in async code (with express-async-errors or try/catch + next(err)).

Example / Code:

```
app.use((err, req, res, next) => { const status = err.statusCode || 500;
res.status(status).json({ success: false, message: err.message || 'Internal
Server Error' }); });
```

19 How does error handling work in Express?

Synchronous errors in route handlers are automatically caught. Async errors must be: 1) Caught with try/catch and passed to next(err). 2) Use express-async-errors package (wraps all routes). 3) Use next(err) pattern.

20 What is a 404 handler?

A catch-all route (placed after all routes) that sends 404 for unmatched paths.

Example / Code:

```
app.use('*', (req, res) => { res.status(404).json({ message: 'Route not found'
}); });
```

21 What is REST API?

REST (Representational State Transfer) is an architectural style for APIs. Principles: stateless, uniform interface (nouns as resources), HTTP methods (GET/POST/PUT/DELETE), representations (JSON/XML), cacheability.

22 REST vs GraphQL?

REST: multiple endpoints per resource, over/under fetching possible, simpler caching, widely understood. GraphQL: single endpoint, client specifies exact data needed, no over-fetching, more complex caching, better for complex data graphs.

23 What is MVC pattern in Express?

Model-View-Controller separates concerns: Model (data + business logic, DB), View (response formatting, JSON in APIs), Controller (handles req/res, orchestrates model). Keeps code organized and testable.

24 What is API versioning?

API versioning maintains backward compatibility while evolving the API. Strategies: 1) URL versioning (/api/v1, /api/v2). 2) Header versioning (Accept-Version: v1). 3) Query param (?version=1). URL versioning is most common.

Example / Code:

```
app.use('/api/v1', v1Router); app.use('/api/v2', v2Router);
```

25 What is environment config?

Use dotenv for environment-specific configuration. Never hardcode secrets. Different .env files for dev/staging/prod. Use a config module to validate and expose env vars.

Example / Code:

```
require('dotenv').config(); const PORT = process.env.PORT || 3000; const DB_URI =
process.env.MONGODB_URI; if(!DB_URI) throw new Error('DB_URI required');
```

■ Advanced Middleware & Patterns (Q26–Q55)

26 How does middleware chain work?

Each middleware calls next() to pass to the next. Order matters — express.json() before routes that need body. Error middleware last. A middleware can end the chain (not call next()) to short-circuit.

Example / Code:

```
app.use(logRequest); app.use(authenticate); app.use(authorize); app.get('/data',
handler); app.use(errorHandler); // last
```

27 What is async middleware?

Middleware with async operations. Must use try/catch + next(err) or a wrapper to catch errors.

Example / Code:

```
// Pattern 1: try/catch app.get('/users', async (req, res, next) => { try { const
users = await User.find(); res.json(users); } catch(err) { next(err); } }); //
Pattern 2: asyncHandler wrapper const asyncHandler = fn => (req, res, next) =>
fn(req, res, next).catch(next); app.get('/users', asyncHandler(async (req, res)
=> { const users = await User.find(); res.json(users); }));
```

28 How to handle errors in async middleware?

Use try/catch and pass error to next(err). Or use express-async-errors which patches Express to automatically handle Promise rejections. Error middleware (4 args) catches all.

29 What is helmet?

helmet sets various security HTTP headers to protect Express apps: X-Frame-Options, X-Content-Type-Options, HSTS, CSP, etc. One line of security.

Example / Code:

```
const helmet = require('helmet'); app.use(helmet()); // sets 11 security headers
```

30 What is morgan?

morgan is an HTTP request logger middleware. Logs request method, URL, status, response time.

Example / Code:

```
const morgan = require('morgan'); app.use(morgan('combined')); // Apache combined
log format app.use(morgan(':method :url :status :response-time ms'));
```

31 What is rate limiter in Express?

express-rate-limit restricts repeated requests from the same IP.

Example / Code:

```
const rateLimit = require('express-rate-limit'); app.use('/api/', rateLimit({
windowMs: 15 * 60 * 1000, // 15 minutes max: 100, message: 'Too many requests'
}));
```

32 What is compression middleware?

compression gzip-compresses HTTP responses, reducing bandwidth.

Example / Code:

```
const compression = require('compression'); app.use(compression({ threshold: 1024
})); // compress responses > 1KB
```

33 What is cookie-parser?

cookie-parser parses Cookie header and populates req.cookies. Works with res.cookie() for setting cookies.

Example / Code:

```
const cookieParser = require('cookie-parser');
app.use(cookieParser(process.env.COOKIE_SECRET)); // Access:
req.cookies.sessionId / req.signedCookies.token
```

34 What is session in Express?

express-session manages server-side sessions. Stores session data on server; sends session ID cookie to client. For production, use Redis store (connect-redis) instead of memory store.

Example / Code:

```
const session = require('express-session'); app.use(session({ secret: 'key',
resave: false, saveUninitialized: false })); // req.session.userId = user.id;
```

35 What is JWT middleware?

Middleware that verifies JWT tokens on protected routes.

Example / Code:

```
function authenticate(req, res, next) { const token =
req.headers.authorization?.split(' ')[1]; if(!token) return
res.status(401).json({ message: 'No token' }); try { req.user = jwt.verify(token,
process.env.JWT_SECRET); next(); } catch(e) { res.status(401).json({ message:
'Invalid token' }); } }
```

36 How does authentication work in Express?

1) POST /auth/login -> verify credentials -> generate JWT -> send token. 2) Subsequent requests: send JWT in Authorization header. 3) Auth middleware verifies JWT -> attaches user to req. 4) Protected routes use auth middleware.

37 What is authorization in Express?

Authorization checks if the authenticated user has permission to perform an action. Comes after authentication. Role-based (admin, user), resource-based (owner), or permission-based (can_edit_post).

Example / Code:

```
function requireRole(role) { return (req, res, next) => { if(req.user.role !==
role) return res.status(403).json({ message: 'Forbidden' }); next(); }; }
app.delete('/users/:id', authenticate, requireRole('admin'), deleteUser);
```

38 Role-based access in Express?

Store user roles in DB. Include role in JWT payload. Create middleware factory that checks role. Apply to routes.

Example / Code:

```
app.get('/admin', authenticate, authorize('admin'), adminController);
app.get('/profile', authenticate, authorize('user', 'admin'), profileController);
```

39 What is file upload in Express?

Use multer middleware for multipart/form-data file uploads. Supports: disk storage, memory storage, file filtering, size limits.

Example / Code:

```
const multer = require('multer'); const upload = multer({ dest: 'uploads/',
limits: { fileSize: 5 * 1024 * 1024 } }); app.post('/upload',
upload.single('file'), (req, res) => { res.json({ filename: req.file.filename });
});
```

40 Multer usage?

multer processes multipart form data. upload.single('field') for one file. upload.array('files', 5) for multiple. upload.fields([...]) for different fields.

Example / Code:

```
const storage = multer.diskStorage({ destination: 'uploads/', filename: (req,
file, cb) => cb(null, Date.now() + '-' + file.originalname) }); const upload =
multer({ storage, fileFilter: (req, file, cb) => {
```

```
if(!file.mimetype.startsWith('image/')) return cb(new Error('Images only'));
cb(null, true); }});
```

41 What is validation in Express?

Validate incoming request data (body, params, query) before processing. Libraries: express-validator, Joi, Zod. Always validate on the server — never trust client input.

Example / Code:

```
const { body, validationResult } = require('express-validator');
app.post('/users', body('email').isEmail(), body('password').isLength({ min: 8
}), (req, res) => { const errors = validationResult(req); if(!errors.isEmpty())
return res.status(422).json({ errors: errors.array() }); createUser(req.body); }
);
```

42 Joi vs Zod?

Joi: mature, JS-only, large community, verbose. Zod: TypeScript-first, schema types as TypeScript types, tree-shakeable, growing fast. Zod preferred in TypeScript projects.

Example / Code:

```
// Zod const schema = z.object({ email: z.string().email(), age:
z.number().min(18) }); const parsed = schema.parse(req.body); // throws on
validation failure
```

43 What is sanitization?

Sanitization cleans user input to prevent XSS and injection attacks: strip HTML tags, escape special characters, normalize whitespace. Use validator.js escape(), DOMPurify, or express-mongo-sanitize.

44 What is the request lifecycle in Express?

Client sends HTTP -> Node.js http module receives -> Express router matches path/method -> middleware chain executes (logger, auth, validation, business logic) -> response sent -> connection maintained or closed.

45 What is dependency injection in Express?

DI provides dependencies (DB, services) to handlers instead of hardcoding them. Makes code testable. In Express: via function factories, container (awilix), or module pattern.

46 What is the service layer?

Service layer contains business logic — separate from route handlers (controllers) and data access (repositories). Makes logic reusable and testable.

Example / Code:

```
// userService.js const createUser = async (data) => { await
validateUnique(data.email); const hash = await bcrypt.hash(data.password, 10);
return User.create({ ...data, password: hash }); }; // userController.js const
register = async (req, res, next) => { try { const user = await
createUser(req.body); res.status(201).json(user); } catch(e) { next(e); } };
```

47 What is the controller layer?

Controllers handle HTTP concerns: parse req, call service, send res. Thin — no business logic. Delegates to services.

Example / Code:

```
const getAllUsers = async (req, res, next) => { try { const { page = 1, limit = 20
} = req.query; const users = await userService.findAll({ page, limit });
res.json({ success: true, data: users }); } catch(err) { next(err); } };
```

48 What is repository pattern in Express?

Repository abstracts DB operations. Controller -> Service -> Repository -> DB. Allows swapping databases without changing business logic. Testable with mock repositories.

Example / Code:

```
class UserRepository { async findById(id) { return User.findById(id); } async create(data) { return User.create(data); } async update(id, data) { return User.findByIdAndUpdate(id, data, {new:true}); } }
```

49 What is modular routing?

Organizing routes by feature/resource in separate router files. Each feature has its own router, mounted in app.js. Keeps codebase organized.

Example / Code:

```
// /routes/users.js, /routes/orders.js, /routes/products.js app.use('/api/v1/users', require('./routes/users')); app.use('/api/v1/orders', require('./routes/orders')); app.use('/api/v1/products', require('./routes/products'));
```

50 What is API gateway?

API gateway is a server that's the single entry point for clients. Handles: routing to microservices, auth, rate limiting, SSL, logging. Examples: Kong, AWS API Gateway, custom Express gateway.

Example / Code:

```
// Express as simple API gateway app.use('/users', proxy('http://users-service:3001')); app.use('/orders', proxy('http://orders-service:3002'));
```

51 What is proxy middleware?

http-proxy-middleware forwards requests to other services. Used in dev (CRA's proxy) and as API gateway.

Example / Code:

```
const { createProxyMiddleware } = require('http-proxy-middleware'); app.use('/api', createProxyMiddleware({ target: 'http://backend:3000', changeOrigin: true }));
```

52 How to handle large payloads?

Increase body size limit in express.json/express.urlencoded. Use streaming for file uploads (multer with disk storage). Paginate large list responses.

Example / Code:

```
app.use(express.json({ limit: '10mb' })); app.use(express.urlencoded({ extended: true, limit: '10mb' }));
```

53 Streaming responses in Express?

For large data (file download, CSV export), stream the response instead of loading all into memory.

Example / Code:

```
app.get('/export', (req, res) => { res.setHeader('Content-Type', 'text/csv'); res.setHeader('Content-Disposition', 'attachment; filename=export.csv'); const stream = db.collection.find().stream(); stream.pipe(csvStringify()).pipe(res); });
```

54 Pagination implementation in Express?

Offset pagination: skip + limit. Cursor pagination: after an ID (more efficient for large datasets).

Example / Code:

```
// Offset pagination const page = parseInt(req.query.page) || 1; const limit = parseInt(req.query.limit) || 20; const skip = (page - 1) * limit; const [data, total] = await Promise.all([Model.find().skip(skip).limit(limit), Model.countDocuments()]); res.json({ data, page, total, pages: Math.ceil(total/limit) });
```

55 Filtering & sorting in Express?

Build query object from query params. Whitelist allowed filter fields.

Example / Code:

```
const { status, sort = '-createdAt', fields } = req.query; const query = {};
if(status) query.status = status; const result = await Order .find(query)
.sort(sort) .select(fields?.split(',').join(' ')) .skip(skip).limit(limit);
```

■ Performance & Security (Q56–Q80)

56 How to optimize Express app?

1) Use compression. 2) Cache DB queries. 3) Use async everywhere. 4) Cluster with PM2. 5) Redis for session/cache. 6) Connection pooling. 7) gzip responses. 8) Disable X-Powered-By. 9) Use CDN for static.

57 Avoid blocking code in Express?

Never use sync file/DB operations: fs.readFileSync, mongoose's callbacks used synchronously. Always async/await or callbacks. Move CPU tasks to worker threads.

58 How to scale Express?

PM2 cluster mode (all cores). Docker + Kubernetes. AWS ECS/EKS. Load balancer (Nginx, AWS ALB). Stateless architecture + Redis for shared state.

59 Load balancing with Express?

Deploy multiple Express instances. Put Nginx or AWS ALB in front. Nginx round-robin distributes requests. For WebSocket: use sticky sessions or Redis pub/sub.

60 Clustering in Express?

Use PM2 cluster mode (pm2 start app.js -i max) or Node.js cluster module. Each worker handles independent requests. No shared in-memory state — use Redis.

61 Caching responses in Express?

1) Cache-Control headers for browser/CDN. 2) Redis cache for DB query results. 3) ETags for conditional requests. 4) apicache or custom middleware.

62 ETag usage in Express?

Express sets ETags by default (res.setHeader auto). Client sends If-None-Match header; if ETag matches, Express returns 304 Not Modified (no body). Saves bandwidth.

Example / Code:

```
// Express enables ETags by default app.set('etag', 'strong'); // or 'weak'
```

63 Compression in production?

Use compression middleware or Nginx gzip. Brotli is better than gzip (20-30% smaller) — configure in Nginx.

64 Rate limiting strategies?

1) Fixed window (simple counter per window). 2) Sliding window (smoother). 3) Token bucket (bursty traffic). 4) Leaky bucket (smooth output). Use Redis for distributed rate limiting.

65 Prevent DDoS in Express?

1) Rate limiting (express-rate-limit). 2) Connection limiting (toobusy-js). 3) Nginx DDoS mitigation. 4) CDN (CloudFlare). 5) AWS Shield. 6) Limit request body size.

66 Input validation for security?

Validate ALL user inputs: type, length, format, range. Use Zod/Joi schema validation. Reject requests that don't match schema. Never trust client-side validation alone.

67 Prevent XSS in Express?

1) Escape output. 2) CSP headers (helmet). 3) httpOnly cookies. 4) Sanitize input. 5) DOMPurify for user-generated HTML.

Example / Code:

```
app.use(helmet.contentSecurityPolicy({ directives: { defaultSrc: ["'self'"],
scriptSrc: ["'self'"] } }));
```

68 Prevent CSRF in Express?

1) SameSite cookies. 2) CSRF tokens (csurf or double submit cookie). 3) Custom request headers (X-Requested-With). 4) Verify Origin/Referer headers.

69 Prevent SQL/NoSQL injection?

SQL: use parameterized queries / ORM (never string concatenation). MongoDB: use mongoose (sanitizes input), use $eq operator, sanitize with express-mongo-sanitize.

Example / Code:

```
// SQL (parameterized) const user = await pool.query('SELECT * FROM users WHERE
id = $1', [req.params.id]); // MongoDB const user = await
User.findById(sanitize(req.params.id));
```

70 Secure headers in Express?

Use helmet middleware — sets: X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, HSTS, Referrer-Policy, Content-Security-Policy.

Example / Code:

```
app.use(helmet()); // all headers app.use(helmet.hsts({ maxAge: 31536000 })); //
HSTS 1 year
```

71 HTTPS setup in Express?

In production, terminate TLS at Nginx or load balancer. In development, use https module with self-signed cert.

Example / Code:

```
const https = require('https'); const opts = { key: fs.readFileSync('key.pem'),
cert: fs.readFileSync('cert.pem') }; https.createServer(opts, app).listen(443);
```

72 Logging strategy in Express?

1) Request logs: morgan (or pino-http). 2) Application logs: winston or pino. 3) Error logs with stack traces. 4) Correlation ID per request. 5) Separate log levels. 6) Log to stdout, aggregate with ELK/Datadog.

73 Error logging in Express?

Log all errors in error middleware with full context: error message, stack, request info, user ID, timestamp.

Example / Code:

```
app.use((err, req, res, next) => { logger.error({ err, req: { method: req.method,
url: req.url, user: req.user?.id } }); res.status(err.status || 500).json({
message: err.message }); });
```

74 Monitoring Express app?

1) Prometheus metrics (prom-client). 2) Datadog/New Relic APM. 3) Health check endpoint (/health). 4) PM2 monitoring. 5) Sentry for errors. 6) Custom middleware for request duration

tracking.

75 Graceful shutdown in Express?

Stop accepting new connections, wait for in-flight requests to complete, close DB connections, exit.

Example / Code:

```
process.on('SIGTERM', () => { server.close(async () => { await
mongoose.disconnect(); await redis.quit(); process.exit(0); }); });
```

76 Handling crashes in Express?

1) PM2 auto-restart. 2) Docker restart policy. 3) Kubernetes liveness/readiness probes. 4) Uncaught exception handler (log + restart).

Example / Code:

```
process.on('uncaughtException', (err) => { logger.error('Uncaught exception:',
err); process.exit(1); // PM2 will restart });
```

77 Health checks in Express?

A /health endpoint returns app status: DB connection, Redis connection, service dependencies. Used by load balancers and Kubernetes to route traffic.

Example / Code:

```
app.get('/health', async (req, res) => { const dbOk =
mongoose.connection.readyState === 1; res.status(dbOk ? 200 : 503).json({ status:
dbOk ? 'OK' : 'ERROR', db: dbOk }); });
```

78 Circuit breaker in Express?

Circuit breaker prevents cascading failures. When a downstream service fails repeatedly, the circuit 'opens' and requests fail fast without trying. Libraries: opossum.

Example / Code:

```
const breaker = new CircuitBreaker(asyncFn, { timeout: 3000,
errorThresholdPercentage: 50, resetTimeout: 30000 }); breaker.on('open', () =>
logger.warn('Circuit opened!'));
```

79 Retry strategy in Express?

Retry failed HTTP requests with exponential backoff. Use axios-retry or custom implementation.

Example / Code:

```
const axiosRetry = require('axios-retry'); axiosRetry(axios, { retries: 3,
retryDelay: axiosRetry.exponentialDelay, retryCondition: err =>
err.response?.status >= 500 });
```

80 Express production best practices?

1) Use helmet. 2) Enable compression. 3) Rate limiting. 4) Graceful shutdown. 5) Structured logging. 6) Environment config (dotenv + validation). 7) Process manager (PM2). 8) Health checks. 9) HTTPS (via Nginx). 10) Monitor + alert.

Part 5 MongoDB

120 Questions — Schema Design to Scaling

■ Basics (Q1–Q30)

1 What is MongoDB?

NoSQL document database. Data stored as BSON documents in collections. Schema-less, horizontally scalable, rich aggregation.

Example / Code:

```
db.users.insertOne({name:'Alice',age:30,email:'a@b.com'});
db.users.find({age:{$gt:25}});
```

2 SQL vs NoSQL?

SQL: fixed schema, tables/rows, ACID, great for structured relational data. NoSQL: flexible schema, documents, horizontal scale, great for hierarchical data and high write throughput.

Example / Code:

```
//SQL: schema migration for every field addition //MongoDB: just add field to
document, no migration
```

3 What is a document?

BSON object with key-value pairs. Can have nested docs and arrays. Auto-generated _id (ObjectId).

Example / Code:

```
{ _id:ObjectId('...'), name:'Alice', address:{city:'Mumbai',pin:'400001'},
tags:['dev','react'], createdAt:new Date() }
```

4 What is a collection?

Group of related documents. Like SQL table. No schema enforcement by default (use Mongoose schema for enforcement).

Example / Code:

```
//Create collection implicitly: db.products.insertOne({name:'Laptop',price:999});
//Creates 'products' collection if not exists
```

5 What is BSON?

Binary JSON — MongoDB's storage format. Extends JSON: Date, Binary, ObjectId, Decimal128, Int32/64. More efficient than JSON text.

Example / Code:

```
//ObjectId: 12 bytes // 4 bytes: timestamp // 5 bytes: random (machine+process)
// 3 bytes: incrementing counter
```

6 What is _id?

Primary key of every document. Auto-generated as ObjectId if not provided. Can be any type but must be unique.

Example / Code:

```
const{ObjectId}=require('mongodb'); ObjectId().getTimestamp(); //creation
timestamp from ObjectId
```

7 What is schema-less?

Documents in same collection can have different fields. Enables flexible, evolving models. Use Mongoose schemas or JSON Schema validation for consistency.

Example / Code:

```
//Same collection, different structures: db.events.insertMany([
{type:'click',element:'button',x:100,y:200},
{type:'purchase',productId:'p1',amount:99.99,currency:'INR'} ]);
```

8 What is indexing?

Special B-tree data structures for fast queries. Without index: collection scan O(n). With index: O(log n).

Example / Code:

```
//Create index: db.users.createIndex({email:1}); //ascending
db.users.createIndex({email:1},{unique:true}); //unique //Check indexes:
db.users.getIndexes();
```

9 Types of indexes?

Single field, compound (multi-field), multikey (arrays), text (full-text search), geospatial, hashed (sharding), partial, sparse, TTL (auto-expire), wildcard.

Example / Code:

```
//Compound: db.orders.createIndex({userId:1,createdAt:-1}); //TTL:
db.sessions.createIndex({createdAt:1},{expireAfterSeconds:3600}); //Text:
db.articles.createIndex({title:'text',body:'text'});
```

10 Unique index?

Enforces uniqueness. Duplicate value throws E11000 DuplicateKeyError.

Example / Code:

```
db.users.createIndex({email:1},{unique:true}); //Mongoose: const schema=new
Schema({email:{type:String,unique:true}});
```

11 Compound index?

Index on multiple fields. {a:1,b:1} supports queries on (a) or (a,b) but NOT (b) alone. ESR rule: Equality, Sort, Range.

Example / Code:

```
//ESR rule - field order for compound index: //1. Equality fields (exact match)
//2. Sort fields //3. Range fields ($gt, $lt, $in)
db.orders.createIndex({userId:1,status:1,createdAt:-1});
```

12 Text index?

Full-text search on string fields.

Example / Code:

```
db.articles.createIndex({title:'text',body:'text'});
db.articles.find({$text:{$search:'mongodb performance'}});
db.articles.find({$text:{$search:'"exact phrase"'}});
```

13 CRUD in MongoDB?

Create: insertOne/insertMany. Read: find/findOne. Update: updateOne/updateMany. Delete: deleteOne/deleteMany.

Example / Code:

```
await User.create({name:'Alice'}); await User.findOne({email:'a@b.com'}); await
User.updateOne({_id:id},{$set:{name:'Bob'}}); await User.deleteOne({_id:id});
```

14 insertOne vs insertMany?

insertOne: single document. insertMany: multiple in one round-trip (much faster than looping). {ordered:false}: continues on error.

Example / Code:

```
//Fast bulk insert: await User.insertMany(users,{ordered:false}); //ordered:false
continues even if some fail
```

15 find vs findOne?

find: returns cursor (lazy, iterate or toArray). findOne: returns first match or null.

Example / Code:

```
const cursor=db.users.find({age:{$gte:18}}); const all=await cursor.toArray();
const one=await User.findOne({email:'a@b.com'}).lean();
```

16 updateOne vs updateMany?

updateOne: first matching document. updateMany: all matching. Always use operators ($set, $push, $inc) — without them document is REPLACED.

Example / Code:

```
//Update with operators (safe): await
User.updateOne({_id:id},{$set:{name:'Bob'},$inc:{loginCount:1}}); //Update all
inactive users: await
User.updateMany({lastLogin:{$lt:thirtyDaysAgo}},{$set:{status:'inactive'}});
```

17 deleteOne vs deleteMany?

deleteOne: first match. deleteMany: all matching. Test filter with find() first.

Example / Code:

```
//Clean expired sessions: await Session.deleteMany({expiresAt:{$lt:new Date()}});
//Soft delete instead: await User.updateOne({_id:id},{$set:{deletedAt:new
Date()}});
```

18 What is projection?

Control returned fields. 1=include, 0=exclude. Can't mix include/exclude (except _id).

Example / Code:

```
//Only name and email: db.users.find({},{name:1,email:1,_id:0}); //Mongoose:
await User.findById(id).select('name email -password');
```

19 Aggregation pipeline?

Framework for data transformation. Sequence of stages, each transforming documents. Uses indexes, supports disk, parallel execution.

Example / Code:

```
await Order.aggregate([ {$match:{status:'completed'}},
{$group:{_id:'$customerId',total:{$sum:'$amount'}}}, {$sort:{total:-1}},
{$limit:10} ]);
```

20 What is pipeline?

Array of aggregation stages. Documents pass through each stage. Each stage's output is next stage's input.

Example / Code:

```
//Pipeline flow: //All docs -> $match -> filtered docs -> $group -> grouped docs
-> $sort -> sorted
```

21 What is $match?

Filters documents (like find filter). Should be FIRST stage to use indexes and reduce data early.

Example / Code:

```
//First stage: use index, reduce data
{$match:{status:'active',createdAt:{$gte:startDate}}}
```

22 What is $group?

Groups documents by key, applies accumulators.
Example / Code:
```
{ $group:{ _id:'$category', count:{$sum:1}, avgPrice:{$avg:'$price'},
maxPrice:{$max:'$price'}, products:{$push:'$name'} } }
```

23 What is $lookup?

Left outer join to another collection.
Example / Code:
```
{ $lookup:{ from:'orders', localField:'_id', foreignField:'userId', as:'orders' }
}
```

24 What is $sort?

Sorts documents. 1=ascending, -1=descending.
Example / Code:
```
//Sort before $limit for top-N: {$sort:{totalAmount:-1,createdAt:1}}
```

25 What is $limit?

Restricts number of documents passed to next stage.
Example / Code:
```
//Top 10 customers: [ {$group:{_id:'$customerId',total:{$sum:'$amount'}}},
{$sort:{total:-1}}, {$limit:10} ]
```

26 What is $skip?

Skips first N documents. Offset pagination (avoid with large datasets — use cursor-based instead).
Example / Code:
```
//Page 3 with 20 items: {$skip:40} //skip (page-1)*limit
```

27 What is $unwind?

Deconstructs array — one document per element.
Example / Code:
```
//Input: {_id:1, tags:['js','react']} //Output: {_id:1,tags:'js'},
{_id:1,tags:'react'} {$unwind:'$tags'}
```

28 What is cursor?

Pointer to result set. Lazy — processes one chunk at a time. Doesn't load all at once.
Example / Code:
```
const cursor=User.find({active:true}).cursor(); for await(const doc of cursor){
await processUser(doc); }
```

29 What is explain()?

Shows query execution plan. IXSCAN (index, good) vs COLLSCAN (collection scan, bad).
Example / Code:
```
db.users.find({email:'a@b.com'}).explain('executionStats'); //Check:
winningPlan.stage === 'IXSCAN' (good) //nReturned vs totalDocsExamined ratio
```

30 Capped collection?

Fixed size, automatically removes oldest documents. For logs, audit trails.
Example / Code:
```
db.createCollection('logs',{capped:true,size:10*1024*1024,max:10000});
//Automatically removes oldest when full
```

■ Advanced (Q31–Q80)

31 What is replication?

Copies of data across multiple servers for high availability. Replica set. Automatic failover.

Example / Code:

```
//Connect with replica set:
mongoose.connect('mongodb://h1,h2,h3/mydb?replicaSet=rs0');
```

32 Replica set?

Group of mongod instances with same data. One primary (reads/writes), secondaries (replicate). Auto failover.

Example / Code:

```
//rs.status() shows replica set health //Primary: takes all writes //Secondaries:
can serve reads (readPreference:secondary)
```

33 Primary vs secondary?

Primary: all writes, propagates via oplog. Secondary: replicates oplog, can serve reads. Election on primary failure.

Example / Code:

```
//Read from secondary for analytics (reduce primary load):
User.find().read('secondary');
```

34 What is failover?

When primary fails, secondaries elect new primary. Takes 10-30 seconds. Writes unavailable during election.

Example / Code:

```
//Minimize failover impact: //1. WriteConcern majority //2. ReadPreference
primaryPreferred //3. Handle write errors with retry
```

35 What is sharding?

Horizontal partitioning across multiple servers (shards). Each shard holds a subset. mongos routes queries.

Example / Code:

```
sh.enableSharding('mydb'); sh.shardCollection('mydb.orders',{userId:'hashed'});
sh.status(); //view distribution
```

36 Horizontal scaling?

Add shards to distribute data and load. Each shard is replica set. mongos router handles transparently.

Example / Code:

```
//Scale from 1 shard to 10 shards: //Add shard: sh.addShard('shard2/host1,host2')
//MongoDB balancer redistributes data
```

37 Shard key?

Field for distributing documents. High cardinality (many unique values), good write distribution, supports common queries.

Example / Code:

```
//Good shard keys: sh.shardCollection('events',{userId:'hashed'}); //hashed =
even distribution //Bad shard key: sh.shardCollection('orders',{status:1}); //low
cardinality -> hot spots
```

38 Chunking in MongoDB?

Data divided into chunks (ranges of shard key). Balancer moves chunks between shards for even distribution.

Example / Code:

```
//Default chunk size: 128MB //Balancer runs in background //sh.startBalancer(), sh.stopBalancer()
```

39 Balancing in MongoDB?

MongoDB balancer automatically moves chunks between shards to maintain even distribution.

Example / Code:

```
sh.isBalancerRunning(); sh.status(); //shows chunk distribution per shard
```

40 Write concern?

Acknowledgment level for writes. w:1 (primary ack), w:majority (majority ack), j:true (journal write).

Example / Code:

```
//High durability: await User.create({name:'Alice'},{writeConcern:{w:'majority',j:true}}); //High performance: await EventLog.create({...},{writeConcern:{w:0}}); //fire and forget
```

41 Read concern?

Consistency level for reads. local (default), majority (majority acknowledged), linearizable (strongest).

Example / Code:

```
//Read only majority-committed data: await User.findOne({_id:id}).readConcern('majority');
```

42 What is journaling?

Write-ahead log for durability. Ensures crash recovery by replaying journal. Enabled by default. j:true in write concern ensures journal acknowledgment.

Example / Code:

```
//Journal ensures data survives crash //Without journal: uncommitted writes lost on crash
```

43 MongoDB transactions?

Multi-document ACID transactions (MongoDB 4.0+).

Example / Code:

```
const session=await mongoose.startSession(); session.startTransaction(); try{ await Account.updateOne({_id:from},{$inc:{balance:-100}},{session}); await Account.updateOne({_id:to},{$inc:{balance:100}},{session}); await session.commitTransaction(); }catch(err){ await session.abortTransaction(); throw err; }finally{session.endSession();}
```

44 ACID in MongoDB?

Atomic (all or nothing), Consistent (data valid), Isolated (concurrent don't interfere), Durable (committed survives crash). Single document always ACID. Multi-document: requires explicit transactions.

Example / Code:

```
//Single document: atomic by default await User.updateOne({_id:id},{$set:{status:'active'},$push:{log:'activated'}}); //atomic //Multi-document: use transactions
```

45 Multi-document transactions?

MongoDB 4.0+ for replica sets, 4.2+ for sharded. Higher overhead — use only when necessary.

Example / Code:

```
//Only when multiple docs MUST be atomic: //Bank transfers, order+inventory updates //Avoid for single-document operations
```

46 Schema design in MongoDB?

Design for access patterns (application-first). Embedding vs referencing based on: how data is accessed, update frequency, document size.

Example / Code:

```
//Step 1: identify access patterns //Step 2: decide embedding vs referencing
//Step 3: consider document growth and size
```

47 Embedding vs referencing?

Embed: accessed together, 1-to-few, fast reads (single query). Reference: large data, changes frequently, many-to-many, accessed independently.

Example / Code:

```
//Embed: user address (always accessed with user)
{_id:userId,name:'Alice',address:{city:'Mumbai'}} //Reference: user orders (many,
queried separately) {_id:orderId,userId:userId,items:[...]}
```

48 One-to-many modeling?

Small many (1-few): embed array. Large many: reference. Very large: array of IDs in parent.

Example / Code:

```
//Few items: embed
{postId:'p1',title:'Post',comments:[{text:'Great!',author:'Bob'}]} //Many items:
reference {commentId:'c1',postId:'p1',text:'Great!',author:'Bob'}
```

49 Many-to-many modeling?

Reference with arrays of IDs, or join collection.

Example / Code:

```
//Student-Course: {studentId:'s1',courses:['c1','c2']}
{courseId:'c1',students:['s1','s2']} //Or join collection:
{studentId:'s1',courseId:'c1',enrolledAt:new Date()}
```

50 Denormalization?

Store redundant data to avoid joins. Faster reads, harder updates.

Example / Code:

```
//Embed author name with post (avoid $lookup on reads):
{_id:'p1',title:'Post',authorName:'Alice',authorId:'u1'} //Tradeoff: update
authorName -> update all posts too
```

51 TTL index?

Automatically delete documents after specified time.

Example / Code:

```
//Delete sessions after 1 hour:
db.sessions.createIndex({createdAt:1},{expireAfterSeconds:3600}); //Delete in 24
hours from custom field:
db.logs.createIndex({expiresAt:1},{expireAfterSeconds:0});
```

52 Partial index?

Index only documents matching filter. Smaller, faster writes, less memory.

Example / Code:

```
//Index only active users (skip inactive):
db.users.createIndex({email:1},{partialFilterExpression:{status:'active'}});
//Smaller index, same query performance for active users
```

53 Sparse index?

Only indexes documents where the field exists. Skips null/missing.

Example / Code:

```
//phoneNumber is optional:
db.users.createIndex({phoneNumber:1},{sparse:true,unique:true}); //Multiple users
can have null phoneNumber (unique sparse allows this)
```

54 What is collation?

Language-specific string comparison rules.

Example / Code:

```
//Case-insensitive index:
db.names.createIndex({name:1},{collation:{locale:'en',strength:2}});
//Case-insensitive query:
db.names.find({name:'alice'}).collation({locale:'en',strength:2});
```

55 What is mapReduce?

Older aggregation mechanism. Superseded by aggregation pipeline. Slower, less expressive.

Example / Code:

```
//Use aggregation pipeline instead: //db.collection.mapReduce() ->
db.collection.aggregate([])
```

56 Change streams?

Real-time notification of data changes. Built on oplog. For event-driven architectures.

Example / Code:

```
const stream=User.watch([{$match:{operationType:'insert'}}]);
stream.on('change',change=>{ console.log('New user:',change.fullDocument);
notifyWebhook(change); });
```

57 What is GridFS?

Store files >16MB (BSON limit). Splits into chunks stored in two collections (fs.files, fs.chunks).

Example / Code:

```
const{GridFSBucket}=require('mongodb'); const bucket=new
GridFSBucket(db,{bucketName:'uploads'});
fs.createReadStream('large-file.pdf').pipe(bucket.openUploadStream('file.pdf'));
```

58 What is bulkWrite?

Execute multiple write operations in one request.

Example / Code:

```
await User.bulkWrite([ {insertOne:{document:{name:'Alice'}}},
{updateOne:{filter:{_id:id},update:{$set:{status:'active'}}}},
{deleteOne:{filter:{email:{$regex:/@spam.com$/}}}} ]);
```

59 What is upsert?

Update if exists, insert if not (update+insert).

Example / Code:

```
await User.updateOne( {email:'alice@example.com'},
{$set:{name:'Alice'},$setOnInsert:{createdAt:new Date()}}, {upsert:true} );
```

60 What is hint?

Force MongoDB to use a specific index for a query.

Example / Code:

```
//Force index usage: db.users.find({email:'a@b.com'}).hint({email:1});
db.users.find({name:/^Alice/}).hint({_id:1}); //force collection scan for testing
```

61 Index intersection?

MongoDB can use multiple indexes for a single query (index AND intersection). Less common than compound index.

Example / Code:

```
//Two single-field indexes: db.users.createIndex({age:1});
db.users.createIndex({status:1}); //Query on both may use intersection
db.users.find({age:{$gt:25},status:'active'});
```

62 Query planner?

Selects optimal query plan. Evaluates candidate plans, picks winner. Use explain() to see decision.

Example / Code:

```
db.users.find({email:'a@b.com'}).explain('allPlansExecution'); //winningPlan,
rejectedPlans, executionStats
```

63 Covered query?

Query satisfied entirely from index — no document fetch. Fastest possible query.

Example / Code:

```
//Create index on {email:1,name:1} //Query returns only email and name:
db.users.find({email:'a@b.com'},{email:1,name:1,_id:0}); //Check: IXSCAN with no
FETCH stage = covered query
```

64 Aggregation optimization?

$match early (use indexes), $project to reduce fields early, $lookup as late as possible, avoid $where.

Example / Code:

```
//Optimal order: [ {$match:{status:'active'}}, //filter first (uses index)
{$project:{name:1,email:1}}, //reduce fields early {$lookup:{...}}, //join late
(fewer docs) {$sort:{name:1}} ]
```

65 Pipeline optimization?

MongoDB auto-optimizes: merges adjacent $match, reorders $sort before $limit, uses indexes where possible.

Example / Code:

```
//MongoDB auto-merges: {$match:{a:1}},{$match:{b:2}} -> {$match:{a:1,b:2}}
{$sort:{x:1}},{$limit:10} -> uses index+limit optimization
```

66 Memory limit in aggregation?

Each aggregation stage limited to 100MB RAM by default. Use allowDiskUse:true for larger.

Example / Code:

```
await Order.aggregate(pipeline,{allowDiskUse:true}); //Or increase limit in
MongoDB config: //aggregate.allowDiskUseByDefault=true
```

67 Disk use in aggregation?

allowDiskUse:true lets stages write temp files to disk when they exceed 100MB RAM limit. Slower but enables large aggregations.

Example / Code:

```
//For large datasets: await
Report.aggregate(pipeline,{allowDiskUse:true,maxTimeMS:30000});
```

68 Data partitioning?

Sharding for horizontal partitioning by shard key. Or application-level: separate collections/databases per tenant.

Example / Code:

```
//Multi-tenant partitioning options: //1. Shard key includes tenantId //2.
Separate collections per tenant: orders_tenant1, orders_tenant2 //3. Separate
databases per tenant
```

69 Write lock in MongoDB?

MongoDB uses document-level locking (MVCC). Multiple concurrent writes to different documents possible. Collection and database locks rare (for DDL operations).

Example / Code:

```
//Document-level locking: safe concurrent updates: await Promise.all([
User.updateOne({_id:id1},{$inc:{count:1}}),
User.updateOne({_id:id2},{$inc:{count:1}}) ]); //concurrent, no conflict
```

70 Read lock?

Readers and writers can work concurrently (MVCC). Reads don't block writes; writes don't block reads.

Example / Code:

```
//MongoDB WiredTiger storage engine: //MVCC (Multi-Version Concurrency Control)
//Readers see consistent snapshot, unblocked by writers
```

71 Isolation level?

MongoDB read concern controls isolation. local (default), majority (committed by majority), linearizable (all acks). Write concern controls durability.

Example / Code:

```
//Strongest isolation: await Order.findOne({_id:id}).readConcern('linearizable');
//Tradeoff: latency for consistency
```

72 What is oplog?

Operation Log — capped collection in local database recording all write operations. Replica sets use oplog for replication. Change streams built on oplog.

Example / Code:

```
//Oplog lives in: local.oplog.rs //Contains: ts (timestamp), op
(insert/update/delete), ns (namespace), o (document)
```

73 What is snapshot?

Read concern 'snapshot' (inside transactions): returns data consistent to transaction start, even if other writes happen during transaction.

Example / Code:

```
const session=await startSession(); await session.withTransaction(async()=>{
//Reads inside use snapshot isolation const balance=await
Account.findOne({_id:id},{session}); });
```

74 Retryable writes?

MongoDB 3.6+ automatically retries certain write operations once on network failure or primary failover. Transparent to application.

Example / Code:

```
//Enabled in connection string: mongoose.connect(uri,{retryWrites:true});
//idempotent writes retried automatically on transient errors
```

75 Read preference?

Where to route read operations: primary, primaryPreferred, secondary, secondaryPreferred, nearest.

Example / Code:

```
//Read from nearest (latency-optimized): await User.find().read('nearest');
//Read from secondary (reduce primary load): await
AnalyticsEvent.find().read('secondaryPreferred');
```

76 Zone sharding?

Route data to specific shards based on shard key ranges. For geographic data locality.

Example / Code:

```
//Data from India -> shard in Mumbai region: sh.addShardToZone('shard1','India');
sh.updateZoneKeyRange('mydb.users',{country:'India'},{country:'India\uffff'},'Ind
ia');
```

77 Index rebuild?

Rebuild indexes to defragment and reclaim space.

Example / Code:

```
db.users.reIndex(); //Or drop and recreate: db.users.dropIndex('email_1');
db.users.createIndex({email:1},{unique:true});
```

78 What is fragmentation?

Empty space in index B-tree after many deletes/updates. Increases index size and slows queries.

Example / Code:

```
db.collection.stats(); //freeStorageSize vs storageSize -> fragmentation ratio
//Fix: reIndex() or compact
```

79 What is compact?

Defragments collection and indexes, reclaims disk space.

Example / Code:

```
db.runCommand({compact:'users',force:true}); //BLOCKS reads/writes during compact
//Schedule during maintenance window
```

80 MongoDB profiling?

Capture slow queries to system.profile collection.

Example / Code:

```
db.setProfilingLevel(1,{slowms:100}); //profile queries >100ms
db.system.profile.find().sort({millis:-1}).limit(10);
```

■ schema (Q81–Q120)

81 Design schema in MongoDB?

Step 1: identify access patterns. Step 2: embedding vs referencing based on access. Step 3: consider size, growth, update frequency.

Example / Code:

```
//Blog app access patterns: //1. Get post with author info -> embed authorName in
post //2. Get all posts by user -> index {userId:1,createdAt:-1} //3. Get post
comments -> reference (comments grow infinitely)
```

82 Embed vs reference decision?

Embed when: data accessed together, one-to-few relationship, data doesn't change independently. Reference when: data large, many-to-many, updated independently, shared across docs.

Example / Code:

```
//Rule of thumb: //< 10 related items AND accessed together -> EMBED //> 10
related items OR accessed independently -> REFERENCE
```

83 Indexing strategy?

Create indexes for all query fields. Use compound indexes for multi-field queries. ESR rule. Analyze with explain(). Remove unused indexes.

Example / Code:

```
//Find slow queries:
db.system.profile.find({millis:{$gt:100}}).sort({millis:-1}); //Check index
usage: db.users.aggregate([{$indexStats:{}}]);
```

84 Query optimization?

1) Create indexes. 2) Project only needed fields. 3) Use $match early. 4) Avoid regex on large fields. 5) Paginate large results.

Example / Code:

```
//Before optimization: db.users.find().sort({createdAt:-1}).skip(1000).limit(20);
//After optimization: db.users.find({_id:{$lt:lastId}}).sort({_id:-1}).limit(20);
//cursor pagination
```

85 Avoid collection scan?

Create indexes on query fields. Check with explain() — avoid COLLSCAN. Compound index for multi-field queries.

Example / Code:

```
//Detect collection scans: db.users.find({name:'Alice'}).explain();
//stage:'COLLSCAN' -> need an index! db.users.createIndex({name:1});
```

86 Use explain()?

explain('executionStats') shows: winning plan, stages, docs examined vs returned, time.

Example / Code:

```
//Key metrics: db.users.find({email:'a@b.com'}).explain('executionStats');
//Good: stage:'IXSCAN', nReturned:1, totalDocsExamined:1 //Bad: stage:'COLLSCAN',
totalDocsExamined:100000
```

87 Avoid large documents?

MongoDB doc limit: 16MB. Large docs: slow reads, replication lag. Keep docs small. Use GridFS for files.

Example / Code:

```
//Breaking large doc into smaller: //Store metadata in doc, actual content in
GridFS {_id:'f1',filename:'report.pdf',size:5242880,gridfsId:ObjectId('...')}
```

88 Pagination best practices?

Skip+limit for small datasets. Cursor-based (range queries) for large. Never use skip > 1000.

Example / Code:

```
//Cursor pagination (fast): await
Post.find({_id:{$lt:lastId},status:'published'}) .sort({_id:-1}).limit(20);
```

89 Skip vs range query pagination?

Skip(N): O(N) — scans N documents before returning. Range (cursor): O(1) — uses index. Use range for large datasets.

Example / Code:

```
//BAD: skip(10000) scans 10000 docs first
Post.find().sort({_id:-1}).skip(10000).limit(20); //GOOD: range query uses index
directly Post.find({_id:{$lt:cursor}}).sort({_id:-1}).limit(20);
```

90 Handle large datasets?

1) Indexes. 2) Cursor-based pagination. 3) Aggregation with $match early. 4) allowDiskUse for large aggregations. 5) Sharding. 6) Archive old data.

Example / Code:

```
//Streaming large aggregation result: const
cursor=Order.aggregate(pipeline).cursor(); for await(const doc of cursor){ await
process(doc); }
```

91 Caching with MongoDB?

Redis for frequently read data. HTTP Cache-Control headers. Pre-aggregate with materialized views (separate collection updated via change streams).

Example / Code:

```
//Materialized view pattern: //Change stream on orders -> update analytics collection //Read analytics collection directly (pre-computed)
```

92 Data archiving?

Move old data to archive collection or cold storage. Keeps working set small for performance.

Example / Code:

```
//Archive orders older than 2 years: const old=await Order.find({createdAt:{$lt:twoYearsAgo}}); await ArchivedOrder.insertMany(old); await Order.deleteMany({createdAt:{$lt:twoYearsAgo}});
```

93 Soft delete?

Add deletedAt timestamp instead of actually deleting. Preserves history.

Example / Code:

```
await User.updateOne({_id:id},{$set:{deletedAt:new Date()}}); //Partial index to exclude deleted from normal queries: db.users.createIndex({email:1},{unique:true, partialFilterExpression:{deletedAt:{$exists:false}}});
```

94 Versioning documents?

Track document history for audit trails.

Example / Code:

```
const schema=new Schema({ data:Object, version:{type:Number,default:1}, history:[{data:Object,changedAt:Date,changedBy:String}] });
schema.pre('save',function(){
this.history.push({data:this.toObject(),changedAt:new Date()}); this.version+=1;
});
```

95 Handling concurrency?

Optimistic locking with version field. Atomic update operators ($set, $inc, $push).

Example / Code:

```
//Optimistic locking: const updated=await Product.findOneAndUpdate( {_id:id,version:currentVersion}, {$inc:{stock:-quantity},$inc:{version:1}}, {returnDocument:'after'} ); if(!updated) throw new Error('Concurrent modification conflict');
```

96 Locking strategy?

Document-level locking (MVCC). $inc, $push, $set are atomic at document level. Use transactions for multi-document atomicity.

Example / Code:

```
//Atomic decrement (race-condition safe): await Product.updateOne( {_id:id,stock:{$gte:quantity}}, {$inc:{stock:-quantity}} ); //atomic check+decrement
```

97 Avoid N+1 in MongoDB?

N+1: load N parents, then N separate queries for children. Fix: $lookup (join), embed data, or batch with $in.

Example / Code:

```
//Bad N+1: const posts=await Post.find({userId:id}); for(const p of posts) p.author=await User.findById(p.userId); //N queries! //Good: const posts=await Post.find({userId:id}).populate('author'); //Or aggregate $lookup
```

98 Use aggregation pipeline?

For complex data transformation, reporting, data enrichment with $lookup, pre-aggregation for dashboards.

Example / Code:

```
//Dashboard stats: await Order.aggregate([ {$match:{createdAt:{$gte:startDate}}}, {$group:{_id:{$dateToString:{format:'%Y-%m-%d',date:'$createdAt'}},total:{$sum:'$amount'},count:{$sum:1}}}, {$sort:{_id:1}} ]);
```

99 Pre-aggregation?

Compute and store aggregated data periodically instead of real-time. For dashboards with expensive queries.

Example / Code:

```
//Cron job or change stream trigger: async function updateDashboardStats(){ const stats=await Order.aggregate(pipeline); await Dashboard.updateOne({_id:'main'},{$set:{stats}},{upsert:true}); }
```

100 Denormalization trade-offs?

Faster reads (no joins), larger documents, complex updates (must update all copies). Suitable for read-heavy, rarely changing data.

Example / Code:

```
//Blog: embed author name (faster reads) //Trade-off: update author name -> update all blog posts //Decision: how often does author name change?
```

101 Hot vs cold data?

Hot data: frequently accessed, in memory/SSD, small fast collection. Cold data: rarely accessed, can be on HDD, archived collection.

Example / Code:

```
//Archive cold data after 1 year: //Hot collection: orders (last 1 year) //Cold collection: orders_archive (older) //Queries auto-route to right collection
```

102 Data partitioning strategy?

Shard by common query field (userId, region, tenantId). Hashed sharding for even distribution. Zone sharding for geo-locality.

Example / Code:

```
//Multi-tenant: shard by tenantId sh.shardCollection('mydb.orders',{tenantId:1,_id:1}); //Geographic: shard by region sh.shardCollection('mydb.users',{region:'hashed'});
```

103 Scale MongoDB?

Vertical (larger server), replica set (HA + read scaling), sharding (horizontal write+storage scaling), caching (Redis), connection pooling.

Example / Code:

```
//Scaling path: //1. Optimize queries and indexes //2. Add read replicas (replica set) //3. Cache hot data in Redis //4. Shard when single server maxed out
```

104 Monitoring MongoDB?

mongostat, mongotop, Atlas monitoring, Prometheus mongodb_exporter, slow query log.

Example / Code:

```
//Connect MongoDB to Prometheus: //mongodb_exporter scrapes metrics //Grafana dashboard for visualization //Alert on: replication lag, connection count, lock %, slow queries
```

105 Metrics to track?

Operations per second, replication lag, connection pool, lock %, memory, cache hit ratio, query execution time.

Example / Code:

```
mongostat --uri='mongodb://localhost:27017' 1 //Shows: inserts/s, queries/s, updates/s, deletes/s, conns, repl lag
```

106 Slow query log?

Enable profiling for slow query detection.

Example / Code:

```
db.setProfilingLevel(1,{slowms:100});
db.system.profile.find().sort({millis:-1}).limit(10);
```

107 Index usage stats?

Monitor index usage to find unused indexes.

Example / Code:

```
db.users.aggregate([{$indexStats:{}}]); //Shows: accesses.ops count for each index //Drop unused indexes: db.users.dropIndex('unusedIndex_1')
```

108 Data consistency?

Write concern majority for strong consistency. Read concern majority for consistent reads. Transactions for multi-doc.

Example / Code:

```
mongoose.connect(uri,{writeConcern:{w:'majority',j:true}}); //For critical ops: use transactions
```

109 Backup strategy?

mongodump (logical), snapshot (filesystem), Atlas continuous backup (point-in-time). Test restores regularly.

Example / Code:

```
mongodump --uri='mongodb+srv://...' --out=/backup/$(date +%Y%m%d) //Restore: mongorestore --uri='mongodb+srv://...' /backup/20240101
```

110 Restore strategy?

mongorestore for logical backups. Point-in-time recovery with Atlas. Test in staging before production.

Example / Code:

```
mongorestore --uri='mongodb://localhost:27017' --nsInclude='mydb.*' /backup/20240101/ //Or Atlas: PITR -> select timestamp -> restore
```

111 Disaster recovery?

Replica set across availability zones. Atlas global clusters for geo-redundancy. Regular backup testing. RTO/RPO targets.

Example / Code:

```
//Multi-AZ replica set: //Primary: us-east-1a //Secondary: us-east-1b //Arbiter: us-east-1c
```

112 Security best practices?

Auth enabled, TLS/SSL, role-based access, network isolation, audit logging, encrypt at rest.

Example / Code:

```
mongod --auth --tls --tlsCertificateKeyFile cert.pem //Least privilege roles: db.createUser({user:'api',pwd:'...',roles:[{role:'readWrite',db:'mydb'}]});
```

113 Encryption in MongoDB?

Encryption at rest (WiredTiger), TLS in transit, client-side field encryption (CSFLE), Atlas encryption.

Example / Code:

```
//Client-side field encryption: //Sensitive fields encrypted before sending to server //MongoDB server never sees plaintext //Atlas: automatic encryption key management
```

114 Role-based access?

Built-in roles (readWrite, dbAdmin) and custom roles for fine-grained access control.

Example / Code:

```
db.createUser({ user:'readonly', pwd:'pass', roles:[{role:'read',db:'mydb'}] });
//Custom role: db.createRole({role:'orderReader',privileges:[{resource:{db:'mydb'
,collection:'orders'},actions:['find']}],roles:[]});
```

115 Auditing in MongoDB?

MongoDB Enterprise: audit log of all operations. Atlas: activity feed. Required for compliance (SOC2, HIPAA).

Example / Code:

```
//Enable audit (Enterprise): //mongod --auditDestination=file --auditFormat=JSON
--auditPath=audit.json //Atlas: Security->Audit Log
```

116 Multi-region setup?

Atlas global clusters: data locality, low-latency reads globally. Zone sharding for geographic distribution.

Example / Code:

```
//Atlas global cluster: //Zones: us-east, eu-west, ap-south //Data pinned to zone
by shard key prefix //Reads served from nearest zone
```

117 Cloud vs self-hosted?

MongoDB Atlas (cloud): managed, auto-scaling, backups, monitoring, no ops. Self-hosted: more control, lower cost at scale, requires MongoDB expertise.

Example / Code:

```
//Atlas: best for startups and medium businesses //Self-hosted: large enterprises
with dedicated DB team
```

118 Cost optimization?

Right-size instances, use proper indexes (avoids full scans), archive old data, use Atlas Serverless for low-traffic, autoscale.

Example / Code:

```
//Cost killers: //1. Full collection scans -> add indexes //2. Over-provisioned
instances -> rightsize //3. Hot collections without cache -> add Redis
```

119 Schema evolution?

Add optional fields (no migration needed). Rename fields: gradual migration + dual-write. Remove fields: optional then clean up.

Example / Code:

```
//Dual-write migration: //Step 1: write to both old and new field //Step 2:
backfill old docs with new field //Step 3: read from new field //Step 4: remove
old field writes
```

120 Real-world design patterns?

Bucket pattern (time-series), Outlier pattern (handle extremes), Extended Reference pattern, Pre-computed Rollup pattern, Schema versioning.

Example / Code:

```
//Bucket pattern for time-series: //Instead of 1 doc per reading, bucket by hour:
{deviceId:'d1',hour:'2024-01-01T10:00',readings:[{ts:...,val:22.1},{ts:...,val:22
.3}]} //Result: fewer docs, better index performance
```

Part 6 authentication

60 Questions — Authentication & authorization

■ Basics (Q1–Q20)

1 What is authentication?

Process of verifying identity — who you are. Checking username/password, biometrics, tokens. AuthN.

Example / Code:

```
//Login: verify credentials, issue token const user=await User.findOne({email}); const valid=await bcrypt.compare(password,user.password); if(valid) const token=jwt.sign({id:user._id},SECRET);
```

2 What is authorization?

Process of verifying permissions — what you can do. After authentication. AuthZ.

Example / Code:

```
//Check if user has permission: if(req.user.role!=='admin') return res.status(403).json({message:'Forbidden'});
```

3 Authentication vs authorization?

AuthN: WHO are you? (login). AuthZ: WHAT can you do? (permissions). AuthN always before AuthZ.

Example / Code:

```
//First authenticate: const user=verifyToken(token); //authN //Then authorize: if(!user.canEdit(resource)) throw ForbiddenError(); //authZ
```

4 Session-based auth?

Server stores session data, sends session ID cookie to client. Stateful. Client sends session ID on each request; server looks up session.

Example / Code:

```
//Express-session: req.session.userId=user._id; //Next request: const userId=req.session.userId; const user=await User.findById(userId);
```

5 Token-based auth?

Server issues token (JWT) containing user info. Client sends token on each request. Stateless. Server verifies token signature.

Example / Code:

```
//Issue JWT: const token=jwt.sign({id:user._id,role:user.role},SECRET,{expiresIn:'1h'}); //Verify JWT: const payload=jwt.verify(req.headers.authorization.split(' ')[1],SECRET);
```

6 What is JWT?

JSON Web Token — compact, self-contained token for secure data transmission. Signed (and optionally encrypted). Stateless authentication.

Example / Code:

```
//JWT = base64(header).base64(payload).signature //header: {alg:'HS256',typ:'JWT'} //payload: {sub:'userId',role:'admin',exp:...} //signature: HMACSHA256(header.payload, secret)
```

7 Structure of JWT?

Three parts separated by dots: Header (algorithm+type), Payload (claims), Signature (verification).

Example / Code:

```
//Decode (NOT verify) a JWT: const[header,payload,sig]=token.split('.');
JSON.parse(Buffer.from(payload,'base64url').toString()); //Contains: {sub, iat,
exp, role, ...}
```

8 Access token vs refresh token?

Access token: short-lived (15min-1hr), used for API calls. Refresh token: long-lived (7-30 days), used only to get new access tokens. Limits window of token theft.

Example / Code:

```
//Access token (short-lived): const
access=jwt.sign({id,role},ACCESS_SECRET,{expiresIn:'15m'}); //Refresh token
(long-lived): const refresh=jwt.sign({id},REFRESH_SECRET,{expiresIn:'7d'});
```

9 Where to store JWT?

HttpOnly cookie (safest — XSS can't steal). Memory (state variable — lost on refresh). Avoid localStorage (XSS vulnerable).

Example / Code:

```
//Secure cookie: res.cookie('token',accessToken,{ httpOnly:true, secure:true,
sameSite:'strict', maxAge:15*60*1000 });
```

10 Cookies vs localStorage?

Cookies: auto-sent with requests, 4KB, HttpOnly/Secure/SameSite support, server can set. localStorage: JS only access, 5-10MB, no auto-send, XSS risk.

Example / Code:

```
//JWT in HttpOnly cookie: //Pros: XSS safe, auto-sent //Cons: CSRF risk (mitigate
with SameSite) //JWT in localStorage: //Pros: easy to use //Cons: XSS can steal
token -> DON'T USE FOR AUTH
```

11 HttpOnly cookie?

Cookie that cannot be accessed by JavaScript (document.cookie). Protects from XSS token theft.

Example / Code:

```
res.cookie('session',token,{ httpOnly:true, //JS can't read this secure:true,
//HTTPS only sameSite:'strict' });
```

12 Secure cookie?

Cookie only sent over HTTPS. Prevents interception over HTTP.

Example / Code:

```
res.cookie('token',value,{ secure:true, //only sent via HTTPS httpOnly:true });
```

13 SameSite cookie?

Controls when cookies are sent cross-site. Strict: only same-site. Lax: same-site + top-level navigation. None: always (requires Secure).

Example / Code:

```
//SameSite=Strict: best CSRF protection //No cross-origin requests include cookie
//Good for APIs consumed by your own domain
```

14 What is OAuth?

Open standard for delegated authorization. Allows third-party access without sharing passwords. 'Login with Google/GitHub/Facebook'.

Example / Code:

```
//OAuth flow: //1. User clicks 'Login with Google' //2. Redirect to Google auth
page //3. User grants permission //4. Google redirects back with code //5.
```

```
Exchange code for access token
```

15 What is OpenID Connect?

Identity layer on top of OAuth 2.0. Provides authentication (who the user is) in addition to OAuth authorization. Returns ID token (JWT with user info).

Example / Code:

```
//OIDC adds identity: //OAuth: authorization (access to resources) //OIDC:
authentication (identity of user) + ID token
```

16 What is SSO?

Single Sign-On — authenticate once, access multiple apps. Examples: Okta, Auth0, Google Workspace. SAML or OIDC protocols.

Example / Code:

```
//SSO flow: //1. User signs in to Identity Provider (IdP) once //2. All connected
apps (Service Providers) trust the IdP //3. User accesses any app without
re-authenticating
```

17 What is MFA?

Multi-Factor Authentication — two or more factors: something you know (password), something you have (OTP/phone), something you are (biometric).

Example / Code:

```
//TOTP (Google Authenticator): const speakeasy=require('speakeasy'); const
token=speakeasy.totp({secret:user.mfaSecret,encoding:'base32'}); const
valid=speakeasy.totp.verify({secret,encoding:'base32',token:userToken,window:1});
```

18 What is hashing?

One-way cryptographic transformation. Same input->same hash. Can't reverse. Used for passwords. Use bcrypt/argon2 (slow by design — prevents brute force).

Example / Code:

```
const bcrypt=require('bcryptjs'); const hash=await bcrypt.hash('myPassword',12);
//cost factor 12 const valid=await bcrypt.compare('myPassword',hash); //true
```

19 What is salting?

Adding random unique value (salt) to password before hashing. Prevents rainbow table attacks. bcrypt includes salt automatically.

Example / Code:

```
//bcrypt auto-salts: const hash=await bcrypt.hash('password',10); //salt included
in hash //Each hash different: await bcrypt.hash('password',10); //$2b$10$...
await bcrypt.hash('password',10); //$2b$10$... (different!)
```

20 What is bcrypt?

Password hashing algorithm. Slow by design (adjustable cost factor), includes salt, resistant to GPU brute-force. Gold standard for password storage.

Example / Code:

```
const hash=await bcrypt.hash('password',10); //10 rounds const valid=await
bcrypt.compare('password',hash); //Cost factor 10: ~100ms on modern hardware
//Cost factor 12: ~400ms (safer but slower UX)
```

Security Attacks (Q21–Q40)

21 What is XSS?

Cross-Site Scripting — attacker injects malicious JS into web pages.

Example / Code:

```
Example: <script>alert('hacked')</script> Prevention: Avoid unsafe HTML
rendering.
```

22 Types of XSS?

Stored, Reflected, DOM-based.

Example / Code:

```
Stored: saved in DB Reflected: via URL DOM: client-side JS
```

23 How to prevent XSS?

Escape output, use CSP, sanitize input.

Example / Code:

```
import xss clean = xss.clean(user_input) # Helmet-like headers handled via
frameworks
```

24 What is CSRF?

Cross-Site Request Forgery — attacker tricks user into performing actions.

Example / Code:

```
Example: malicious form auto-submits request
```

25 How CSRF works?

User logged in → attacker sends request → browser auto-attaches cookies.

Example / Code:

```
Example: bank transfer triggered via hidden request
```

26 CSRF prevention techniques?

CSRF token, SameSite cookies.

Example / Code:

```
# Example (Flask/Django) csrf_token = generate_token() headers = {"X-CSRF-Token":
csrf_token}
```

27 What is SQL Injection?

Malicious SQL query execution.

Example / Code:

```
# BAD query = f"SELECT * FROM users WHERE email = '{email}'" # GOOD
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

28 How to prevent SQL Injection?

Use parameterized queries, ORM.

Example / Code:

```
Use libraries like SQLAlchemy, Django ORM
```

29 What is NoSQL injection?

Injection in MongoDB queries.

Example / Code:

```
# BAD db.users.find({"email": request.json}) # GOOD db.users.find({"email":
str(request.json)})
```

30 What is CORS attack?

Unauthorized cross-origin requests.

Example / Code:

```
# Example (Flask) from flask_cors import CORS CORS(app,
origins=["https://yourdomain.com"])
```

31 What is clickjacking?

Tricking user to click hidden elements.

Example / Code:

```
Use X-Frame-Options header
```

32 What is MITM attack?

Man-In-The-Middle attack intercepts communication.

Example / Code:

```
Prevent using HTTPS
```

33 What is brute force attack?

Trying multiple passwords repeatedly.

Example / Code:

```
Prevent using rate limiting
```

34 What is rate limiting?

Limiting number of requests.

Example / Code:

```
# Example (Flask-Limiter) from flask_limiter import Limiter limiter.limit("100 per hour")
```

■ Time: O(1)

35 What is DDoS attack?

Flooding server with traffic.

Example / Code:

```
Use CDN, load balancer, firewall
```

36 What is HTTPS?

Secure HTTP using encryption (TLS).

37 SSL vs TLS?

TLS is modern version of SSL.

38 What is HSTS?

Forces HTTPS usage.

Example / Code:

```
Strict-Transport-Security header
```

39 What is CSP?

Content Security Policy prevents XSS.

Example / Code:

```
Restricts allowed sources
```

40 What are security headers?

HTTP headers for protection.

Example / Code:

```
Example: CSP, HSTS, X-Frame-Options
```

■ Advanced (Q41–Q60)

41 How to design auth system?

1) Choose auth method (JWT vs session). 2) Password hashing (bcrypt). 3) Token storage (HttpOnly cookie). 4) Refresh token rotation. 5) RBAC for authorization.

Example / Code:

```
// Complete auth flow: // Register: hash password -> store user // Login: verify
password -> issue JWT+refresh tokens // Authenticate: verify JWT on each request
// Refresh: exchange refresh token for new access token // Logout:
blacklist/delete tokens
```

42 How to secure APIs?

1) Authentication (JWT). 2) Authorization (RBAC). 3) Input validation. 4) Rate limiting. 5) HTTPS. 6) CORS whitelist. 7) Security headers (helmet).

43 Zero trust architecture?

Never trust, always verify. No implicit trust based on network location. Verify every request regardless of source.

Example / Code:

```
// Zero trust principles: // 1. Verify explicitly (authenticate + authorize
always) // 2. Use least privilege access // 3. Assume breach (encrypt, log,
monitor)
```

44 OAuth flow types?

Authorization Code: server-side. Implicit: deprecated. Client Credentials: machine-to-machine. PKCE: SPAs and mobile.

Example / Code:

```
// PKCE flow for SPAs: // 1. Generate code_verifier and code_challenge // 2. Auth
request with code_challenge // 3. Get authorization code // 4. Exchange code +
code_verifier for tokens // 5. PKCE prevents authorization code interception
```

45 How to store passwords securely?

Use bcrypt/argon2 with appropriate cost factor. Never store plaintext, MD5, SHA1, or SHA256.

Example / Code:

```
// bcrypt with cost 12 (~250ms): const hash=await bcrypt.hash(password,12); //
argon2 (more secure, newer): const hash=await argon2.hash(password,{
type:argon2.argon2id, memoryCost:2**16 // 64MB });
```

46 Hashing vs encryption?

Hashing: one-way, cannot reverse (for passwords). Encryption: two-way, can decrypt with key (for data you need to retrieve).

Example / Code:

```
// Hashing — passwords: const hash=await bcrypt.hash(password,12); // one-way //
Encryption — sensitive data you need back: const
encrypted=crypto.createCipheriv("aes-256-gcm",key,iv).update(data);
```

47 How to secure microservices?

Service-to-service auth (JWT, mTLS), API gateway for external auth, secrets management, network policies.

Example / Code:

```
// Service-to-service JWT: const serviceToken=jwt.sign(
{service:"orders",iss:"orders-service"}, SERVICE_SECRET, {expiresIn:"5m"} ); //
Each service validates incoming service tokens
```

48 What is key management?

Process of generating, distributing, storing, rotating, and revoking cryptographic keys. Use KMS (AWS, GCP, Azure) for production.

49 Permission-based auth?

User has explicit permissions (can_read_posts, can_edit_users). More granular than RBAC.

Example / Code:

```
// Permission check: if(!user.permissions.includes("posts:edit")) return
res.status(403).json({error:"Missing permission: posts:edit"});
```

50 Stateless vs stateful auth?

Stateless (JWT): any server validates, easy to scale, cannot revoke easily. Stateful (session): can revoke instantly, requires shared storage.

Example / Code:

```
// JWT revocation workaround: // Store revoked tokens in Redis until expiry await
redis.set(`bl:${token}`,1,"EX",3600); // Check on each request: if(await
redis.get(`bl:${token}`)) throw new Error("Revoked");
```

51 What is password policy?

Minimum length (8+), complexity (upper, lower, number), no common passwords, no reuse, regular rotation for admins.

52 What is 2FA vs MFA?

2FA: exactly two factors. MFA: two or more factors. TOTP (Google Authenticator), SMS (less secure — SIM swapping), hardware keys (YubiKey — most secure).

53 What is JWT blacklisting?

Store revoked tokens in Redis until they expire. Enables logout and forced session termination.

Example / Code:

```
// On logout: await redis.set(`bl:${token}`,1,"EX",ttl); // TTL = token remaining
lifetime // On auth: if(await redis.get(`bl:${token}`)) return
res.status(401).json({error:"Logged out"});
```

54 What is PKCE?

Proof Key for Code Exchange. Prevents authorization code interception attacks in SPAs and mobile apps.

Example / Code:

```
// Prevents: // 1. Generate random code_verifier // 2. Hash it -> code_challenge
// 3. Send code_challenge in auth request // 4. Exchange code + code_verifier for
tokens // Intercepted code is useless without code_verifier
```

55 Content Security Policy nonces?

Unique token per page load. Only scripts with matching nonce attribute execute. Defeats most XSS.

Example / Code:

```
// Server generates nonce: const nonce=crypto.randomBytes(16).toString("base64");
res.setHeader("Content-Security-Policy",`script-src 'nonce-${nonce}'`); // HTML:
<script nonce="${nonce}">...</script>
```

56 What is subresource integrity (SRI)?

Verify CDN resources have not been tampered with using hash attribute.

Example / Code:

```
<script src="https://cdn.example.com/lib.js" integrity="sha384-abc..."
crossorigin="anonymous"></script>
```

57 What is API key auth?

Simple authentication using a pre-shared key. For server-to-server or developer APIs. Should be long, random, and stored hashed.

Example / Code:

```
// Generate: const apiKey=crypto.randomBytes(32).toString("hex"); // Store
hashed: await
ApiKey.create({key:createHash("sha256").update(apiKey).digest("hex"),userId}); //
Validate: const
hash=createHash("sha256").update(req.headers["x-api-key"]).digest("hex"); const
key=await ApiKey.findOne({key:hash});
```

58 Security audit checklist?

HTTPS, CSP, CORS whitelist, no secrets in code, input validation, rate limiting, secure cookies, logging, dependency audits, pen testing.

59 What is npm audit?

Checks npm packages for known vulnerabilities.

Example / Code:

```
npm audit # check vulnerabilities npm audit fix # auto-fix where possible npm
audit --json # JSON output for CI
```

60 Security best practices summary?

1) HTTPS everywhere. 2) helmet. 3) Input validation. 4) Rate limiting. 5) HttpOnly cookies. 6) bcrypt passwords. 7) CORS whitelist. 8) No secrets in code. 9) Log suspicious activity. 10) Regular security audits.

Part 7
SYSTEM DESIGN
(90 Questions)— Authentication & authorization

■ Basics (Q1–Q25)

1 What is system design?

Process of defining architecture, components, modules, interfaces, and data flow to satisfy requirements. Covers scalability, reliability, performance, maintainability.

2 What is scalability?

Ability to handle increasing load gracefully. Vertical: more resources to same server. Horizontal: more servers. Node.js ideal for horizontal scaling (stateless).

3 Horizontal vs vertical scaling?

Vertical: bigger server (limited, single failure point). Horizontal: more servers + load balancer (unlimited, redundant). Design stateless services for horizontal scaling.

4 What is load balancing?

Distributes requests across multiple server instances. Algorithms: round-robin, least connections, IP hash. Tools: Nginx, AWS ALB, HAProxy.

Example / Code:

```
upstream api{ server 10.0.0.1:3000; server 10.0.0.2:3000; server 10.0.0.3:3000; }
```

5 What is caching?

Store frequently accessed data in fast memory to reduce database load and latency. Layers: browser, CDN, application (Redis), database.

6 What is CDN?

Content Delivery Network. Distributes static assets (images, JS, CSS) to servers close to users. Reduces latency, bandwidth costs.

Example / Code:

```
// Cache static assets: app.use(express.static("public",{maxAge:"1y"})); // 1
year cache // CDN: CloudFlare, AWS CloudFront, Fastly
```

7 What is database scaling?

Read replicas (horizontal read scaling), sharding (horizontal write scaling), connection pooling, caching layer.

8 What is sharding?

Horizontal partitioning — distribute data across multiple database servers. Each shard holds subset. Trade-off: complex queries, cross-shard transactions.

9 What is replication?

Copies of data across multiple servers for high availability and read scaling. Primary for writes, replicas for reads.

10 What is CAP theorem?

Distributed system can guarantee only 2 of: Consistency, Availability, Partition Tolerance. Must tolerate partitions -> choose C or A.

Example / Code:

```
// CP (MongoDB): consistent, may be unavailable // AP (Cassandra): available, eventually consistent // Real choice: when network partition occurs, choose C or A
```

11 What is consistency?

All nodes see same data at same time. Strong: all reads get latest write. Eventual: reads may get stale but eventually consistent.

12 What is availability?

System remains operational and returns responses even with some node failures. High availability: 99.9% (8.7h downtime/year).

13 What is partition tolerance?

System continues functioning when network partitions occur. Must have in distributed systems — choose C or A given P.

14 Latency vs throughput?

Latency: time for single request (ms). Throughput: requests per second (RPS). Often trade-off: optimizing one can hurt other.

15 What is fault tolerance?

System continues operating despite failures. Achieved via: redundancy, failover, circuit breakers, graceful degradation.

16 What is high availability?

99.9% (8.7h/yr), 99.99% (52min/yr), 99.999% (5min/yr). Achieved via: redundancy, failover, health checks, monitoring.

17 Eventual consistency?

Nodes may temporarily have different data but will eventually converge. Acceptable for: caches, recommendations, analytics.

18 Strong consistency?

All reads reflect the most recent write. Required for: financial transactions, inventory, seat booking. More expensive.

19 What is microservices?

Architecture where application is split into small, independent, loosely coupled services. Each service: owns its data, can be deployed independently, communicates via APIs.

Example / Code:

```
// Services: // - User Service // - Order Service // - Payment Service // - Notification Service // Each has own database, deployed independently
```

20 Monolith vs microservices?

Monolith: simple, easy to develop, single deployment. Microservices: scalable, independent deployments, technology flexibility, but complex operations.

21 What is API gateway?

Single entry point for clients. Handles: routing, auth, rate limiting, SSL, logging, request transformation.

Example / Code:

```
// Express as simple API gateway:
app.use("/users",authenticate,proxy({target:"http://users:3001"}));
app.use("/orders",authenticate,proxy({target:"http://orders:3002"}));
```

22 What is reverse proxy?

Server in front of application servers. Handles: SSL termination, load balancing, caching, rate limiting, static files.

23 What is message queue?

Decouples services, enables async processing. Producer sends; consumer processes when ready. BullMQ (Redis), RabbitMQ, Kafka.

Example / Code:

```
// Async email — don't block API response: await
emailQueue.add("welcome",{to:user.email}); res.json({created:true}); // respond
immediately, email sent in background
```

24 What is pub-sub?

Publish-Subscribe pattern. Publishers emit events; subscribers listen. Decoupled communication. Redis pub/sub, Kafka topics.

Example / Code:

```
// Redis pub/sub for real-time: await redis.publish("chat","message data");
subscriber.subscribe("chat");
subscriber.on("message",(ch,msg)=>io.emit("message",msg));
```

25 What is idempotency?

Operation produces same result regardless of how many times executed. Critical for payments — prevent double charges.

Example / Code:

```
// Idempotency key: const key=req.headers["idempotency-key"]; const
existing=await redis.get(`idem:${key}`); if(existing) return
res.json(JSON.parse(existing)); const result=await processPayment(req.body);
await redis.setex(`idem:${key}`,86400,JSON.stringify(result)); res.json(result);
```

■ Always use idempotency keys for payment APIs.

■ Intermediate Designs (Q26–Q60)

26 Design a URL shortener?

Base62 encode an atomic counter for the short code. Redis cache for popular URLs. Async analytics. 301 redirect (browser caches).

Example / Code:

```
const short=base62.encode(atomicCounter.increment()); await
redis.setex(`url:${short}`,3600,originalUrl); // Redirect: const url=await
redis.get(`url:${code}`)||await DB.findOne({code}); res.redirect(301,url);
```

27 Design a chat system?

WebSocket (Socket.io) + Redis pub/sub for multi-server. MongoDB for message persistence. Redis for online presence.

Example / Code:

```
// Multi-server chat: // User on Server1 sends to User on Server2 // Server1
publishes to Redis channel // Server2 subscribes and delivers to User
```

28 Design Instagram/photo sharing?

Upload: pre-signed S3 URL (direct to S3, no server bandwidth). Feed: Redis sorted set (user_id, score=timestamp). CDN for serving images.

Example / Code:

```
// Photo upload: // 1. Client requests pre-signed S3 URL from server // 2. Client uploads directly to S3 // 3. S3 triggers Lambda -> thumbnail generation // 4. Store metadata in DB
```

29 Design a Twitter clone?

Fanout on write (push to followers) for small follower counts. Pull-based for celebrity accounts (100M+ followers).

Example / Code:

```
// Fanout on write: tweet.followers.forEach(async followerId=>{ await redis.lpush(`feed:${followerId}`,tweetId); await redis.ltrim(`feed:${followerId}`,0,999); });
```

30 Design YouTube?

Upload -> S3 -> transcoding (FFmpeg workers) -> CDN. Adaptive bitrate streaming. Redis for view count (batched DB writes).

Example / Code:

```
// Upload flow: // Client -> Server -> S3 (original) // S3 event -> Transcoding queue // Worker: FFmpeg -> multiple bitrates -> S3/CDN // DB: video metadata, CDN URLs
```

31 Design Netflix streaming?

CDN for video delivery, adaptive bitrate (HLS/DASH), content fingerprinting, recommendation ML, multiple transcoding formats.

32 Design Uber/ride-sharing?

Geospatial: Redis GeoSet for driver locations. WebSocket for real-time tracking. Surge pricing algorithm. Matching algorithm.

33 Design a payment system?

Idempotency keys, distributed transactions (Saga pattern), audit log, reconciliation, PCI DSS compliance, circuit breakers.

Example / Code:

```
// Saga pattern for distributed transactions: // 1. Create order // 2. Reserve inventory // 3. Process payment // 4. Send confirmation // On any failure: compensating transactions rollback
```

34 Design notification system?

Multi-channel (email, SMS, push, in-app). Message queue for async delivery. Template engine. User preferences. Rate limiting per user.

Example / Code:

```
// Notification service: const job=await notificationQueue.add("send",{ userId,type:"email", template:"welcome",data:{name:user.name} });
```

35 Design a search system?

Elasticsearch for full-text search. Indexing pipeline. Search relevance tuning. Autocomplete with Redis sorted sets.

Example / Code:

```
// Elasticsearch via Node.js: const{Client}=require("@elastic/elasticsearch"); const client=new Client({node:"http://localhost:9200"}); const results=await client.search({ index:"products", body:{query:{multi_match:{query:"laptop",fields:["name","description"]}}} });
```

36 Design a rate limiter?

Token bucket (allows bursts), sliding window (accurate), fixed window (simple). Redis for distributed counting.

Example / Code:

```
// Sliding window with Redis: async function rateLimit(key,limit,windowMs){ const now=Date.now(); const window=now-windowMs; await redis.zremrangebyscore(key,0,window); const count=await redis.zcard(key); if(count>=limit) return false; await redis.zadd(key,now,now); await redis.expire(key,windowMs/1000); return true; }
```

37 Design logging system?

Structured JSON logs -> Kafka -> Elasticsearch -> Kibana (ELK stack). Correlation IDs for tracing. Log levels, retention policies.

38 Design a file storage system?

S3 for storage, CDN for serving, metadata in DB, chunked upload for large files, virus scanning, access control.

Example / Code:

```
// Chunked upload: // 1. Init upload: POST /upload/init -> uploadId // 2. Upload chunks: PUT /upload/:id/chunk/:num // 3. Complete: POST /upload/:id/complete // S3 multipart upload handles this natively
```

39 What is LRU cache?

Least Recently Used cache eviction. Remove least recently accessed item when cache is full. Implemented with HashMap + doubly linked list.

Example / Code:

```
// Redis LRU: // maxmemory-policy allkeys-lru // Redis automatically evicts LRU keys when memory full
```

40 What is consistent hashing?

Distribute data across nodes so that adding/removing a node minimizes key remapping. Used in distributed caches, sharding.

Example / Code:

```
// Consistent hashing ring: // Add node: only neighbor keys remapped // Remove node: only that node's keys remapped // Without: adding node -> rehash ALL keys (expensive)
```

41 What is bloom filter?

Space-efficient probabilistic data structure. Check if element MIGHT be in set (no false negatives, some false positives). Used for: URL deduplication, spam detection.

Example / Code:

```
// Redis bloom filter (RedisBloom): await redis.call("BF.ADD","urls","https://example.com"); const exists=await redis.call("BF.EXISTS","urls","https://example.com");
```

42 What is circuit breaker?

Prevents cascading failures. States: Closed (normal), Open (failing fast), Half-Open (testing recovery).

Example / Code:

```
const breaker=new CircuitBreaker(callExternalService,{ timeout:3000, errorThresholdPercentage:50, resetTimeout:30000 }); // Open: fast fail without calling service // Reduces cascade failures
```

43 What is retry strategy?

Retry failed operations with backoff. Exponential backoff: 1s, 2s, 4s, 8s. Limit retries to prevent thundering herd.

44 What is exponential backoff?

Wait time doubles after each retry. With jitter (random offset) to prevent thundering herd.

Example / Code:

```
// Exponential backoff with jitter: const
delay=Math.min(1000*2**attempt,30000)+Math.random()*1000; await new
Promise(r=>setTimeout(r,delay));
```

45 What is Saga pattern?

Manage distributed transactions without ACID. Sequence of local transactions. On failure: compensating transactions.

Example / Code:

```
// Order Saga: // 1. Create order // 2. Reserve inventory (or compensate: cancel
order) // 3. Process payment (or compensate: unreserve inventory) // 4. Send
confirmation
```

46 What is CQRS?

Command Query Responsibility Segregation. Separate read and write models. Writes to one DB, reads from optimized read model (denormalized).

Example / Code:

```
// Write model: User (normalized, transactions) // Read model: UserProfile
(denormalized, optimized for reads) // Event synchronizes write -> read model
```

47 What is event sourcing?

Store events (what happened) instead of current state. Replay events to reconstruct state. Enables time-travel, audit log.

Example / Code:

```
// Instead of: user.balance = 100 // Store:
[{event:"deposit",amount:100},{event:"withdraw",amount:20}] // Current state =
replay all events
```

48 What is distributed lock?

Coordinate access across multiple processes/servers. Redis SETNX with TTL. Use Redlock algorithm for reliability.

Example / Code:

```
// Redis distributed lock: const lock=await
redis.set("lock:resource","token","NX","EX",30); if(!lock) return "Already
locked"; try{ await doWork(); }finally{ await redis.del("lock:resource"); }
```

49 What is leader election?

Choose one node as leader for coordination. Zookeeper, etcd, Redis. Leader handles writes; followers replicate.

50 What is quorum?

Minimum number of nodes that must agree for operation to proceed. N/2+1 for fault tolerance. MongoDB replica set writes.

51 Data partitioning strategies?

Range (by value range), Hash (evenly distributed), Directory (lookup table), Geographic (by location).

52 What is hot partition?

One partition receiving disproportionate traffic. Causes: bad shard key, celebrity user, popular content. Mitigate with better key design or micro-sharding.

53 What is schema design for scale?

Denormalize for reads, normalize for writes. NoSQL for flexibility/scale, SQL for complex queries/transactions.

54 What is denormalization for scale?

Store redundant data to avoid expensive joins. Trade-off: storage and update complexity for read performance.

55 Database indexing strategy?

Index all query fields. Compound indexes in ESR order. Monitor with explain(). Remove unused indexes. Balance read speed vs write overhead.

56 What is pre-computation?

Compute expensive results ahead of time and cache. For: leaderboards, aggregates, recommendation feeds.

Example / Code:

```
// Leaderboard — pre-computed with Redis: await
redis.zadd("leaderboard",score,userId); await
redis.zrevrange("leaderboard",0,9,"WITHSCORES"); // top 10
```

57 What is write-through cache?

Write to cache AND database simultaneously. Cache always has fresh data. Consistent but slower writes.

58 What is write-back cache?

Write to cache only. Periodically sync to database. Faster writes but risk data loss on cache failure.

59 What is cache-aside (lazy loading)?

Check cache first. If miss, load from DB and populate cache. Most common pattern.

60 What is read-through cache?

Cache automatically loads data from DB on cache miss. Transparent to application.

■ Advanced (Q61–Q90)

61 Scale to millions of users?

1) Cache aggressively (Redis). 2) Horizontal scaling. 3) DB read replicas. 4) Async processing (queues). 5) CDN for static. 6) Microservices for independent scaling.

62 Handle traffic spikes?

Auto-scaling (K8s HPA), queue-based load leveling, circuit breakers, graceful degradation.

Example / Code:

```
// K8s auto-scaling: // Target CPU 70% // Scale from 2 to 20 pods automatically
```

63 Design globally distributed system?

Multi-region deployment, data replication, latency-based routing, local caches, eventual consistency.

64 How to reduce latency?

CDN for static, edge computing, database indexes, caching, connection pooling, async processing, reduce round trips.

65 How to ensure consistency?

Strong: ACID transactions, synchronous replication. Eventual: async replication + conflict resolution.

66 How to handle failures?

Redundancy, circuit breakers, retries with backoff, graceful degradation, health checks, auto-recovery.

67 Design resilient system?

Design for failure. Redundancy everywhere. Circuit breakers. Chaos engineering (Netflix chaos monkey). Health checks. Multiple availability zones.

68 How to monitor system?

Metrics (Prometheus+Grafana), logs (ELK), traces (Jaeger), alerts (PagerDuty). SLO/SLA/SLI tracking.

69 What metrics to track?

Latency (p99), throughput (RPS), error rate, saturation (CPU/memory), availability. RED method: Rate, Errors, Duration.

70 What is observability?

Ability to understand system state from its outputs. Pillars: Logs (what), Metrics (how much), Traces (where).

Example / Code:

```
// OpenTelemetry in Node.js: const{trace}=require("@opentelemetry/api"); const
tracer=trace.getTracer("my-service"); const
span=tracer.startSpan("processOrder"); try{await
process(order);}finally{span.end();}
```

71 Logging vs monitoring vs tracing?

Logging: event records (what happened). Monitoring: metrics over time (how system performing). Tracing: request flow across services.

72 What is distributed tracing?

Track request as it flows across microservices. Correlation IDs, spans, parent-child relationships. Jaeger, Zipkin, OpenTelemetry.

73 What is autoscaling?

Automatically add/remove instances based on load. K8s HPA (CPU/memory), custom metrics, scheduled scaling.

74 What is blue-green deployment?

Two identical environments (blue=current, green=new). Switch traffic from blue to green. Zero downtime. Easy rollback.

75 What is canary deployment?

Gradually route percentage of traffic to new version. Start 1%, monitor, increase to 100%. Catch issues early.

76 What is feature flag?

Feature flag allows enabling/disabling features without deployment. Useful for A/B testing and gradual rollout.

Example / Code:

```
// Example: const isEnabled = await redis.get("feature:newUI"); if(isEnabled) showNewUI(); else showOldUI();
```

■ Used for safe releases and experimentation.

77 What is chaos engineering?

Practice of intentionally introducing failures to test system resilience. Ensures system can handle unexpected failures. Tools: Chaos Monkey.

Example / Code:

```
// Example: // Randomly kill instances to test recovery
```

■ Helps build fault-tolerant systems.

78 What is SLO, SLA, SLI?

SLI: metric (latency, error rate). SLO: target (e.g., 99.9% uptime). SLA: legal agreement with penalties.

Example / Code:

```
// Example: // SLI: API latency < 200ms // SLO: 99% requests meet SLI // SLA: compensation if violated
```

■ Used for reliability tracking.

79 What is multi-region deployment?

Deploy system across multiple geographic regions for low latency and high availability. Uses replication and geo-routing.

Example / Code:

```
// Example: // AWS regions: us-east-1, ap-south-1
```

■ Improves fault tolerance and performance.

80 What is data replication strategy?

Replicate data across nodes for availability. Types: synchronous (strong consistency) and asynchronous (eventual consistency).

■ Critical for fault tolerance.

81 What is backup strategy?

Regularly store copies of data. Types: full, incremental, differential backups. Store in separate location (S3, cold storage).

■ Ensures data recovery.

82 What is disaster recovery?

Plan to recover system after failure. Includes backups, failover systems, and recovery procedures.

Example / Code:

```
// Example: // Primary DB fails → switch to replica
```

■ Minimizes downtime.

83 What is zero downtime deployment?

Deploy new version without interrupting users. Techniques: blue-green, rolling updates.

Example / Code:

```
// Example: // Deploy new version gradually
```

■ Ensures uninterrupted service.

84 What is data migration strategy?

Process of moving data safely between systems. Strategies: big bang, phased migration.

Example / Code:

```
// Example: // Migrate users table in batches
```

■ Avoid downtime and data loss.

85 What is cost optimization?

Reducing infrastructure cost while maintaining performance.

Example / Code:

```
// Techniques: // Use autoscaling, spot instances, caching
```

■ Important for large-scale systems.

86 Trade-offs in system design?

Every design decision has trade-offs: consistency vs availability, latency vs throughput, cost vs performance.

■ No perfect solution — choose based on requirements.

87 When to use NoSQL vs SQL?

SQL: structured data, transactions. NoSQL: flexible schema, high scalability.

Example / Code:

```
// SQL: MySQL, Postgres // NoSQL: MongoDB, DynamoDB
```

■ Choose based on use case.

88 How to design for security?

Use authentication, authorization, encryption, validation, rate limiting.

Example / Code:

```
// Example: // JWT auth + HTTPS
```

■ Security is critical in system design.

89 Real-world architecture discussion?

Discuss trade-offs, scaling, caching, DB design, APIs, monitoring. Explain decisions clearly.

■ Focus on reasoning, not just solution.

90 End-to-end system design approach?

Steps: 1) Requirements 2) High-level design 3) Components 4) Scaling 5) Bottlenecks 6) Trade-offs.

Example / Code:

```
// Flow: // Client → API → Service → DB → Cache
```

■ Structured thinking is key in interviews.

Part 8
TESTING
(40 Questions)

■ Basics (Q1–Q20)

1 What is testing in software?

Testing verifies software works as expected, is free of bugs, and meets requirements. It ensures correctness, reliability, performance, and security. Without testing, bugs reach production causing real user harm and costly fixes.

Example / Code:

```
// Example: function add(a,b){return a+b;} // Test: test("add works",()=>{
expect(add(2,3)).toBe(5); expect(add(-1,1)).toBe(0); expect(add(0,0)).toBe(0);
});
```

■ Cost of fixing a bug in production is 100x more than fixing it in development.

2 Types of testing?

Unit (individual functions), Integration (modules together), E2E (full user flow), Regression (no old bugs), Smoke (basic sanity), Performance (speed/load), Security (vulnerabilities), Contract (API compatibility).

Example / Code:

```
// Testing pyramid: // Unit tests (many, fast, cheap) // Integration (some,
medium) // E2E tests (few, slow, expensive) // Ratio: 70% unit, 20% integration,
10% E2E
```

3 Unit vs integration vs E2E?

Unit: test single function/class in isolation — mock all dependencies. Integration: test how modules work together (API + DB). E2E: test complete user flow in real browser.

Example / Code:

```
// Unit test (isolated): test("calcTax",()=>expect(calcTax(100,0.18)).toBe(18));
// Integration test: const res=await
request(app).post("/register").send({email:"a@b.com"});
expect(res.status).toBe(201); // E2E test (Playwright): await
page.goto("/login"); await page.fill("[name=email]","a@b.com"); await
page.click("[type=submit]"); await expect(page).toHaveURL("/dashboard");
```

4 What is a test case?

A test case specifies: inputs, expected outputs, and conditions to verify specific functionality. Good test cases cover: happy path, edge cases, error cases.

Example / Code:

```
// Good test case covers happy path + edge cases: describe("validateEmail",()=>{
it("accepts valid email",()=>expect(validateEmail("a@b.com")).toBe(true));
it("rejects empty string",()=>expect(validateEmail("")).toBe(false)); it("rejects
no @",()=>expect(validateEmail("notanemail")).toBe(false)); it("rejects no
domain",()=>expect(validateEmail("a@")).toBe(false)); });
```

5 What is a test suite?

A collection of related test cases grouped using describe(). Organizes tests by feature or component.

Example / Code:

```
describe("User Service",()=>{ describe("register",()=>{ it("creates user with
hashed password",async()=>{...}); it("throws on duplicate email",async()=>{...});
it("validates email format",async()=>{...}); }); describe("login",()=>{
it("returns token for valid credentials",async()=>{...}); it("throws for wrong
password",async()=>{...}); }); });
```

6 What is mocking?

Mocking replaces real dependencies with controlled fake implementations. Allows testing in isolation without real APIs, databases, or external services.

Example / Code:

```
// Mock axios: jest.mock("axios"); const axios=require("axios");
axios.get.mockResolvedValue({data:{users:[{id:1,name:"Alice"}]}}); // Now testing
code that calls axios.get won't hit real API const users=await
userService.getUsers(); expect(users).toHaveLength(1);
expect(axios.get).toHaveBeenCalledWith("/api/users");
```

7 What is stubbing?

Stubbing provides predefined responses for specific function calls. Unlike mocks, stubs just return fixed values without verifying how they were called.

Example / Code:

```
// Stub DB call to return fake data:
jest.spyOn(User,"findById").mockResolvedValue({ _id:"123", name:"Alice",
email:"a@b.com" }); // Test runs without real DB: const user=await
userService.getById("123"); expect(user.name).toBe("Alice"); // DB stub returns
Alice regardless of ID
```

8 What is spying?

Spying tracks function calls (how many times called, with what args) without replacing the actual implementation. Useful to verify side effects.

Example / Code:

```
// Spy on console.error: const
consoleSpy=jest.spyOn(console,"error").mockImplementation(()=>{}); await
userService.handleError(new Error("DB error"));
expect(consoleSpy).toHaveBeenCalledWith("DB error");
expect(consoleSpy).toHaveBeenCalledTimes(1); console.error.mockRestore(); //
Original console.error restored
```

9 What is assertion?

Assertion checks if actual result matches expected result. Tests pass when all assertions pass; fail when any assertion fails.

Example / Code:

```
// Jest matchers: expect(2+2).toBe(4); // strict equality
expect({a:1}).toEqual({a:1}); // deep equality expect(null).toBeNull();
expect(arr).toHaveLength(3); expect(fn).toThrow("Error msg");
expect(str).toContain("hello"); expect(num).toBeGreaterThan(0); await
expect(asyncFn()).rejects.toThrow("Error");
```

10 What is code coverage?

Measures what percentage of code is executed by tests. Coverage types: statement, branch, function, line. Jest --coverage generates reports.

Example / Code:

```
// Jest coverage: jest --coverage // Output: // Statements: 87% | Branches: 72% |
Functions: 90% // Find uncovered code in coverage/lcov-report/index.html //
package.json: "jest":{ "coverageThreshold":{
"global":{"lines":80,"functions":80,"branches":70} } } // Build fails if coverage
```

```
drops below threshold
```

■ 70%+ coverage for production apps. 100% is often impractical.

11 What is TDD (Test-Driven Development)?

Write the test FIRST, then write code to make it pass, then refactor. Cycle: Red (test fails) -> Green (make it pass) -> Refactor (clean up).

Example / Code:

```
// TDD cycle example: // 1. RED: Write failing test test("calcDiscount applies 10% for premium users",()=>{ expect(calcDiscount(100,"premium")).toBe(90); }); // 2. GREEN: Write minimum code to pass function calcDiscount(price,userType){ if(userType=="premium") return price*0.9; return price; } // 3. REFACTOR: Improve without changing behavior const DISCOUNTS={premium:0.1,vip:0.2}; function calcDiscount(price,userType){ const discount=DISCOUNTS[userType]||0; return price*(1-discount); }
```

12 What is BDD (Behavior-Driven Development)?

BDD describes behavior in natural language (Given/When/Then). Bridges communication between devs, QA, and business. Uses tools like Cucumber.

Example / Code:

```
// Cucumber feature file: // Feature: User Login // Scenario: Successful login // Given a user "alice@test.com" exists // When they submit valid credentials // Then they receive a JWT token // In practice, Jest tests with BDD style: describe("User Login",()=>{ it("returns token given valid credentials",async()=>{ const res=await request(app).post("/auth/login") .send({email:"alice@test.com",password:"Pass1234!"}); expect(res.status).toBe(200); expect(res.body.token).toBeDefined(); }); });
```

13 What is snapshot testing?

Captures rendered output (component tree or JSON) and saves it. Future runs compare against saved snapshot. Fails if output changes unexpectedly.

Example / Code:

```
import {render} from "@testing-library/react"; import Button from "./Button"; test("Button renders correctly",()=>{ const{container}=render(<Button label="Click me" variant="primary"/>); expect(container).toMatchSnapshot(); // First run: creates __snapshots__/Button.test.js.snap // Future runs: fails if rendered HTML changes }); // Update snapshot when change is intentional: // jest --updateSnapshot
```

■ Use snapshots for UI components. Update when intentional changes are made.

14 What is regression testing?

Ensures new code changes do not break existing functionality. Run full test suite after every change/merge.

Example / Code:

```
// Regression test approach: // 1. Bug found: user can submit form with empty email // 2. Write test: test("rejects empty email",async()=>{ const res=await request(app).post("/register").send({email:""}); expect(res.status).toBe(400); }); // 3. Fix the bug // 4. Test passes = regression prevented // 5. Test stays in suite to prevent recurrence
```

15 What is smoke testing?

Quick basic tests to verify the application starts and core functionality works. Run before more extensive tests.

Example / Code:

```
// Smoke test — does the app start and respond? describe("Smoke Tests",()=>{ it("server starts and responds",async()=>{ const res=await
```

```
request(app).get("/health"); expect(res.status).toBe(200);
expect(res.body.status).toBe("OK"); }); it("database is connected",async()=>{
const res=await request(app).get("/health"); expect(res.body.db).toBe(true); });
it("main page loads",async()=>{ const res=await request(app).get("/");
expect(res.status).toBeLessThan(500); }); });
```

16 What is sanity testing?

Quick focused check of specific functionality after a change. Narrower than smoke testing. Verifies the specific change works without full regression.

Example / Code:

```
// Sanity test after changing password hashing: describe("Password hashing sanity
check",()=>{ it("hashes password on register",async()=>{ const res=await
request(app).post("/register").send({email:"t@t.com",password:"Pass1!"}); const
user=await User.findOne({email:"t@t.com"}).select("+password");
expect(user.password).not.toBe("Pass1!"); expect(user.password).toMatch(/^\$2/);
}); });
```

17 What is Jest?

Most popular JS testing framework. Built-in: test runner, assertions, mocking, coverage. Zero config for most projects.

Example / Code:

```
// jest.config.js: module.exports={ testEnvironment:"node", // or "jsdom" for
React setupFilesAfterFramework:["./jest.setup.js"],
collectCoverageFrom:["src/**/*.js","!src/**/*.test.js"],
coverageThreshold:{global:{lines:80}} }; // Run: // jest — run all tests // jest
--watch — watch mode // jest --coverage — with coverage // jest auth.test.js —
specific file // jest -t "login" — tests matching pattern
```

18 What is Mocha?

Flexible Node.js testing framework. Needs separate assertion (Chai) and mocking (Sinon) libraries. More control but more setup.

Example / Code:

```
const{expect}=require("chai"); const sinon=require("sinon");
describe("UserService",()=>{ let sandbox;
beforeEach(()=>sandbox=sinon.createSandbox()); afterEach(()=>sandbox.restore());
it("finds user by email",async()=>{ const
stub=sandbox.stub(User,"findOne").resolves({email:"a@b.com"}); const user=await
userService.findByEmail("a@b.com"); expect(user.email).to.equal("a@b.com");
expect(stub.calledOnce).to.be.true; }); });
```

19 What is Cypress?

End-to-end testing tool for browser-based apps. Real browser execution, time-travel debugging, automatic waiting.

Example / Code:

```
// cypress/e2e/login.cy.js: describe("Login flow",()=>{
beforeEach(()=>cy.visit("/login")); it("logs in with valid credentials",()=>{
cy.get("[name=email]").type("alice@test.com");
cy.get("[name=password]").type("Pass1234!"); cy.get("[type=submit]").click();
cy.url().should("include","/dashboard"); cy.contains("Welcome,
Alice").should("be.visible"); }); it("shows error for wrong password",()=>{
cy.get("[name=email]").type("alice@test.com");
cy.get("[name=password]").type("wrongpassword"); cy.get("[type=submit]").click();
cy.contains("Invalid credentials").should("be.visible"); }); });
```

20 What is Playwright?

Modern E2E testing tool. Supports Chromium, Firefox, Safari. Faster than Cypress, better for parallel testing.

Example / Code:

```
// playwright.test.ts: import{test,expect}from "@playwright/test"; test("user can register and login",async({page})=>{ await page.goto("http://localhost:3000/register"); await page.fill("[name=email]","new@test.com"); await page.fill("[name=password]","Pass1234!"); await page.click("[type=submit]"); await expect(page).toHaveURL("/dashboard"); await expect(page.getByText("Welcome")).toBeVisible(); }); // Run all browsers in parallel: // npx playwright test --project=chromium,firefox,webkit
```

■ Playwright is faster than Cypress and supports multiple browsers natively.

■ Advanced (Q21–Q40)

21 How to test React components?

Use React Testing Library (RTL). Test behavior not implementation. Query by role, label, text — not CSS selectors.

Example / Code:

```
import{render,screen,fireEvent,waitFor}from "@testing-library/react"; import userEvent from "@testing-library/user-event"; import LoginForm from "./LoginForm"; test("submits form with correct data",async()=>{ const mockSubmit=jest.fn().mockResolvedValue({token:"abc"}); const user=userEvent.setup(); render(<LoginForm onSubmit={mockSubmit}/>); await user.type(screen.getByLabelText(/email/i),"alice@test.com"); await user.type(screen.getByLabelText(/password/i),"Pass1234!"); await user.click(screen.getByRole("button",{name:/login/i})); await waitFor(()=>{ expect(mockSubmit).toHaveBeenCalledWith({email:"alice@test.com",password:"Pass1234!"}); }); });
```

■ getByRole is preferred over getByTestId — forces accessible HTML.

22 How to test React hooks?

Use renderHook from @testing-library/react. Test custom hooks in isolation.

Example / Code:

```
import{renderHook,act}from "@testing-library/react"; import{useCounter}from "./useCounter"; test("counter increments and decrements",()=>{ const{result}=renderHook(()=>useCounter(0)); expect(result.current.count).toBe(0); act(()=>result.current.increment()); expect(result.current.count).toBe(1); act(()=>result.current.decrement()); expect(result.current.count).toBe(0); act(()=>result.current.reset()); expect(result.current.count).toBe(0); });
```

23 How to test Express APIs?

Use Jest + Supertest. Export app without calling listen(). Use a test database. Clean data between tests.

Example / Code:

```
const request=require("supertest"); const app=require("../src/app"); // no .listen()! const User=require("../src/models/User"); describe("POST /api/auth/register",()=>{ beforeEach(async()=>await User.deleteMany({})); it("creates user with 201",async()=>{ const res=await request(app) .post("/api/auth/register") .send({name:"Alice",email:"a@test.com",password:"Pass1234!"}); expect(res.status).toBe(201); expect(res.body.token).toBeDefined(); expect(res.body.user.password).toBeUndefined(); }); it("returns 409 for duplicate email",async()=>{ const body={email:"a@test.com",password:"Pass1!"}; await
```

```
request(app).post("/api/auth/register").send(body); const res=await
request(app).post("/api/auth/register").send(body); expect(res.status).toBe(409);
}); });
```

24 Supertest usage?

HTTP testing library for Node.js. Makes requests to your Express app without starting a real server. No port needed.

Example / Code:

```
const request=require("supertest"); const app=require("./app"); // GET request:
const res=await request(app).get("/api/users"); // POST with body: const
res=await request(app).post("/api/users").send({name:"Bob"}); // With auth
header: const res=await
request(app).get("/api/profile").set("Authorization",`Bearer ${token}`); // With
cookie: const res=await
request(app).get("/api/me").set("Cookie",[`token=${jwt}`]); // Upload file: const
res=await request(app).post("/upload").attach("file","./test.jpg");
```

25 Mocking API calls in tests?

Mock network requests with jest.mock, nock, or MSW (Mock Service Worker). MSW is best for React tests.

Example / Code:

```
// Method 1: Jest mock jest.mock("axios"); const axios=require("axios");
axios.post.mockResolvedValueOnce({data:{id:1,name:"Alice"}}); // Method 2: nock
(Node.js HTTP mocking) const nock=require("nock");
nock("https://api.example.com").get("/users/1").reply(200,{id:1,name:"Alice"});
// Method 3: MSW (best for React) import{rest}from "msw"; const
server=setupServer(
rest.get("/api/users",(req,res,ctx)=>res(ctx.json([{id:1,name:"Alice"}]))) );
```

26 Testing async code?

Use async/await in tests. waitFor for async UI updates. For Promises use .rejects/.resolves.

Example / Code:

```
// Test async function: test("fetches user data",async()=>{ const user=await
userService.getById("123"); expect(user.name).toBe("Alice"); }); // Test Promise
rejection: await expect(userService.getById("invalid")).rejects.toThrow("User not
found"); // React async state update: render(<UserProfile userId="123"/>); await
waitFor(()=>{ expect(screen.getByText("Alice")).toBeInTheDocument(); }); // Or
use findBy (auto-waits): const name=await screen.findByText("Alice");
expect(name).toBeInTheDocument();
```

27 Testing Redux?

Test reducers as pure functions. Test thunks with mock dispatch/getState. Use renderWithProviders for connected components.

Example / Code:

```
// Reducer test: test("increment",()=>{ const
state=counterReducer({value:0},{type:"counter/increment"});
expect(state.value).toBe(1); }); // Thunk test: const dispatch=jest.fn(); const
getState=jest.fn(()=>({users:{}})); await fetchUser("123")(dispatch,getState);
expect(dispatch).toHaveBeenCalledWith(expect.objectContaining({
type:"users/fetchById/fulfilled" })); // Connected component: const
renderWithStore=(ui,{initialState}={})=>{ const
store=configureStore({reducer,preloadedState:initialState}); return
render(<Provider store={store}>{ui}</Provider>); };
```

28 Testing Express middleware?

Test middleware as standalone function or as part of request chain.

Example / Code:

```
// Test auth middleware directly: const
authenticate=require("./middleware/authenticate"); test("rejects request without
token",async()=>{ const req={headers:{}}; const
res={status:jest.fn().mockReturnThis(),json:jest.fn()}; const next=jest.fn();
await authenticate(req,res,next); expect(res.status).toHaveBeenCalledWith(401);
expect(next).not.toHaveBeenCalled(); }); test("calls next with valid
token",async()=>{ const token=jwt.sign({id:"123"},process.env.JWT_SECRET); const
req={headers:{authorization:`Bearer ${token}`}}; const res={}; const
next=jest.fn(); await authenticate(req,res,next);
expect(next).toHaveBeenCalledWith(); expect(req.user.id).toBe("123"); });
```

29 Testing database operations?

Use a test database. MongoMemoryServer for in-memory MongoDB. Postgres: use a separate test DB, run migrations.

Example / Code:

```
// MongoMemoryServer setup:
const{MongoMemoryServer}=require("mongodb-memory-server"); let mongoServer;
beforeAll(async()=>{ mongoServer=await MongoMemoryServer.create(); await
mongoose.connect(mongoServer.getUri()); }); afterAll(async()=>{ await
mongoose.disconnect(); await mongoServer.stop(); }); beforeEach(async()=>{ await
mongoose.connection.dropDatabase(); }); // Tests run against real MongoDB in
memory — no mocking needed!
```

30 What is in-memory database for testing?

MongoMemoryServer runs real MongoDB in process memory. Faster than connecting to external DB. No data persists between test runs.

Example / Code:

```
// jest.setup.js: const{MongoMemoryServer}=require("mongodb-memory-server"); let
mongo; beforeAll(async()=>{ mongo=await MongoMemoryServer.create();
process.env.MONGODB_URI=mongo.getUri(); await mongoose.connect(mongo.getUri());
}); afterAll(async()=>{ await mongoose.disconnect(); await mongo.stop(); }); //
Each test gets clean DB via: beforeEach(async()=>{ const
collections=mongoose.connection.collections; for(const coll of
Object.values(collections)) await coll.deleteMany({}); });
```

31 Test isolation best practices?

Each test should be independent. No shared state. Use beforeEach to reset. Never rely on test execution order.

Example / Code:

```
// Bad: tests share state let user; beforeAll(async()=>user=await
User.create({email:"a@b.com"})); test("get user",()=>expect(user).toBeDefined());
// Good: each test creates its own data beforeEach(async()=>await
User.deleteMany({})); test("get user",async()=>{ const user=await
User.create({email:"a@b.com",password:"Pass1!"}); const found=await
User.findById(user._id); expect(found.email).toBe("a@b.com"); });
```

32 What are flaky tests?

Tests that sometimes pass and sometimes fail with same code. Causes: timing/async issues, shared state, random data, external services.

Example / Code:

```
// Flaky test — timing issue: test("async update reflects in UI",()=>{
fireEvent.click(button); expect(screen.getByText("Updated")).toBeInTheDocument();
// Fails sometimes — state update is async! }); // Fixed: test("async update
```

```
reflects in UI",async()=>{ fireEvent.click(button); await waitFor(()=>{
expect(screen.getByText("Updated")).toBeInTheDocument(); }); });
```

33 Parallel testing in Jest?

Jest runs test files in parallel (separate workers) by default. Tests within a file run sequentially.

Example / Code:

```
// jest.config.js: module.exports={ maxWorkers:"50%", // use 50% of CPU //
maxWorkers:4, // or exact number }; // Run tests in band (sequential, for
debugging): jest --runInBand // Parallel is default and usually faster for large
test suites
```

34 CI/CD testing pipeline?

Run tests automatically on every pull request and merge. Fail pipeline if tests fail — block bad code from merging.

Example / Code:

```
# .github/workflows/test.yml: jobs: test: runs-on: ubuntu-latest services:
mongodb: image: mongo:7 ports: ["27017:27017"] steps: - uses: actions/checkout@v4
- uses: actions/setup-node@v4 with: {node-version: 20, cache: npm} - run: npm ci
- run: npm run lint - run: npm test -- --coverage --ci env: TEST_MONGODB_URI:
mongodb://localhost:27017/test - name: Upload coverage uses:
codecov/codecov-action@v3
```

35 Coverage tools in Node?

Jest has built-in coverage via Istanbul. Shows: statement, branch, function, line coverage. Identify untested paths.

Example / Code:

```
// Generate HTML report: jest --coverage # Open: coverage/lcov-report/index.html
# Shows exactly which lines/branches not covered // Enforce coverage: //
jest.config.js: coverageThreshold:{ global:{lines:80,functions:80,branches:70},
"./src/services/payment.js":{lines:95} // stricter for critical files }
```

36 Performance testing?

Test system speed and response time. Tools: k6, Artillery, Apache JMeter. Test: response time, throughput, concurrency.

Example / Code:

```
// k6 performance test: import http from "k6/http"; import{check,sleep}from "k6";
export const options={ vus:100, // 100 virtual users duration:"30s" // for 30
seconds }; export default function(){ const
res=http.get("http://localhost:3000/api/users"); check(res,{ "status
200":r=>r.status===200, "response < 200ms":r=>r.timings.duration<200 });
sleep(1); } // Run: k6 run script.js
```

37 Load testing?

Simulate many concurrent users to find system bottlenecks. Find: max RPS before errors, memory leaks under load, response time degradation.

Example / Code:

```
// Artillery load test: // artillery.yml: config: target: http://localhost:3000
phases: - duration: 60 arrivalRate: 10 # 10 users/second ramp up - duration: 120
arrivalRate: 50 # 50 users/second sustained scenarios: - flow: - get: url:
/api/products - post: url: /api/cart/add json: {productId: "123", qty: 1} // Run:
artillery run artillery.yml
```

38 Security testing?

Test for OWASP Top 10 vulnerabilities: XSS, SQL/NoSQL injection, CSRF, broken auth, etc.

Example / Code:

```
// Test for NoSQL injection: test("rejects NoSQL injection in login",async()=>{
const res=await request(app).post("/auth/login").send({ email:"admin@test.com",
password:{$ne:""} // NoSQL injection attempt! }); expect(res.status).toBe(400);
// validation rejects it }); // Test for XSS: test("sanitizes HTML in user
input",async()=>{ const res=await request(app).post("/api/comments").send({
text:"<script>alert('xss')</script>" }).set("Authorization",`Bearer ${token}`);
expect(res.body.text).not.toContain("<script>"); });
```

39 What is contract testing?

Ensure API compatibility between services. Consumer defines expected contract; provider verifies it. Prevents breaking API changes.

Example / Code:

```
// Pact contract testing: // Consumer test (frontend expectations):
const{pact}=require("@pact-foundation/pact"); await provider.addInteraction({
state:"user exists", uponReceiving:"GET /users/1",
withRequest:{method:"GET",path:"/users/1"}, willRespondWith:{ status:200,
body:{id:"1",name:Matchers.string("Alice"),email:Matchers.email()} } }); //
Provider verifies: does actual API match consumer expectations?
```

40 Testing best practices?

1) Test behavior not implementation. 2) One assertion per test concept. 3) Use descriptive test names. 4) Keep tests fast. 5) Mock external deps. 6) Run in CI on every PR.

Example / Code:

```
// Good test name: test("returns 401 when no auth token provided",...); // Bad:
test("test auth",...); // Good: test behavior
expect(screen.getByRole("alert")).toHaveTextContent("Invalid email"); // Bad:
test implementation expect(component.state.errorMessage).toBe("Invalid email");
// AAA pattern: test("creates user",async()=>{ // Arrange: const
userData={email:"a@b.com",password:"Pass1!"}; // Act: const res=await
request(app).post("/register").send(userData); // Assert:
expect(res.status).toBe(201); expect(res.body.user.email).toBe("a@b.com"); });
```

Part 9
DEVOPS
(40 Questions)

■ Basics (Q1–Q20)

1 What is DevOps?

DevOps combines development (Dev) and operations (Ops) to deliver software faster and more reliably. Culture of automation, collaboration, continuous delivery, and shared responsibility.

Example / Code:

```
// DevOps lifecycle: // Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor // Key practices: // 1. CI/CD automation // 2. Infrastructure as Code (Terraform) // 3. Monitoring and alerting // 4. Containerization (Docker) // 5. Container orchestration (Kubernetes)
```

■ DevOps reduces deployment frequency from quarterly to multiple times per day.

2 CI vs CD?

CI (Continuous Integration): automatically test and integrate code on every push. CD (Continuous Delivery): always ready to deploy. CD (Continuous Deployment): automatically deploy to production.

Example / Code:

```
// CI: developers push code git push origin feature-branch // CI pipeline runs automatically: // 1. Install dependencies (npm ci) // 2. Lint code (eslint) // 3. Run unit tests (jest) // 4. Run integration tests // 5. Build (npm run build) // 6. Security scan (npm audit) // ■ All pass -> PR can be merged // ■ Any fail -> PR blocked
```

3 What is a CI/CD pipeline?

Automated sequence of steps to build, test, and deploy code. Each stage must pass before the next runs.

Example / Code:

```
# GitHub Actions pipeline: name: CI/CD on: {push: {branches: [main]}} jobs: test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-node@v4 with: {node-version: 20} - run: npm ci - run: npm test - run: npm run build deploy: needs: test runs-on: ubuntu-latest steps: - run: ./scripts/deploy.sh
```

4 What is a build process?

Converting source code into deployable artifact. For Node.js: install deps, compile TypeScript, bundle, create Docker image.

Example / Code:

```
# Dockerfile multi-stage build: FROM node:20-alpine AS builder WORKDIR /app COPY package*.json ./ RUN npm ci --only=production COPY . . RUN npm run build FROM node:20-alpine WORKDIR /app COPY --from=builder /app/dist ./dist COPY --from=builder /app/node_modules ./node_modules EXPOSE 3000 CMD ["node","dist/index.js"]
```

5 What is deployment?

Releasing the application to a server or cloud environment. Types: manual (SSH + scripts), automated (CI/CD), blue-green, canary, rolling.

6 What is Docker?

Platform to build, run, and manage containers. Packages app + dependencies into portable container. "Works on my machine" solved.

Example / Code:

```
# Build Docker image: docker build -t my-api:1.0.0 . # Run container: docker run -p 3000:3000 --env-file .env my-api:1.0.0 # List running containers: docker ps # View logs: docker logs my-api-container # Execute command inside container: docker exec -it container_id sh
```

■ Docker ensures your app runs identically in dev, staging, and production.

7 Container vs VM?

VM: full OS per VM (GBs), slow to start (minutes), strong isolation. Container: share host OS kernel (MBs), start in milliseconds, lightweight isolation.

Example / Code:

```
# VM: 2GB+ per instance, minutes to start # Container: ~50MB, starts in <1 second # Same Dockerfile runs on: # - Your laptop (macOS) # - Staging server (Ubuntu) # - Production (AWS ECS) # - Kubernetes pod # Identical behavior everywhere
```

8 What is Docker image?

Read-only template/blueprint for creating containers. Layers of filesystem changes. Tagged with name:version.

Example / Code:

```
# List images: docker images # Pull from Docker Hub: docker pull node:20-alpine # Build from Dockerfile: docker build -t myapp:latest . # Push to registry: docker push myrepo/myapp:1.0.0 # Image layers: # FROM node:20-alpine <- base layer # COPY package*.json ./ <- new layer # RUN npm ci <- new layer # COPY . . <- new layer
```

9 What is Kubernetes (K8s)?

Container orchestration platform. Manages: deployment, scaling, networking, storage for containerized apps.

Example / Code:

```
# Key K8s concepts: # Pod: smallest unit (1+ containers) # Deployment: manages pods (desired replicas) # Service: stable network access to pods # Ingress: HTTP routing into cluster # ConfigMap: non-secret configuration # Secret: passwords, tokens # Create deployment: kubectl apply -f deployment.yaml # Scale: kubectl scale deployment my-api --replicas=5 # Get status: kubectl get pods
```

10 What is a Pod in Kubernetes?

Smallest deployable unit. One or more containers sharing network/storage. Usually one container per pod.

Example / Code:

```
# pod.yaml: apiVersion: v1 kind: Pod spec: containers: - name: api image: myapp:1.0.0 ports: [{containerPort: 3000}] resources: requests: {memory: 256Mi, cpu: 100m} limits: {memory: 512Mi, cpu: 500m} env: - name: NODE_ENV value: production - name: JWT_SECRET valueFrom: secretKeyRef: {name: api-secrets, key: jwt-secret}
```

11 What is a K8s Service?

Provides stable network access to pods. Pods are ephemeral (IP changes). Service provides stable DNS name and IP.

Example / Code:

```
# service.yaml: apiVersion: v1 kind: Service metadata: {name: api-service} spec: selector: {app: my-api} # matches pod labels ports: - port: 80 targetPort: 3000
```

```
type: ClusterIP # internal only # LoadBalancer: external access via cloud LB #
NodePort: access via node port
```

12 What is K8s Ingress?

Manages external HTTP/HTTPS access to services. Like reverse proxy — routes paths to services.

Example / Code:

```
# ingress.yaml: apiVersion: networking.k8s.io/v1 kind: Ingress spec: rules: -
host: myapp.com http: paths: - path: /api pathType: Prefix backend: service:
{name: api-service, port: {number: 80}} - path: / pathType: Prefix backend:
service: {name: frontend-service, port: {number: 80}}
```

13 What is K8s scaling?

HPA (Horizontal Pod Autoscaler) automatically adds/removes pods based on CPU/memory or custom metrics.

Example / Code:

```
# hpa.yaml: apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler spec:
scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: my-api minReplicas: 2
maxReplicas: 20 metrics: - type: Resource resource: name: cpu target: {type:
Utilization, averageUtilization: 70} # Scales up when CPU > 70%, down when CPU <
70%
```

14 What is load balancer in K8s?

Service type LoadBalancer provisions cloud load balancer (AWS ALB, GCP LB). Routes external traffic to pods.

Example / Code:

```
# service.yaml with LoadBalancer: spec: type: LoadBalancer selector: {app:
my-api} ports: - port: 443 targetPort: 3000 # Cloud creates external IP
automatically # All traffic on :443 -> pods on :3000
```

15 What is reverse proxy in DevOps?

Server in front of application. Nginx/HAProxy handles: SSL termination, load balancing, caching, rate limiting, static files.

Example / Code:

```
# nginx.conf: server { listen 443 ssl; ssl_certificate /etc/ssl/cert.pem;
location /api { proxy_pass http://api-service:3000; proxy_http_version 1.1;
proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; } location
/static { root /var/www; expires 1y; add_header Cache-Control public; } }
```

16 What is Nginx?

Web server, reverse proxy, load balancer. Handles 10,000+ concurrent connections efficiently. Used for: SSL, static files, proxying to Node.js.

Example / Code:

```
# Install and configure: nginx -t # test config nginx -s reload # reload config #
Nginx upstream (load balancing): upstream nodejs_cluster { server 127.0.0.1:3001;
server 127.0.0.1:3002; server 127.0.0.1:3003; keepalive 64; } server { location /
{ proxy_pass http://nodejs_cluster; } }
```

17 What is environment config in DevOps?

Different config for dev/staging/production. Never hardcode. Use environment variables, config files, or secrets manager.

Example / Code:

```
# Development: NODE_ENV=development DB_URI=mongodb://localhost:27017/myapp-dev
LOG_LEVEL=debug # Production: NODE_ENV=production
DB_URI=mongodb+srv://user:pass@prod-cluster/myapp LOG_LEVEL=error
JWT_SECRET=super-secret-key # Load in code: require("dotenv").config(); const
```

```
PORT=process.env.PORT||3000; // Validate on startup: if(!process.env.JWT_SECRET)
throw new Error("JWT_SECRET required");
```

■ Never commit .env files to git.

18 What is .env file?

Stores environment variables locally. Loaded with dotenv. Never commit to git — add to .gitignore.

Example / Code:

```
# .env: PORT=3000 MONGODB_URI=mongodb://localhost:27017/myapp
JWT_SECRET=mysecretkey REDIS_URL=redis://localhost:6379 # .env.example (commit
this — no real values): PORT= MONGODB_URI= JWT_SECRET= REDIS_URL= # .gitignore:
.env .env.local .env.production
```

19 What is secrets management?

Securely storing sensitive data. Never in code or git. Use: env vars (basic), AWS Secrets Manager, HashiCorp Vault, Kubernetes Secrets.

Example / Code:

```
# AWS Secrets Manager: const{SecretsManagerClient,GetSecretValueCommand}=require(
"@aws-sdk/client-secrets-manager"); const client=new
SecretsManagerClient({region:"ap-south-1"}); const secret=await client.send(new
GetSecretValueCommand({ SecretId:"prod/myapp/jwt-secret" }));
const{JWT_SECRET}=JSON.parse(secret.SecretString); # Kubernetes Secret: kubectl
create secret generic api-secrets \ --from-literal=JWT_SECRET=myvalue \
--from-literal=DB_PASSWORD=dbpass
```

■ Rotate secrets regularly. Never log secrets.

20 What is monitoring?

Continuously observing system health using metrics, logs, and alerts. Detect issues before users do.

Example / Code:

```
// Metrics with Prometheus + prom-client: const
promClient=require("prom-client"); const httpDuration=new promClient.Histogram({
name:"http_request_duration_seconds", help:"Duration of HTTP requests",
labelNames:["method","route","status"] }); // Middleware:
app.use((req,res,next)=>{ const end=httpDuration.startTimer(); res.on("finish",()
=>end({method:req.method,route:req.route?.path,status:res.statusCode})); next();
}); // Grafana dashboard shows: RPS, latency, error rate
```

■ Advanced (Q21–Q40)

21 CI/CD pipeline design for Node.js?

Stages: lint -> test -> security scan -> build -> Docker build -> push to registry -> deploy to staging -> smoke test -> deploy to production.

Example / Code:

```
# Full pipeline (.github/workflows/deploy.yml): jobs: test: steps: - run: npm ci
- run: npm run lint - run: npm test -- --coverage - run: npm audit
--audit-level=high build: needs: test steps: - run: docker build -t
$ECR_REGISTRY/app:$GITHUB_SHA . - run: docker push $ECR_REGISTRY/app:$GITHUB_SHA
deploy-staging: needs: build steps: - run: kubectl set image deployment/api
api=$ECR_REGISTRY/app:$GITHUB_SHA - run: ./scripts/smoke-test.sh
https://staging.myapp.com deploy-prod: needs: deploy-staging environment:
production # requires manual approval steps: - run: kubectl set image
deployment/api api=$ECR_REGISTRY/app:$GITHUB_SHA
```

22 GitHub Actions best practices?

Cache dependencies, use specific action versions, use secrets for sensitive data, matrix testing for multiple Node versions.

Example / Code:

```
# Optimized GitHub Actions: jobs: test: strategy: matrix: node: [18, 20, 22] # test all versions steps: - uses: actions/setup-node@v4 with: {node-version: "${{ matrix.node }}", cache: npm} - run: npm ci # faster than npm install - run: npm test - name: Upload coverage if: matrix.node == 20 # only once uses: codecov/codecov-action@v3
```

23 Dockerfile best practices?

Use minimal base images, multi-stage builds, non-root user, .dockerignore, pin versions, combine RUN commands.

Example / Code:

```
# Dockerfile best practices: FROM node:20-alpine AS builder # alpine = smaller WORKDIR /app # Copy deps files first (layer caching) COPY package*.json ./ RUN npm ci --only=production # ci = exact versions COPY . . RUN npm run build FROM node:20-alpine WORKDIR /app # Non-root user (security) RUN addgroup -g 1001 appgroup && \ adduser -u 1001 -G appgroup -s /bin/sh -D appuser COPY --from=builder --chown=appuser:appgroup /app/dist ./dist COPY --from=builder --chown=appuser:appgroup /app/node_modules ./node_modules USER appuser EXPOSE 3000 HEALTHCHECK CMD wget -qO- http://localhost:3000/health || exit 1 CMD ["node","dist/index.js"]
```

24 Multi-stage Docker build?

Use multiple FROM statements. Builder stage has build tools (devDeps). Production stage only has runtime deps. Dramatically reduces image size.

Example / Code:

```
# Without multi-stage: 1.2GB image # With multi-stage: 120MB image # Stage 1: build TypeScript FROM node:20-alpine AS builder WORKDIR /app COPY package*.json tsconfig.json ./ RUN npm ci # includes devDeps for building COPY src/ ./src/ RUN npm run build # compile TS -> JS # Stage 2: production (no devDeps, no TS files) FROM node:20-alpine WORKDIR /app COPY package*.json ./ RUN npm ci --only=production # no devDeps COPY --from=builder /app/dist ./dist CMD ["node","dist/index.js"]
```

■ Multi-stage builds can reduce Node.js Docker image from 1.2GB to ~120MB.

25 Container orchestration with K8s?

K8s manages: desired state (replicas), rolling updates, health checks, auto-restart, service discovery, secrets.

Example / Code:

```
# deployment.yaml: apiVersion: apps/v1 kind: Deployment spec: replicas: 3 strategy: type: RollingUpdate rollingUpdate: {maxUnavailable: 1, maxSurge: 1} template: spec: containers: - name: api image: myapp:1.0.0 livenessProbe: # restart if unhealthy httpGet: {path: /health, port: 3000} initialDelaySeconds: 30 readinessProbe: # only route traffic when ready httpGet: {path: /ready, port: 3000} initialDelaySeconds: 10
```

26 Auto-scaling in K8s?

HPA scales pods based on CPU/memory/custom metrics. VPA adjusts resource requests. KEDA for event-driven scaling (queue depth).

Example / Code:

```
# KEDA (Kubernetes Event-driven Autoscaling): # Scale based on Redis queue depth: apiVersion: keda.sh/v1alpha1 kind: ScaledObject spec: scaleTargetRef: {name: worker-deployment} minReplicaCount: 0 # scale to zero! maxReplicaCount: 50 triggers: - type: redis metadata: address: redis:6379 listName: job-queue
```

```
listLength: "5" # 1 pod per 5 jobs
```

27 Rolling deployment?

Update pods gradually — never all at once. Zero downtime. New version replaces old one by one.

Example / Code:

```
# Rolling update strategy: strategy: type: RollingUpdate rollingUpdate:
maxUnavailable: 25% # max 25% pods down at once maxSurge: 25% # max 25% extra pods
# Update process: # Pod 1: new version deployed, old terminated # Pod 2: new
version deployed, old terminated # ... continues until all pods updated # Traffic
routes to healthy pods only # Deploy: kubectl set image deployment/api
api=myapp:2.0.0 # Monitor: kubectl rollout status deployment/api # Rollback if
needed: kubectl rollout undo deployment/api
```

28 Canary deployment?

Route small % of traffic to new version. Monitor for errors. Gradually increase. Full rollout or rollback based on metrics.

Example / Code:

```
# Using Nginx for canary: upstream backend { server api-stable:3000 weight=9; #
90% traffic server api-canary:3000 weight=1; # 10% traffic } # Using Istio (K8s
service mesh): apiVersion: networking.istio.io/v1alpha3 kind: VirtualService
spec: http: - route: - destination: {host: api, subset: stable} weight: 90 -
destination: {host: api, subset: canary} weight: 10 # Monitor canary: # - Error
rate same as stable? # - Latency same? # - No new alerts? # -> Yes: increase to
25%, 50%, 100% # -> No: route 0% to canary (instant rollback)
```

29 Blue-green deployment?

Two identical environments (blue=current, green=new). Deploy new version to green. Switch traffic. Instant rollback by switching back.

Example / Code:

```
# Blue-Green with Nginx: # 1. Both environments running: # blue: api-blue:3000
(serving traffic) # green: api-green:3000 (new version, idle) # 2. Deploy new
version to green: docker-compose -f docker-compose.green.yml up -d # 3. Run smoke
tests on green: curl https://green.myapp.com/health # 4. Switch traffic (update
Nginx upstream): upstream backend { server api-green:3000; } nginx -s reload # 5.
Keep blue running for quick rollback # upstream backend { server api-blue:3000; }
# instant rollback
```

30 Centralized logging with ELK?

ELK Stack: Elasticsearch (search/store), Logstash (collect/transform), Kibana (visualize). Beats for log shipping.

Example / Code:

```
# Application logs -> Filebeat -> Logstash -> Elasticsearch -> Kibana //
Structured logging (pino): const pino=require("pino"); const
logger=pino({level:"info"}); // Log with context: logger.info({ reqId:"abc-123",
userId:"user-456", method:"POST", path:"/api/orders", duration:45, status:201
},"Order created"); // Kibana can search: // reqId:"abc-123" AND status:>=400 //
Shows all related logs for a request across services
```

31 Infrastructure as Code (IaC)?

Manage infrastructure using code instead of manual UI. Version-controlled, repeatable, consistent.

Example / Code:

```
# Terraform for AWS infrastructure: provider "aws" { region = "ap-south-1" }
resource "aws_instance" "app" { ami = "ami-0c55b159cbfafe1f0" instance_type =
"t3.medium" tags = { Name = "my-api", Environment = "production" } } resource
"aws_db_instance" "mongodb" { identifier = "myapp-db" instance_class =
```

```
"db.t3.medium" engine = "docdb" } # Commands: terraform init terraform plan #
preview changes terraform apply # apply changes terraform destroy # remove
resources
```

■ IaC enables creating identical environments in minutes.

32 Terraform for Node.js apps?

Define AWS ECS, RDS, ElastiCache, ALB infrastructure in code. Same infra for all environments.

Example / Code:

```
# terraform/main.tf # ECS Fargate for Node.js: resource "aws_ecs_service" "api" {
name = "node-api" cluster = aws_ecs_cluster.main.id task_definition =
aws_ecs_task_definition.api.arn desired_count = 3 launch_type = "FARGATE"
load_balancer { target_group_arn = aws_alb_target_group.api.arn container_name =
"api" container_port = 3000 } }
```

33 AWS services for Node.js apps?

EC2 (VMs), ECS/Fargate (containers), Lambda (serverless), RDS (SQL), DocumentDB/Atlas (MongoDB), ElastiCache (Redis), S3 (storage), CloudFront (CDN), ALB (load balancer).

Example / Code:

```
# Node.js on AWS ECS (Fargate): # 1. Build Docker image # 2. Push to ECR (Elastic
Container Registry) # 3. ECS pulls from ECR # 4. ALB routes traffic # 5.
Auto-scaling based on CPU # Serverless (Lambda): # AWS Lambda + API Gateway #
Auto-scales to 0, pay per request # Good for event-driven, low traffic # Bad for
WebSockets, long-running processes
```

34 What is Prometheus + Grafana?

Prometheus: collects and stores metrics. Grafana: visualizes metrics in dashboards. Standard monitoring stack.

Example / Code:

```
// Node.js metrics with prom-client: const prometheus=require("prom-client"); //
Default metrics (CPU, memory, event loop): prometheus.collectDefaultMetrics(); //
Custom metric: const requestCounter=new prometheus.Counter({
name:"http_requests_total", help:"Total HTTP requests",
labelNames:["method","route","status"] }); // Expose metrics endpoint:
app.get("/metrics",async(req,res)=>{
res.set("Content-Type",prometheus.register.contentType); res.end(await
prometheus.register.metrics()); }); // Prometheus scrapes /metrics every 15s //
Grafana queries Prometheus and shows dashboards
```

35 What is alerting?

Automatically notify when system metrics exceed thresholds. Tools: Alertmanager (Prometheus), PagerDuty, Slack.

Example / Code:

```
# Alertmanager rules: groups: - name: node-api rules: - alert: HighErrorRate
expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05 for: 5m labels:
{severity: critical} annotations: summary: "Error rate > 5%" description: "API
error rate is {{ $value | humanize }}" - alert: HighLatency expr:
histogram_quantile(0.99,http_request_duration_seconds) > 1 for: 5m annotations:
summary: "P99 latency > 1 second"
```

36 What is full observability?

Three pillars: Logs (what happened), Metrics (how much/fast), Traces (where in distributed system). Together enable deep system understanding.

Example / Code:

```
// OpenTelemetry in Node.js: const{NodeSDK}=require("@opentelemetry/sdk-node");
const sdk=new NodeSDK({ traceExporter:new
```

```
OTLPTraceExporter({url:"http://jaeger:4318/v1/traces"}), instrumentations:[ new HttpInstrumentation(), new ExpressInstrumentation(), new MongooseInstrumentation() ] }); sdk.start(); // Automatically traces: // - HTTP requests // - Express routes // - MongoDB operations // See full request flow across services in Jaeger UI
```

37 Health checks in DevOps?

K8s livenessProbe (restart if unhealthy) and readinessProbe (only route traffic if ready). Essential for zero-downtime.

Example / Code:

```
// Express health endpoints: app.get("/health",(req,res)=>{ // Simple alive check (liveness): res.json({status:"OK",pid:process.pid}); });
app.get("/ready",async(req,res)=>{ // Full readiness check: const checks={ db: mongoose.connection.readyState===1, redis: redisClient.status==="ready" }; const ready=Object.values(checks).every(Boolean); res.status(ready?200:503).json({ ready,checks, uptime:process.uptime() }); }); // K8s: only routes traffic when /ready returns 200 // K8s: restarts pod when /health fails
```

38 Cost optimization in cloud?

Auto-scaling (pay for what you use), spot instances (80% cheaper for fault-tolerant workloads), right-sizing, reserved instances.

Example / Code:

```
# Cost optimization strategies: # 1. Auto-scaling (scale to 0 at night) # 2. Spot instances for workers (80% discount) # 3. Right-size instances (monitor actual CPU/RAM usage) # 4. Reserved instances for predictable workloads (1-3yr) # 5. S3 lifecycle policies (move old data to Glacier) # 6. CloudFront CDN (reduce origin requests) # 7. Database connection pooling (less DB instances) # Monitor costs: # AWS Cost Explorer, Budget Alerts # Alert when bill exceeds threshold
```

39 Zero downtime deployment?

Deploy new version without any interruption to users. Techniques: rolling updates, blue-green, canary.

Example / Code:

```
# Zero downtime requirements: # 1. Multiple replicas (never deploy to all at once) # 2. Health checks (K8s waits until new pods healthy) # 3. Graceful shutdown (finish in-flight requests) # 4. readinessProbe (don't route until ready) // Graceful shutdown (critical for zero downtime): process.on("SIGTERM",()=>{ // 1. Stop accepting new connections server.close(async()=>{ // 2. Wait for in-flight requests to complete await new Promise(r=>setTimeout(r,5000)); // 3. Close DB connections await mongoose.disconnect(); process.exit(0); }); // 4. Force exit if graceful fails setTimeout(()=>process.exit(1),30000); });
```

■ Graceful shutdown is essential for zero downtime deployments.

40 Production DevOps best practices?

1) CI/CD for all deployments. 2) Infrastructure as Code. 3) Monitoring + alerting. 4) Centralized logging. 5) Secrets management. 6) Health checks. 7) Graceful shutdowns. 8) Multiple replicas. 9) Regular backups. 10) Disaster recovery plan.

Example / Code:

```
# Production checklist: # ■ CI/CD pipeline with tests # ■ Docker + K8s with health checks # ■ Auto-scaling (HPA) # ■ Rolling deployments # ■ Prometheus + Grafana monitoring # ■ ELK Stack centralized logging # ■ PagerDuty alerting # ■ Secrets in Vault/AWS Secrets Manager # ■ Regular DB backups # ■ Load testing before major releases # ■ Runbook for common incidents
```

Part 10
BEHAVIORAL
(50 Questions)

■ Core Behavioral (Q1–Q25)

1 Tell me about yourself?

Structure: Present role -> Key achievement -> Past background -> Why this opportunity. Keep to 90 seconds. Include tech stack, one quantified achievement, and connect to why you want this role.

Example / Code:

```
// Template structure: // "I am a Full-Stack Developer specializing in MERN
stack. // Currently at [Company], I [what you do + key achievement]. // Before
that, I [relevant past experience]. // I'm excited about this role because
[specific reason // about their product/mission/tech stack]. // I believe my
experience with [relevant skill] // directly applies to [their challenge]." //
Key tips: // - Quantify: "reduced load time by 40%" // - Show passion: "I love
building real-time systems" // - Research company: mention their product
```

■ Practice until it sounds natural (90 seconds). End with a bridge to why you are here.

2 What is your biggest strength?

Pick strength relevant to the role. Give specific example with impact. Show self-awareness — acknowledge how you use it constructively.

Example / Code:

```
// Good answer structure: // "My biggest strength is [specific skill]. // For
example, at [Company] I [specific situation]. // This resulted in [measurable
outcome]. // I use this strength to [how it benefits the team/product]." //
Strong strengths for MERN developers: // - Problem-solving under pressure // -
Building scalable systems // - Cross-functional collaboration // - Fast learning
of new technologies // - Code quality and review skills
```

■ Make it specific and verifiable, not generic ("I'm a hard worker").

3 What is your weakness?

Pick a real weakness you are actively improving. Show self-awareness and growth mindset. Avoid clichés like "I work too hard".

Example / Code:

```
// Good weakness answer: // "I used to struggle with [real weakness]. // For
example, early in my career I [specific example]. // I recognized this when
[moment of awareness]. // Since then, I have been working on it by [specific
actions]. // Now I [improvement with example]." // Real developer weaknesses: //
- Estimating task time accurately (improved with time-boxing) // - Saying no to
scope creep (learned to ask for priorities) // - Writing tests early (now
practice TDD) // - Asking for help too late (now set 30-min rule before asking
```

4 Why should we hire you?

Connect your specific skills to their specific needs. Show you understand their challenges. Give 2-3 concrete reasons backed by examples.

Example / Code:

```
// Structure: // "You should hire me because: // 1. I have direct experience with
[their main tech/challenge] // 2. I have solved problems similar to [their
problem] at [past role] // 3. I bring [unique value] that others might not // I
noticed from your job description that you need someone // who can [specific
```

```
requirement]. In my current role, // I [specific achievement that matches]. // I
am also excited about [specific aspect of their product] // and have ideas about
[relevant contribution]." // Key: research the company deeply before the
interview
```

5 Why do you want this job?

Show genuine interest in the company, product, and tech challenges. Research the company thoroughly. Connect their mission to your goals.

Example / Code:

```
// Good answer: // "Three reasons I want this specifically: // 1. Product: I use
[their product] and believe in [mission] // 2. Tech: I want to work at scale with
[their tech challenges] // 3. Growth: I want to learn [specific skill] your team
excels at // Avoid vague: "It's a great opportunity" // Avoid desperate: "I need a
job" // Be specific: "Your real-time notification system // using Kafka interests
me because I want to // work at that scale of event processing"
```

6 Tell me about a challenge you overcame?

Use STAR method: Situation -> Task -> Action -> Result. Focus on YOUR specific actions and the measurable outcome.

Example / Code:

```
// STAR example: // S: "At my company, our loan processing API // was failing 15%
of the time at peak load" // T: "I was responsible for finding and fixing // the
root cause" // A: "I profiled the app with clinic.js and found // the event loop
was blocking on synchronous // file reads. I rewrote them as async streams // and
added connection pooling to MongoDB" // R: "Failure rate dropped to 0.1%, p99
latency // improved from 2.3s to 190ms. Zero incidents // in the 6 months since"
// Key: specific technical detail + measurable outcome
```

■ The more specific the better. "I improved performance" vs "p99 latency dropped from 2.3s to 190ms".

7 How did you handle a conflict with a teammate?

Show emotional intelligence, professionalism, and communication skills. Focus on resolution, not blame.

Example / Code:

```
// Good answer structure: // S: Describe the disagreement professionally // A:
How you approached it (direct conversation, not escalation) // R: What was
resolved and what you learned // Example: // "A senior developer and I disagreed
on using // REST vs GraphQL for a new service. // Instead of arguing, I prepared a
comparison // document with benchmarks and trade-offs. // We had a technical
discussion with the team. // We chose REST for initial release but agreed // to
revisit GraphQL if query complexity grew. // The collaborative approach
strengthened our // technical decision-making process."
```

■ Never badmouth a colleague. Show you can disagree professionally and find resolution.

8 Tell me about a leadership experience?

Leadership does not require a title. Leading a feature, mentoring, driving a technical decision, or improving team processes all count.

Example / Code:

```
// Example (tech lead without title): // "Our team was struggling with
inconsistent // code reviews taking too long. // I took initiative to create a
code review // checklist and PR template. // I facilitated a team discussion to
align // on standards. // Review time decreased from 3 days to 6 hours // and
onboarding new devs became easier. // Three developers told me it improved their
// confidence in submitting PRs."
```

9 Tell me about a failure or mistake?

Interviewers want to see: self-awareness, ownership (no blame), learning, and how you applied that learning.

Example / Code:

```
// Good failure answer: // "I once deployed a change that caused a // production outage for 45 minutes. // I had skipped the staging deployment // because it seemed like a small CSS change. // I take full responsibility — I broke the process. // I immediately rolled back, communicated to // the team, and wrote a detailed post-mortem. // I added a deployment gate requiring staging // verification even for small changes. // I also set up automated smoke tests. // That was 2 years ago — no deployment // outage since."
```

■ Owning a mistake and showing concrete learning is impressive. Deflecting is a red flag.

10 Tell me about a success story?

Quantify the impact. Show your specific contribution in a team effort. Connect it to business value.

Example / Code:

```
// Good success story: // "At [Company], I redesigned our user dashboard // API from REST to GraphQL. // This reduced the number of API calls from // 12 to 3 per page load. // Page load time improved by 60%. // User session time increased by 25% in A/B test. // The approach was later adopted for 3 other // features in the app. // I presented the learnings to the whole // engineering team."
```

11 How do you handle pressure and tight deadlines?

Show: prioritization, early communication, systematic approach, self-care. Avoid "I just work harder".

Example / Code:

```
// Good answer: // "When under pressure, I: // 1. Break the work into daily milestones // 2. Communicate early if something is at risk // (not the day before deadline) // 3. Cut scope if needed — ask which features // are truly required vs nice-to-have // 4. Focus on the critical path first // Example: regulatory compliance deadline, // 2 weeks for security updates. // I communicated on day 3 that full audit logging // would take longer. We shipped simplified version // that passed compliance, enhanced it next sprint. // Delivered on time without burning out the team."
```

12 How do you prioritize tasks?

Show a framework: impact vs effort, deadlines, dependencies, stakeholder alignment.

Example / Code:

```
// Prioritization framework: // 1. Urgency: deadline or blocker for others? // 2. Impact: how many users/revenue affected? // 3. Effort: can I finish in time available? // 4. Dependencies: is someone waiting on this? // In practice: // Use Eisenhower matrix: Urgent+Important first // Daily standups to align with team // If two high-priority tasks conflict, // escalate to manager immediately // Never silently delay something important
```

13 Tell me about a time you missed a deadline?

Be honest. Show what happened, what you learned, and what changed in your process.

Example / Code:

```
// Structure: // What happened (no excuses) // What you did about it // What you learned // What you changed // Example: // "I underestimated a feature complexity and // missed a sprint deadline by 3 days. // I should have flagged the risk earlier. // When I realized I would miss it, // I immediately told my manager and PM, // provided a revised estimate, // and asked to reprioritize other tasks. // Since then, I add 20% buffer to estimates // and raise risks on day 2, not day 9."
```

■ Interviewers expect imperfection. They want to see how you handle it.

14 How do you learn new technologies?

Show a systematic approach. Demonstrate you can learn quickly and deeply.

Example / Code:

```
// My learning system: // 1. Official documentation first (not YouTube) // 2. Build something REAL (not just tutorial) // 3. Read source code for deep understanding // 4. Teach it (LinkedIn post, team presentation) // 5. Apply in production gradually // Recent example: learned Docker // - Read official docs (2 days) // - Dockerized existing Node.js app // - Learned multi-stage builds by doing // - Wrote post explaining it to others // - Implemented in production next sprint // - Sharing knowledge solidifies my own understanding
```

15 Why are you leaving your current job?

Be positive and honest. Focus on what you are moving TOWARD, not running FROM. Never badmouth your current employer.

Example / Code:

```
// Good answers: // "I want to work at larger scale. // My current role has been great for learning // but I am ready for more complex challenges." // "I am looking for a role where I can specialize // in [specific area] more deeply." // "The company direction shifted away from the // technical problems that excite me." // Avoid: // "My boss is terrible" // "The company is poorly managed" // "I need more money" (even if true, don't lead with this)
```

16 What are your salary expectations?

Research market rates. Give a range based on research. Show flexibility while being realistic.

Example / Code:

```
// Approach: // 1. Research: Glassdoor, LinkedIn Salary, AmbitionBox // 2. Know your range (bottom and top) // 3. Anchor slightly above your target // Example: // "Based on my research of market rates // for senior MERN developers in [city] // and my [X] years of experience, // I am looking for [range]. // But I am flexible depending on // the total compensation package, // growth opportunities, and role scope." // Tip: If asked first, say: // "I would like to understand the full role // first — what budget do you have in mind?"
```

17 Where do you see yourself in 5 years?

Show ambition aligned with the role. Show commitment to growing here, not planning to leave.

Example / Code:

```
// Good answer: // "In the short term, I want to master [their stack] // and deliver real value on [their product]. // In 2-3 years, I see myself taking on more // architecture responsibility and mentoring // junior developers. // Long-term, I want to be a tech lead who // influences key technical decisions. // I am drawn to [this company] specifically // because [their growth trajectory/product] // gives me room to grow alongside the team."
```

18 How do you handle feedback?

Show you welcome feedback, act on it, and see it as a gift. Distinguish between personal criticism and professional feedback.

Example / Code:

```
// Good answer: // "I actively seek feedback rather than // waiting for annual reviews. // After each PR review, I think about // what patterns I can improve. // When I receive critical feedback, // my instinct is to listen fully first, // ask clarifying questions to understand, // then decide what to act on. // Example: a senior dev once told me my // code had no error handling. // Instead of getting defensive, I asked // them to show me examples of good patterns. // That conversation changed how I code."
```

19 Team vs individual work preference?

Show you value both. Give examples of excelling in each context. Adapt to what the role needs.

Example / Code:

```
// Good answer: // "I enjoy both and see them as complementary. // My best work
often comes from deep individual // focus (architecting a solution, debugging) //
combined with team collaboration // (design reviews, pair programming on complex
parts). // At my current role, I do about 60% individual work // and 40%
collaborative, which feels right. // I am most effective when I have blocks of //
uninterrupted time but also regular touchpoints // to align direction with the
team."
```

20 How do you handle criticism?

Show maturity. Distinguish constructive criticism from personal attacks. Show you use criticism to grow.

Example / Code:

```
// Mature response: // "I try to separate my ego from my work. // Code criticism
is not personal criticism. // When someone says my code is wrong, // my first
question is: are they right? // If yes, I thank them and fix it. // If I disagree,
I ask questions to understand // their reasoning before defending my approach. //
The times I have grown most as a developer // were when someone challenged my
assumptions // and I realized I was wrong."
```

21 What motivates you?

Be authentic. Show intrinsic motivation (not just money). Align with what this role offers.

Example / Code:

```
// Genuine motivators for developers: // 1. Solving hard problems (technical
challenges) // 2. Seeing users benefit from what you built // 3. Learning new
technologies and techniques // 4. Mentoring others and growing a team // 5.
Building things at scale // Example: // "I am most motivated when I can see direct
// user impact from my work. // When a performance optimization I made // reduced
load times by 60% // and users started completing checkout more, // that feedback
loop is deeply satisfying."
```

22 What demotivates you?

Be honest but diplomatic. Frame it as conditions that reduce effectiveness, not complaints.

Example / Code:

```
// Diplomatic answer: // "I am least effective when: // 1. Requirements change
frequently without reason // (I thrive with clear, stable goals) // 2. Technical
debt prevents building good solutions // (I like doing things right) // 3. No
opportunity for ownership // (I do best when I can make decisions) // I work best
with clear direction, technical freedom, // and an environment where quality is
valued. // How would you describe the culture here around // technical decisions
and quality?"
```

23 Work-life balance philosophy?

Show you are sustainable and professional. Avoid "I work 24/7" (unsustainable) or "I never work overtime" (inflexible).

Example / Code:

```
// Healthy answer: // "I believe consistent sustainable pace // produces better
results than sprinting and burning out. // I set clear expectations, communicate
risks early, // so we rarely need heroics. // I do go the extra mile for real
emergencies, // but I think good planning prevents most of them. // I also believe
rest and hobbies make me a better // engineer — I come back refreshed with new
ideas."
```

24 How do you manage your time?

Show concrete systems and tools, not vague answers.

Example / Code:

```
// Concrete answer: // "I use a combination of: // 1. Time-boxing: 90-minute deep work blocks // with phone/Slack on do-not-disturb // 2. Daily prioritization: top 3 tasks before // opening email or Slack // 3. Async communication: I batch Slack responses // rather than responding instantly // 4. Weekly review: Friday I review what got done // and plan the next week // 5. Jira/Linear for task tracking // This lets me do 6+ hours of focused work daily // rather than 8 hours of reactive work."
```

25 Describe your work style?

Show self-awareness. Be specific. Align with what the team needs.

Example / Code:

```
// Self-aware answer: // "I am a systems thinker — I like to understand // how pieces fit together before diving in. // I tend to spend more time upfront thinking // through edge cases, which means I write // less code but ship fewer bugs. // I am a written communicator — I prefer // async discussions for complex topics // (gives everyone time to think). // I value code review as a learning tool, // not just a quality gate. // I do my best work in the morning, // so I protect that time for complex tasks."
```

■ Advanced (Q21–Q40)

26 Tell me about a complex project you built?

Show: technical depth, problem-solving, collaboration, delivery. Use specific technical details and business impact.

Example / Code:

```
// Structure: // Problem -> Architecture -> Challenges -> Outcome // Example: // "I designed and built a real-time loan // processing system at Kanmotech. // Problem: manual approval took 3 days, // losing customers to faster competitors. // Architecture: Node.js API + MongoDB + Redis // + WebSocket for real-time status. // Async document verification via BullMQ queue. // Challenges: race conditions in approval flow // (solved with MongoDB atomic transactions), // real-time performance at 100 concurrent loans. // Outcome: processing time dropped from 3 days // to 4 hours. 40% reduction in drop-off rate."
```

27 How did you design a system?

Show design thinking: requirements -> constraints -> trade-offs -> decision. Not just the final answer.

Example / Code:

```
// Good system design story: // "For our notification system, I started by // asking: what are the SLOs? // (Email: 30s, SMS: 10s, Push: 5s) // Then identified constraints: budget, team size // Considered options: // - Simple: direct sends (simple but no retry) // - Queue-based: BullMQ (retry, scale, monitoring) // - Event-driven: Kafka (overkill for our scale) // Chose BullMQ with Redis because: // - Fits our team expertise // - Handles retry logic // - Scales to our needs // - Low operational overhead // Delivered in 2 sprints. SLOs met consistently."
```

28 What is the biggest bug you fixed?

Show: investigation methodology, root cause analysis, fix, prevention. Interviewers love real technical war stories.

Example / Code:

```
// Good bug story: // "Users reported occasional double charges. // It took 2
weeks to reproduce it. // Investigation: // Added detailed logging to payment
flow. // Reproduced under load test: two concurrent // requests both passed 'not
paid' check // before either recorded payment. // Root cause: TOCTOU race
condition. // Both requests read order.status=='pending' // before either wrote
'processing'. // Fix: MongoDB atomic findOneAndUpdate: //
{_id,status:{$ne:'paid'}} filter ensures // only one update succeeds atomically.
// Prevention: idempotency key + DB constraint."
```

■ Race conditions in payment flows are real — atomic DB operations are the correct fix.

29 Tell me about a scaling challenge?

Show you understand system bottlenecks and made data-driven scaling decisions.

Example / Code:

```
// Scaling story: // Our API worked fine at 100 users/day // but started timing
out at 5000 users/day. // Diagnosis: New Relic showed DB queries at // 2-3 seconds
on the user fetch endpoint. // explain() revealed no index on email field. //
Immediate fix: added compound index // {email:1, active:1} -> queries to <5ms. //
For sustained load: added Redis caching // for user profiles (TTL 1hr). // Longer
term: added DB read replica // for analytics queries. // Result: handled 50,000
users/day, // p99 latency <100ms.
```

30 How did you improve performance?

Show measurement-first approach. Quantify before and after.

Example / Code:

```
// Performance story: // "Our React dashboard was taking 8 seconds to load. //
Profiling: // - React DevTools: 3 components re-rendering 100x // - Network: 45
separate API calls on load // - Bundle analyzer: moment.js was 67KB // Fixes: //
1. React.memo + useMemo -> reduced renders by 90% // 2. Replaced 45 calls with 3
batched GraphQL queries // 3. Replaced moment.js with dayjs (2KB) // 4. Added
React.lazy for non-critical components // Result: // Load time: 8s -> 1.2s (85%
improvement) // Lighthouse score: 42 -> 91"
```

■ Always measure before and after. "I optimized it" means nothing without numbers.

31 How did you handle a production incident?

Show calm, systematic approach: detect -> triage -> communicate -> fix -> prevent.

Example / Code:

```
// Incident story (be specific): // "At 2am I got paged: 500 errors spiking. // 1.
Assess: Datadog showed DB connection pool exhausted // 2. Immediate mitigation:
restarted 2 stuck workers // (bought time to investigate) // 3. Communicate:
posted in #incidents channel // with status every 15 minutes // 4. Root cause: a
new query had N+1 problem, // creating 1000 DB connections per request // 5. Fix:
added .lean() and batched the query // 6. Post-mortem: documented + added query
// connection monitoring alert // Total downtime: 18 minutes // Users affected:
~200 // Prevented future: 0 similar incidents since"
```

32 What decision do you regret?

Show honest self-reflection, what you learned, and how you would decide differently.

Example / Code:

```
// Mature answer: // "Early in my career I decided to rebuild // an entire backend
from scratch because // the existing code felt messy. // It took 3 months and the
business // lost 3 months of feature delivery. // I should have asked: // What
specific problems is this solving? // Can we refactor incrementally instead? //
What is the cost to the business? // Now I apply the strangler fig pattern: //
Incrementally replace parts while // keeping the existing system running. // I
always calculate business impact // before proposing large rewrites."
```

33 How do you mentor junior developers?

Show teaching approach, patience, and growth mindset for your team.

Example / Code:

```
// Mentoring approach: // "I believe in 'teach a man to fish': // 1. Pair
programming: sit with them, ask questions // rather than giving answers // 2.
Code reviews: explain WHY, not just what to change // Reference patterns and
principles // 3. Weekly 1:1: discuss blockers, career goals // 4. Challenge
appropriately: assign tasks slightly // beyond their comfort zone with support //
5. Normalize questions: create safe environment // Example: a junior was spending
4 hours on bugs // before asking for help. // I introduced the 30-minute rule: //
try for 30 min, then ask. // Their productivity tripled."
```

34 How do you handle ambiguity?

Show you can work with incomplete information, clarify requirements, and make reasonable assumptions.

Example / Code:

```
// Good answer: // "When requirements are ambiguous I: // 1. List my assumptions
explicitly in writing // 2. Ask specific clarifying questions // (not just 'tell
me more') // 3. Build for the most likely interpretation first // with clear
extension points // 4. Timebox the clarification (don't wait forever) // Example:
// Got ticket: 'make the API faster' // No metrics, no specific endpoint. // I
asked: what endpoint? what's slow? // what is the target metric? // While
waiting, I ran a profiler to find // the three slowest endpoints proactively. //
That data made the follow-up meeting // much more productive."
```

35 How do you meet deadlines under pressure?

Show planning, communication, and pragmatism.

Example / Code:

```
// Practical answer: // "Three habits that help me meet deadlines: // 1. Estimate
with 20% buffer explicitly // (tell stakeholders 5 days for 4 day work) // 2.
Communicate on day 2 if timeline at risk // (never day before deadline) // 3. Cut
scope intelligently: // identify must-have vs nice-to-have early // Example:
2-week sprint, realized day 5 // that notification feature was more complex. //
Told PM immediately with options: // A: delay launch 2 days // B: ship without
email notifications // They chose B. Shipped on time. // Email notifications
shipped next week."
```

36 How do you handle multiple projects?

Show prioritization system, context-switching management, and communication.

Example / Code:

```
// Practical answer: // "I manage multiple projects by: // 1. Explicit priority
ranking with stakeholders // (not implicit or assumed) // 2. Time-blocking:
Monday AM = Project A, // Monday PM = Project B (avoid context switches) // 3.
Weekly status update to all stakeholders: // what I did, what's next, any
blockers // 4. Scope control: push back when both projects // need the same
resource at the same time // Currently managing: // - Core API feature
development (60%) // - Production bug fixes (30%) // - Code review/mentoring
(10%) // This ratio is explicit and agreed with my manager."
```

37 How do you communicate with non-technical stakeholders?

Show translation skills and empathy for different audiences.

Example / Code:

```
// Communication approach: // "I adapt my communication to the audience: // For
PMs: I talk about user impact and timelines // not database indexes and API design
```

```
// For executives: I translate to business metrics: // 'This will reduce churn by
~5%' not // 'This reduces API latency by 300ms' // For designers: I focus on
implementation // constraints and feasibility // Example: // When proposing a
React migration, // I presented: user experience improvement, // estimated dev
velocity increase, // and reduced time-to-market for features. // Not bundle
sizes and render performance. // Got approval in first meeting."
```

38 How do you handle disagreements with tech decisions?

Show professional advocacy, open-mindedness, and ability to commit once decided.

Example / Code:

```
// Good answer: // "When I disagree with a technical decision: // 1. First,
understand fully — ask questions // to see if I am missing context // 2. If still
disagreeing, present my case with: // data, examples, and trade-offs // (not just
opinion) // 3. Escalate to architecture review if needed // 4. Once team decides —
commit fully // even if I disagreed // Example: disagreed on REST vs GraphQL. //
Built a proof-of-concept showing 60% fewer // round trips with GraphQL. // Team
chose REST for simplicity. // I fully implemented and optimized REST. // No
passive resistance."
```

39 How do you ensure code quality?

Show concrete practices, not just vague claims about clean code.

Example / Code:

```
// Quality practices: // 1. Test coverage: 80%+ unit tests, 100% for // critical
paths (payments, auth) // 2. Code review: every PR reviewed before merge. // I
give specific actionable feedback // 3. Static analysis: ESLint + Prettier +
Husky // (pre-commit hooks prevent bad code) // 4. TypeScript: catches errors at
compile time // 5. Integration tests: API contract testing // 6. Performance
profiling: before each major release // 7. Security scans: npm audit in CI
pipeline // I also run a weekly tech debt session // with the team to address
accumulated issues.
```

40 Describe an example of ownership?

Show you go beyond your job description. Take initiative and see things through.

Example / Code:

```
// Ownership example: // "I noticed our deployment process was fragile: // manual
steps, no rollback plan, 2-hour process. // This wasn't my explicit
responsibility. // I spent 3 evenings designing a CI/CD pipeline // using GitHub
Actions + Docker + K8s. // I presented it to the team with a demo. // The team
adopted it, and deployment went from // 2 hours manual to 8 minutes automated. //
Zero deployment incidents in 6 months. // The process is now the company
standard. // I did this because I cared about the team's // quality of life, not
because I was asked."
```

41 Give an example of innovation?

Show you identify problems proactively and create solutions beyond the obvious.

Example / Code:

```
// Innovation example: // "Our support team was spending 2 hours/day // manually
answering the same 20 questions. // No one asked me to solve this. // I built a
FAQ chatbot using Node.js + // simple keyword matching + MongoDB for answers. //
Support queries handled automatically: 60%. // Support team redirected to complex
cases. // I then made it easy for non-devs to add FAQs // via a simple admin
interface. // Cost: 3 days of my time. // Value: 60+ support hours saved monthly."
```

42 What is the most impactful thing you built?

Show business impact, not just technical complexity.

Example / Code:

```
// Impact-focused answer: // "The most impactful thing I built was // an automated loan eligibility checker. // Before: loan officers spent 45 minutes // manually checking each application. // I built: Node.js API + MongoDB + rules engine // that checks 20 criteria in <2 seconds. // Business impact: // - Processing time: 45 min -> 2 seconds // - Capacity: 20/day -> 500/day per officer // - Accuracy: 97% (vs 89% manual) // - Revenue: company onboarded 3 new bank clients // This is the project I'm most proud of because // the business outcome was transformative."
```

43 How do you stay current with technology?

Show active, systematic learning habits.

Example / Code:

```
// Learning system: // 1. Weekly: read JavaScript Weekly, Node.js digest // 2. Monthly: build a small project with new tech // 3. Quarterly: deep dive into one topic // (last quarter: WebAssembly; this quarter: AI APIs) // 4. Daily: LinkedIn content creation forces me // to clarify and research what I share // 5. Conference talks: YouTube (JSConf, NodeConf) // 6. Open source: reading code of tools I use // Recent learning: // Studied React Server Components deeply // Built a Next.js 14 App Router project // Wrote a post explaining it -> 45K impressions // Teaching it forced me to understand it deeply
```

44 How do you give constructive feedback?

Show empathy, specificity, and focus on behavior not person.

Example / Code:

```
// Feedback approach: // "I follow the SBI model: // Situation (specific context) // Behavior (what I observed, not personality) // Impact (effect on team/product) // Example of good feedback: // 'In yesterday's code review (Situation), // the PR had no error handling for edge cases // like null database responses (Behavior). // If deployed, this would cause silent failures // for 5-10% of users (Impact). // Can we add try-catch blocks here?' // Not: // 'Your code is sloppy' (personal, not specific) // 'You should add error handling' (no context) // I also give positive feedback publicly // and critical feedback privately.
```

45 How do you handle a production outage?

Show: stay calm, systematic triage, clear communication, post-mortem culture.

Example / Code:

```
// Incident response process: // 1. Detect: alert fired or user report // 2. Acknowledge: respond in #incidents within 5 min // 3. Assess severity: how many users affected? // 4. Mitigation first: rollback > debug // (restore service, then find root cause) // 5. Communicate: status every 15 min to stakeholders // 6. Root cause analysis: after service restored // 7. Post-mortem: blame-free, focus on process // What happened? Timeline? Why? Prevention? // Key principle: // Restoration before investigation. // Communication before perfection. // Blameless culture enables honest reporting.
```

46 Example of teamwork success?

Show your specific contribution in a team achievement.

Example / Code:

```
// Teamwork example: // "We were building a real-time dashboard // for a client with a 6-week deadline. // 4 developers, tight timeline. // My contribution: // - Designed the WebSocket architecture // - Mentored junior dev on React state management // - Created shared utility library used by everyone // - Unblocked backend dev who was stuck on MongoDB // aggregation pipeline // Team dynamics: // - Daily standups kept us aligned // - Async Slack for most communication // - Pair
```

```
programming for complex features // Result: delivered 3 days early. // Client
extended contract for 6 more months.
```

47 Example of difficult decision?

Show structured decision-making under uncertainty.

Example / Code:

```
// Difficult decision example: // "I had to decide whether to delay launch // by 2
weeks to add proper test coverage, // or launch on schedule without tests. //
Considerations: // - Business: client had marketing campaign planned // -
Technical: untested code risk // - Team: developers knew the code well //
Decision process: // Estimated risk scenarios, // consulted senior dev and PM, //
proposed compromise: launch with // manual QA + monitoring, write tests week 1
post-launch. // Outcome: launched on time, two minor bugs caught // and fixed
within 24 hours. // Tests written as promised in week 1. // I now advocate for
testing throughout development // to avoid this dilemma.
```

48 Example of learning a new skill quickly?

Show fast, structured learning with a real outcome.

Example / Code:

```
// Fast learning example: // "The team needed to integrate Stripe payments. // I
had never used Stripe before. // Day 1: Read official Stripe docs end-to-end (4
hrs) // Day 2: Built sandbox integration, tested all edge cases // Day 3:
Implemented in production with: // - Webhook handling for async payment status //
- Idempotency keys for retry safety // - Proper error codes to frontend // Result:
integration delivered in 3 days, // zero payment failures in first month. // The
structured approach (docs -> sandbox -> production) // is now my template for any
new integration.
```

49 How do you contribute to company culture?

Show you build team culture, not just code.

Example / Code:

```
// Culture contributions: // 1. Knowledge sharing: // Run monthly 'Tech Tuesday'
sessions // to share what the team learned // 2. Code review culture: //
Introduced async code review guidelines // with response SLAs // 3. Onboarding:
// Improved new developer guide reducing // onboarding time from 2 weeks to 3 days
// 4. Recognition: // Regularly call out good work in standups // and Slack
(public praise) // 5. Psychological safety: // Created blameless incident culture
where // people report issues without fear // 6. Fun: // Organized virtual
hackathon that // produced 2 features now in production
```

50 Final pitch — why should we hire you?

Summarize your unique value proposition. Connect your specific experience to their specific needs. Show enthusiasm.

Example / Code:

```
// Compelling final pitch: // "To summarize why I am the right fit: // 1. Relevant
experience: // I have built exactly the type of [product] // you are building — at
[Company] I [achievement] // 2. Problem-solving track record: // I have
consistently [key result] by [approach] // 3. Culture fit: // I value [their
stated values] because [example] // 4. Growth potential: // I learn quickly — I
will add value from week 1 // and grow into [their future needs] // 5. Genuine
enthusiasm: // I am excited about [specific product/problem] // and have already
been thinking about // [specific contribution or idea] // I would love to be part
of what you are building. // What would success look like in the first 90 days?"
```

■ End every interview by asking a thoughtful question. It shows interest and gives you real insight into the role.